技術解説書

OpenMX 4.0 GPU 版 実装解説書

CUDA 実装の詳細と AMD HIP バックエンドの統合
CPU／GPU 共通の ELPA ライブラリ
— GPU ライブラリ・CMake ビルド・実行時制御・検証 —

統合版の参照リビジョン: 2026-10-17、コミット `c0689442`

元の CUDA 調査: 2026-07-23、コミット `9fe8ce13`

バックエンド: NVIDIA CUDA / AMD HIP、ELPA 2026.02.002

改訂日: 2026 年 10 月 17 日(第 4 版)

> **注記:** 本書は第一原理計算コード OpenMX(GPL v3)を NVIDIA・AMD GPU 向けに移植した**非公式版**の技術解説書であり、OpenMX 開発チームによる公式文書ではありません。CUDA の詳細コード例と過去の変更件数は元の調査時点を基準とします。現行の統合・ビルド・検証情報は §9.4・§9.7〜9.8・§11.5 を参照してください。

目次

1. はじめに

2. 全体アーキテクチャ

設計方針 / レイヤ構成 / 有効化とディスパッチ / GPU 化対象マップ / 複数ランク共有 GPU 対策

3. cuBLAS の使い方

使用ルーチン / ハンドルとストリーム / メモリ管理 / ソルバ別 GEMM エンジンの実像 / エラー処理 / 注意点

4. GEMMul8 の使い方

原理と出自 / 統合アーキテクチャ / 精度パラメータ / ワークスペース管理 / 複素対応 / ビルド / 注意点

5. cuSOLVER(gpusolver 層)の使い方

層の構成と命名意図 / 使用 API / 一般化固有値問題 / devInfo 処理 / 適応並列度制御 / ELPA フォールバック / デバイス割当 / 注意点

6. OpenACC と生 CUDA の使い分け

データ常駐の 3 パターン / host_data 連携 / set_hamiltonian_gpu.cu / NVHPC 固有の注意

7. 計算経路別の GPU 実装詳細

Band / Cluster / DC / DC-LNO / Krylov / 行列要素構築 / VNA 射影子 / 密度グリッド / 全エネルギー / 力 / 混合スキーム / 派生ソルバ群

8. GPU 化以外の変更点

OpenMP バグ修正 / メモリ管理の堅牢化 / その他の修正

9. ビルドシステム

CUDA ツールチェーン / リンク構成 / 共通 ELPA / Makefile / CMake による CUDA・HIP ビルド / 回帰試験

10. 実行時制御リファレンス

入力キーワード / 環境変数一覧 / 標準出力の GPU 診断行 / チューニング指針

11. 制約・注意点の総まとめ

付録 A. 変更ファイル一覧(2026 年 7 月、47 ファイル)

付録 B. 新規追加ファイル一覧(2026 年 7 月)

付録 C. 開発履歴(2026 年 7 月、91 コミット)

参考文献

1. はじめに

1.1 本書の目的と対象読者

本書は、密度汎関数理論(DFT)に基づく第一原理計算コード **OpenMX 4.0** を NVIDIA CUDA・AMD HIP 向けに 移植した「OpenMX 4.0 GPU 版」の解説書です。リポジトリのソースに基づき、CUDA 実装の詳細、統合された HIP バックエンド、CPU／GPU 共通の ELPA、ビルドと実行時の設定を説明します。第 3〜7 章は CUDA 固有 API の詳細を扱い、HIP の実装と相違点は §11.5 にまとめます。想定読者は次のとおりです。

- OpenMX を GPU で高速に実行したいユーザー(第 2・10・11 章が中心)
- 本実装を理解・改修したい開発者(第 3〜9 章)
- CUDA と HIP の両バックエンドを保守する開発者(§9.7・§11.5)

前提知識として、DFT・LCAO 法の初歩、C 言語、MPI、CUDA プログラミングの基礎を仮定します。

1.2 OpenMX と本移植の位置づけ

OpenMX(Open source package for Material eXplorer)は、局在擬原子軌道(LCAO)基底とノルム保存擬ポテンシャルを用いた DFT コードで、クラスタ計算・バンド計算に加え、発散重畳(DC)法・DC-LNO 法・Krylov 部分空間法などの O(N) 法、非平衡グリーン関数(NEGF)輸送計算などを備えます。ライセンスは GPL v3 です。

本 GPU 版は OpenMX 4.0 の公式ソースを出発点とします。2026 年 7 月の元の調査は **91 コミット**を対象としました。初期の開発(〜第 2 版の対象であった 51 コミット)では SCF 計算で支配的な「密行列の固有値問題」と「行列積(GEMM)」を GPU 化し、その後の 40 コミットで対象が大きく広がりました。現在は、行列要素構築(Set_Hamiltonian の格子積分・Set_Nonlocal・Set_ProExpn_VNA)、密度グリッド構築(Set_Density_Grid)、力の主要トレース(#2/#3/#4/#4B/#5/HNL)、全エネルギーの格子縮約、電荷/DM 混合(RMM-DIISH の GPU 化ほか全スキームの高速パス)までが GPU またはそれに準ずる高速経路で実行されます。疎行列処理、通信、入出力などはオリジナルの構造を保っています。

1.3 変更規模の概要

以下の件数と付録 A〜C は 7 月 23 日時点の CUDA 調査結果です。その後の HIP バックエンド統合、CMake によるビルド分離、ELPA 共通化を含む現在のツリー全体の件数ではありません。

項目	値	備考
変更されたソースファイル	47 ファイル(+ `makefile` → `Makefile` の改名・全面改修)	削除されたソースは `my_cublasDgemm.c` / `my_cublasZgemm.c` の 2 件(休眠ラッパの撤去)
変更行数(生の diff)	約 +47,000 / −21,200 行	全面再インデント(`Force.c` 等)を含む。`Force.c` 単体で +15,123/−11,241
新規ソースファイル	14 ファイル、計約 4,000 行	gpusolver ラッパ、GEMMul8 ブリッジ、GPU 用エネルギー/密度グリッドカーネル、Set_Hamiltonian 用 CUDA カーネルなど
新規ビルドファイル	3 ファイル	`Makefile` / `Makefile.bunshiken` / `makefile.cpu_intel.bak` (原本の同一保存)
third_party(git submodule)	2 件	GEMMul8(コミット固定)、FFTW 3.3.11(自前ビルド)。どちらも `make` が自動取得・自動ビルド

1.4 対象環境

- **CUDA:** NVIDIA HPC SDK(nvc/OpenACC、nvfortran、nvcc)。現在の主 Makefile は NVHPC 26.9 / CUDA 13.4 が既定で、パスを上書きできる。元の調査環境は NVHPC 26.3 / CUDA 13.1。

- **HIP:** ROCm clang/clang++、hipcc、flang、OpenMP target offload、互換 MPI、AOCL。MI300A と Radeon はデバイス別の実行方針を持つ(§11.5)。
- **並列モデル:** MPI プロセス並列。ノード内 local rank によるラウンドロビンで GPU を割当て、**複数ランクが 1 GPU を共有する**ことを前提に設計。共有状況は物理 GPU の UUID 単位で検出され、必要 VRAM の見積り・実予約に基づいて並列度が自動調整される(§2.5、§5.5)
- **OpenMP:** MPI/OpenMP 混合実行に対応。`-nt` に合わせて各ランクのコアを確保する。CUDA の一部経路(`Set_OLP_Kin` の任意の動径積分オフロードなど)は 1 スレッドを要求する(§11.1)。

1.5 表記法

- ソース位置は `ファイル名:行番号` で示す(すべて `source/` ディレクトリ基準、2026-07-23 時点)。
- `GPU` 既定で GPU 実行、`条件付き` 環境変数などによるオプトイン/メモリ状況依存、`CPU` CPU 実行(意図的に GPU 化しない)、`休眠` 実装済みだが現行経路では呼ばれないコード。
- 「オリジナル」は OpenMX 4.0 公式配布のソースを指す。

> **本書の読み方:** ライブラリの使い方だけを知りたい場合は第 3〜5 章、運用パラメータは第 10 章、既知の落とし穴は第 11 章にまとまっています。第 7 章は計算経路(ソルバ)別の縦割り解説で、第 3〜6 章(ライブラリ・技法別の横割り)と相互参照になっています。第 2 版(2026-07-05)からの主な差分は、行列要素構築・密度グリッド・力・混合の GPU 化(§7.7〜7.12)、全経路の適応並列度制御と ELPA フォールバック(§5.5〜5.6)、GPU フェーズ間のメモリ協調機構(§2.5)です。

2. 全体アーキテクチャ

2.1 設計方針

1. **実行時切替・CPU 経路の完全温存** — GPU 化は `#ifdef` による一括置換ではなく、既存コードへの**分岐追加**で実現している。既定で有効(`scf.eigen.lib` の既定値が `gpusolver`)であり、入力ファイルの 1 行 `scf.eigen.lib elpa2`(または `elpa1` / `lapack`)を書けばオリジナルと同じ ELPA/ScaLAPACK/LAPACK 経路が走る。ゲートは実行時フラグ `scf_eigen_lib_flag == GPUSOLVER`(enum 値 3)の一点に集約されている。
2. **大規模な密行列問題のみ GPU 化** — 行列次元がしきい値未満の場合は転送コストが勝るため CPU にフォールバックする。しきい値は経路ごとに実測して決めた既定値を持つ:バンド Col/NonCol は 800(n / 2n と比較。CPU フォールバックが k 点ごとにワールド内直列 ELPA を 2 回解く構造のため GPU が早く優位になる)、クラスタ Col/NonCol は 400 (1 ランクの root-dense 解が多ランク ELPA に測定した全サイズで勝った)、DC は Col 400 / NonCol 300 (部分クラスタの NUM / 2·NUM と比較)、DC-LNO は Col 800 / NonCol 400(LNO 縮約後の次元。コリニアは プロキシ的なオーバーヘッドがあるため高め)、Krylov は 300(射影次元 Msize3 と比較)。その他の経路は `GPU_CPU_SWITCH_NUM = 1000`(`openmx_common.h:4135`)のまま。各しきい値は 経路ごとの環境変数で調整できる(§ 10)。
3. **MPI ランク単位の GPU 利用と「共有」前提の設計** — ノード内 local rank % GPU 数のラウンドロビン割当。1 GPU を複数ランクが共有する状況を正面から想定し、物理 GPU の **UUID 単位でデバイス共有ランク群を検出**したうえで、VRAM 見積り/実予約に基づく適応並列度制御・実行時 ELPA フォールバック・失敗時リトライ・GPU フェーズ間のメモリ協調(需要公開レジストリ)という多層の制御を持つ(§ 2.5)。
4. **デバイスに応じた GEMM 選択** — GeForce は INT8 Tensor Core による GEMMul8 FP64 エミュレーション (Ozaki Scheme II)が既定。A100/H100 は基本的に native cuBLAS FP64 を選び、A100 のクラスター前方変換には サイズに応じた例外がある。明示設定で上書きでき、ワークスペース不足時も native FP64 に復旧する。HIP は MI300A/Radeon 別の方針を持つ(§ 11.5)。
5. **対角化は cuSOLVER の 64-bit generic API** — `cusolverDnXsyevdx`(部分固有値)を中心に使い、一般化固有値問題 $Hc = \varepsilon Sc$ は S の固有分解によるレイディン(対称)直交化を GPU 上で自前実装する(第 5 章)。固有値のみ必要な場面では `jobz=NOVECTOR` で解く。
6. **ベンダー中立の命名(gpusolver)** — 両ビルドでラッパ層の識別子と入力キーワードに `gpusolver` を使う。CMake がビルド時に CUDA または HIP のソースを選ぶ(§ 9.7)。cuSOLVER は CUDA 固有の実装詳細であり、HIP は hipSOLVER と互換ヘルパを使う(§ 11.5)。現在の MAGMA は複数 GPU による容量復旧と `gpusolver3` の強制実行に使い、CUDA では任意、HIP では依存物としてビルドする。
7. **数値再現性への配慮** — GPU 経路は原則として「CPU ループと同一の加算順序」を保つように書かれている: 密度行列構築や格子トレースの縮約は `#pragma acc loop seq` または要素単位の逐次累積、力の GPU バッチは float 積 → double 加算のキャストの位置まで CPU を再現、混合の高速パスはポインタ回転で ビット単位の同値性を保証する。GEMMul8 自体も bitwise 再現性を持つ(一方で atomic 加算による順序非決定箇所も存在する。§ 11 参照)。
8. **フォールバックは「静かに正しく」** — VRAM が足りない場合、各フェーズは事前見積り/実予約で発動可否を判定し、不可なら CPU/ELPA の従来経路へ落ちる。通知はノードあたり 1 回(または値が変わったときだけ)に抑制され、結果の正しさは変わらない。abort するのは「回復不能なエラー」と「明示強制(環境変数)との矛盾」だけに縮小された。

2.2 レイヤ構成

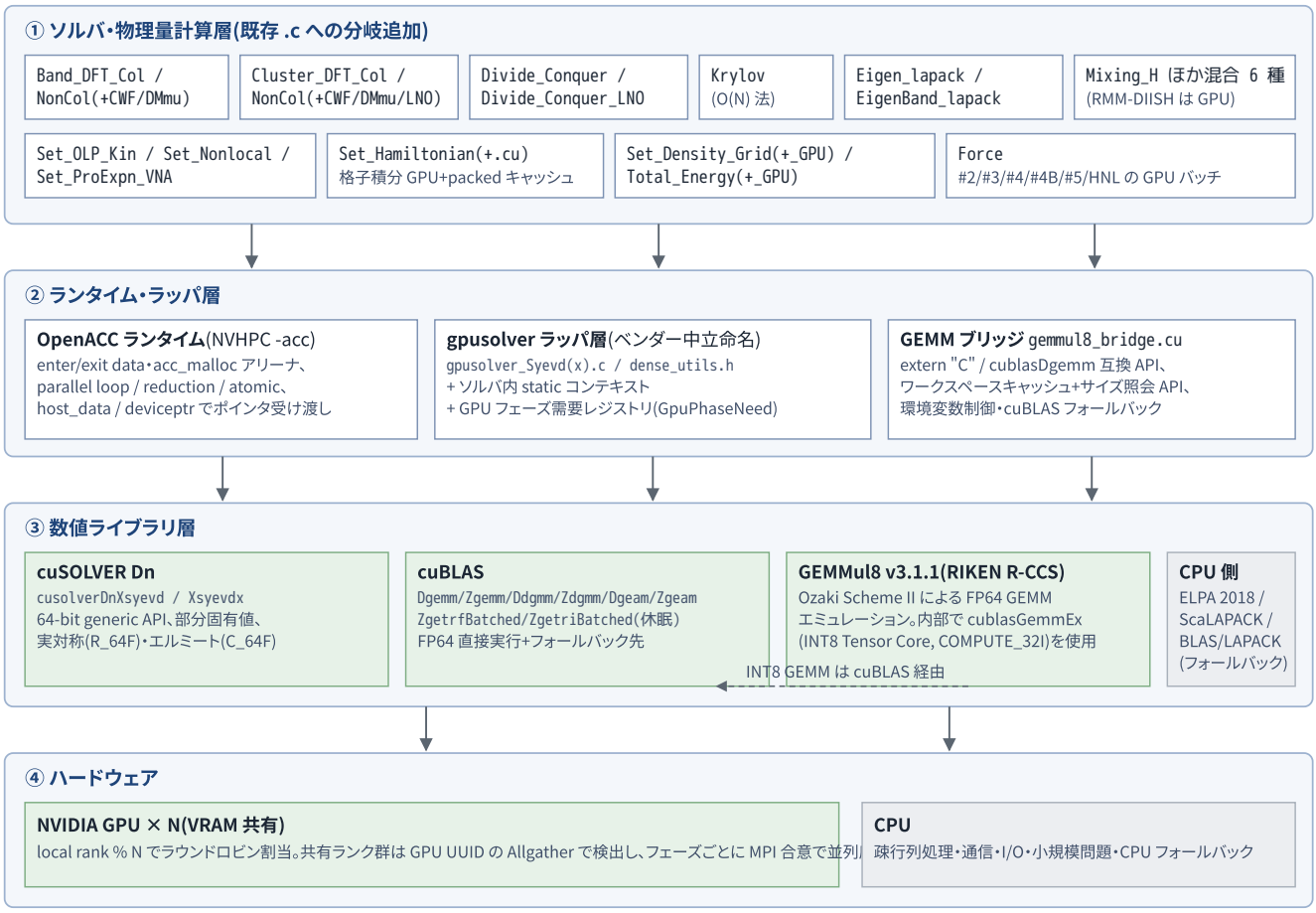


図 1: OpenMX 4.0 GPU 版のレイヤ構成。緑は GPU 実行、灰は CPU 実行。

GPU 実行の形態は 3 種類が併用されている。

1. **OpenACC ディレクティブ** — 格子積分・縮約・行列組立などのループ並列。データ常駐は `enter data` / `exit data` に加え、力計算・混合・クラスタ系では `acc_malloc` / `cudaMalloc` による**単一確保のアリーナ**+ `deviceptr` / `acc_map_data` が主流になった(第 6 章)。
2. **ソルバ内 static コンテキスト+生 CUDA API** — `cudaMalloc` による永続デバイスバッファと cuBLAS/cuSOLVER ハンドルを ファイルスコープの構造体に保持し、ライブラリを直接呼ぶ(`BandColGpuSolverCtx` など。第 3・5 章)。
3. **extern "C" ブリッジ経由の CUDA C++** — GEMMul8(第 4 章)と、Set_Hamiltonian の共有メモリタイル型 行列要素カーネル `set_hamiltonian_gpu.cu` (§ 7.7)。C コードから C++/CUDA を呼ぶための単一翻訳単位方式。

2.3 有効化の流れと SCF 内の GPU 化ポイント

GPU 経路が有効になるまでの流れは次のとおり。

1. **入力解釈**(`Input_std.c`): `scf.eigen.lib gpusolver` (キーワード省略時の既定値)で `scf_eigen_lib_flag = GPUSOLVER` 。旧名 `cusolver` は非推奨エイリアスとして警告付きで受理される。ユーザー指定値は `scf_eigen_lib_flag_input` に保存。
2. **デバイス初期化**(`DFT.c` の `DFT_GPU_DeviceInit()`): `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` でノード内 local rank を求め、`cudaSetDevice` と `acc_set_device_num` を**対で**設定する。CUDA/OpenACC デバイスが見つからなければ警告を出して `scf_eigen_lib_flag = ELPA2` に自動降格する。

3. **各フェーズ入口での条件判定**: `GPUSOLVER フラグ && サイズ条件 && メモリ条件(見積り or 実予約)` を満たすときだけ GPU 経路に入り、満たさなければ従来経路(ELPA2/CPU)がそのまま走る。判定はデバイス共有ランク群または `mpi_comm_level1` 全体の MPI 合意(`Allreduce(MIN)`)で全ランク一致に揃えられる。

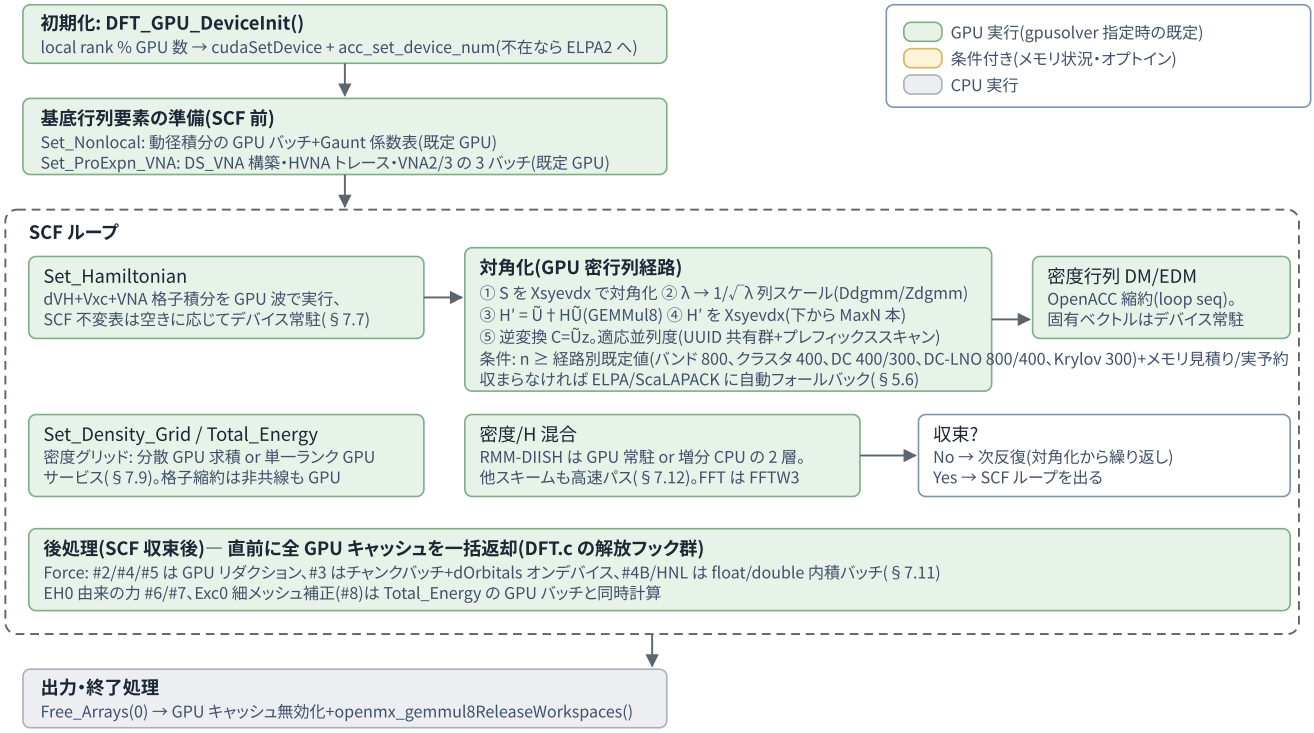


図 2: SCF 計算 1 サイクルの流れと GPU 化ポイント(コリニア・バンド/クラスタ計算の場合)。

2.4 GPU 化対象マップ

計算経路(`scf.EigenvalueSolver`)別・補助計算別の GPU 化状況を表 2-1 に示す。詳細は第 7 章。

経路 / 計算	主担当ファイル	GPU 化内容	発動条件
クラスタ(Col)	`Cluster_DFT_Col.c`	root-dense 対角化一式(S 対角化 → レイディン直交化 → H′ 対角化 → 逆変換)+ DM/EDM の GPU 蓄積。固有ベクトルはデバイス常駐のまま DM へ。2 スピンが同居できない場合はスピンワールド 0 のオーナーが両スピンを直列に解く	GPUSOLVER かつ n ≥ 400(`OPENMX_CLUSTER_GPU_SWITCH_NUM`)かつ **実予約成功**(不成立なら ELPA へ)
クラスタ(NonCol)	`Cluster_DFT_NonCol.c`	単一 cudaMalloc アリーナ上の root-dense 対角化(Dsyevx/Zheevx)+ U † HU(GEMMul8 × 12)+ DM/EDM 縮約。S/Ss2・疎→密 index・cublas ハンドル・zheevdx ワークスペースを反復間キャッシュ	GPUSOLVER かつ 2n ≥ 400(`OPENMX_CLUSTER_NONCOL_GPU_SWITCH_NUM`)かつアリーナ予約成功(不成立なら ELPA へ)
バンド(Col)	`Band_DFT_Col.c`	k 点ごとの密行列対角化一式をデバイス常駐で実行。GEMM は GEMMul8 Zgemm。S 変換行列をキャッシュ。第 1 パスは NOVECTOR 対角化。UUID 共有群+降順プレフィックススキャンの適応並列度	GPUSOLVER かつ n ≥ 800(`OPENMX_BAND_GPU_SWITCH_NUM`)かつ 1 ターン分の VRAM 見積りが成立(不成立なら ELPA へ)
バンド(NonCol)	`Band_DFT_NonCol.c`	OpenACC 常駐型 root-dense 経路。固有値のみ/固有ベクトル/DM の 3 ステージ別 VRAM 見積りで並列度を独立に決定	GPUSOLVER かつ 2n ≥ 800(`OPENMX_BAND_NONCOL_GPU_SWITCH_NUM`)かつ見積り成立(不成立なら ELPA へ)

経路 / 計算	主担当ファイル	GPU 化内容	発動条件
DC 法(O(N))	`Divide_Conquer.c`	原子ごとの部分クラスタ行列を Xsyevdx で対角化。GEMMul8→cuBLAS→CPU の多段フォールバック。S12 パネルのデバイス常駐キャッシュ(S-cache)+ cublasDgeam デバイス転置	GPUSOLVER かつコリニアは NUM ≥ 400(`OPENMX_DC_GPU_THRESHOLD`)、非コリニアは 2·NUM ≥ 300(`OPENMX_DC_NONCOL_GPU_THRESHOLD`)
DC-LNO 法	`Divide_Conquer_LNO.c`	direct per-rank dispatch。**非コリニアも GPU 化済み**(スピン倍化変換+GEMMul8 Zgemm × 3)。デバイス共有ランク群の合算メモリ判定で CPU フォールバック	GPUSOLVER かつコリニアは局所次元 ≥ 800(`OPENMX_DCLNO_GPU_THRESHOLD`)、非コリニアはスピノル次元 ≥ 400(`OPENMX_DCLNO_NONCOL_GPU_THRESHOLD`)
Krylov 法(O(N))	`Krylov.c`	原子ごとの射影解パイプラインを 1 回のデバイス訪問に融合(GEMM×3 + Xsyevd)。Krylov 基底のデバイス常駐 KU キャッシュ	GPUSOLVER かつ 3 GEMM とも $m \cdot n \cdot k \geq 5\times10^7$(対角化は n ≥ 1000)
汎用固有値ルーチン	`Eigen_lapack.c` `EigenBand_lapack.c`	n ≥ 1000 で gpusolver_Syevdx(_Complex) へ分岐。これらを呼ぶ全 20 以上のファイルが自動的に GPU 化の恩恵を受ける	GPUSOLVER かつ n ≥ 1000
分極・DMmu・CWF・LNO 系	11 ファイル	ELPA2 分岐の手前に定型の GPUSOLVER 分岐(dense_utils 経由の Dsyevx/Zheevx)を挿入	GPUSOLVER かつ n ≥ 1000 かつ na_rows==n 等
行列要素構築	`Set_Hamiltonian.c` `set_hamiltonian_gpu.cu` `Set_Nonlocal.c` `Set_OLP_Kin.c`	Set_Hamiltonian の dVH+Vxc+VNA 格子積分は**既定で GPU**(共有メモリタイルの CUDA カーネル+適応波スケジューリング+SCF 不変表の常駐キャッシュ)。Set_Nonlocal は動径積分のデバイス生成球ベッセル+バッチ縮約。Set_OLP_Kin はオプトイン	GPUSOLVER(+メモリ判定、§7.7)
VNA 射影子	`Set_ProExpn_VNA.c`	DS_VNA 構築・HVNA トレース・VNA2/VNA3 の 3 バッチを GPU 実行(球ベッセルのデバイス漸化、Gaunt/変換表のホスト表引き化)	GPUSOLVER(+バッチ別メモリ判定)
密度グリッド	`Set_Density_Grid.c` `Set_Density_Grid_GPU.c`	分散 GPU 求積(各ランクが自原子を積分)→ 単一ランク GPU サービス(オーナーが全域を構築し scatter)→ CPU の 3 段フォールバック	GPUSOLVER かつ Cluster/Band かつ Cnt_switch==0
全エネルギー	`Total_Energy.c` `Total_Energy_GPU.c`	EXC/EH1・双極子・CWF-Dc・EH0 二中心項・Exc0 細メッシュ補正の格子縮約を OpenACC 実行。**非共線(NC)でも有効**	GPUSOLVER(`OPENMX_TOTALENERGY_GPU=0` で無効化)
力	`Force.c`	#2/#4/#5 の GPU リダクション、#3 のチャンクバッチ(dOrbitals オンデバイス再計算つき)、**#4B・HNL の GPU 内積バッチ**(旧版で CPU 維持とされていた #4B も GPU 化)。アリーナ+ピア待ち機構で共有デバイスのメモリ競合に耐性	GPUSOLVER(+項別メモリ判定、§7.11)
電荷/DM 混合	`Mixing_H.c` ほか 5 ファイル	RMM-DIISH は GPU 常駐 Gram(Tier A)/増分 CPU(Tier B)の 2 層。RMM-DIIS/DIISK/DIISV/GR-Pulay/Kerker はバッチ Gram・増分 Gram・ポインタ回転シフトの CPU 高速パス	GPUSOLVER(`OPENMX_MIX_FAST=0` で無効化)
NEGF・複素逆行列	`Lapack_LU_inverse.c`	休眠 cublasZgetrf/ZgetriBatched による GPU 版 LU 逆行列を実装済みだが呼び出しはコメントアウト	—(将来の布石)

表 2-1: GPU 化対象マップ

2.5 複数ランク共有 GPU への対策(概観)

本実装の特徴は、「16 GB 級の GPU 1 台を 8〜36 MPI ランクで共有する」ような構成を明示的に想定した多層防御にある。第 2 版以降、この防御は「固定ノブによる直列化」から「**実測に基づく適応制御+実行時フォールバック**」へ全面的に進化した。

機構	内容	実装箇所
デバイス共有ランク群の検出	`MPI_Comm_split_type(SHARED)` + `cudaGetDeviceProperties` の **GPU UUID** を Allgather し、同一物理 GPU を共有するランクのコミュニケータを作る。空き VRAM は `acc_clear_freelists()` でプールを返却してから `cudaMemGetInfo` を全員で測り、MIN で共有	`Band_DFT_Col.c`、`Band_DFT_NonCol.c`、`Set_Hamiltonian.c`、`Mixing_H.c`
適応並列度(プレフィックススキャン)	各ランクの必要 VRAM を Allgather して降順ソートし、「上位 c ランクの合計+予約が空きに収まる最大の c」を 1 本のスキャンで決定。全ランクが同時に収まる場合はバリアを畳み込み、収まらなければ波(ターン)に分割。初期ウェーブはデバイス共有ランク数(旧版の固定はしご 32→…→1 は撤廃)	`BandCol_AutoGpuTurnLimit`、`BandNonCol_AutoGpuTurnLimit`、`Set_Hamiltonian_CreateGpuTurnPlan`
実予約プローブ+ELPA フォールバック	クラスタ経路は「見積り」ではなく**実際に cudaMalloc を試して**可否を判定(8 回上限のリトライ、失敗時は全解放)。バンド経路は 1 ターン分の見積りで判定。どちらも不成立なら ELPA/ScaLAPACK に自動フォールバックし、判定は MPI_Allreduce(MIN) で全ランク一致	`ClusterCol_GpuDiagFits`、`ClusterNonCol_GpuDiagFits`、`BandCol_GpuDenseFits`、`BandNonCol_GpuDiagFits`(§ 5.6)
GPU フェーズ需要レジストリ	対角化・密度グリッドが「このデバイスでフル並列時に必要な総量」を `OpenMX_GpuPhaseNeed_Register()` で公開し、Set_Hamiltonian の常駐キャッシュが**その分を空けて**から入場・不足時は自主退去する(フェーズ間のメモリ協調)	`openmx_common.c` のレジストリ、`Set_Hamiltonian.c`(§ 7.7)
力バッチのアリーナ+ピア待ち	力計算の GPU バッチは全確保を 512 B 整列の単一アリーナにまとめ、必須確保は「50 ms 刻みで最大 180 秒ピアの解放を待つ」、任意確保(リダクションバッファ)は失敗時その場でホスト計算。**abort し得るデバイス確保は力ステージに存在しない**	`Force_gpu_arena_wait/try`(§ 7.11)
SCF フェーズ間の一括返却	毎 SCF サイクルの力計算直前に、Set_Hamiltonian 常駐表・DC S-cache・Krylov KU キャッシュ・クラスタソルバコンテキスト・混合履歴アリーナをすべて解放し、力バッチが真の空き VRAM を測れるようにする	`DFT.c:2312-2319` の解放フック群
アロケーション失敗リトライ	`wait_cudafunc` マクロが CUDA/cuBLAS/cuSOLVER 呼び出しを成功までランダム待ち(≤10 μs)付きで再試行し、失敗ごとにエラーラッチを掃除。恒久的不足があり得る確保は有界リトライの `TryDeviceMalloc` 系へ移行済み	`openmx_common.h:4148-4162`
ワークスペースのターン毎解放	約 1 GiB の GEMMul8 ワークスペースをターン/ソルブ終了ごとに返却(Band・Cluster・DC-LNO、既定有効)	`OPENMX_*_GEMMUL8_TURN_RELEASE` 系

> **休眠コードの扱い:** 本書では、定義は存在するが現行経路では呼ばれないコード(`gpusolver_Syevd`、`LU_Zinverse_GPU`、一部の GEMMul8 ヘルパなど)を `休眠` と明示して区別する。一覧は §11.7 を参照。なお第 2 版で休眠扱いだった `openmx_cuda_kernels.cu`(未結線の生 CUDA カーネル集)は ソースツリーから削除され、代わりに**結線済み**の `set_hamiltonian_gpu.cu` が加わった。

3. cuBLAS の使い方

3.1 使用ルーチンと役割

cuBLAS は本実装で 3 つの役割を持つ: (1) 対角スケーリング(`Ddgmm` / `Zdgmm`)や デバイス内転置(`Dgeam` / `Zgeam`)など GEMM 以外の密行列演算の直接実行、(2) GEMMul8 が使えないときの FP64 フォールバック先、(3) GEMMul8 内部の INT8 Tensor Core GEMM(`cublasGemmEx`)の実行基盤。かつて一部ソルバに残っていた `cublasDgemm/Zgemm/Dsymm` の直接呼び出しは、コミット `66ce1e62` (Use GEMMul8 for remaining GPU GEMM paths)で**すべて GEMMul8 経由に統一**された。使用ルーチンの全体像を表 3-1 に示す。

cuBLAS ルーチン	用途	主な使用箇所
`cublasDgemm` / `cublasZgemm`	GEMMul8 ブリッジ内の FP64 フォールバック先、および DC 法の多段フォールバック 2 段目(直接呼び出しはこの 2 箇所のみ)	`gemmul8_bridge.cu` 、`Divide_Conquer.c`
`cublasDdgmm` / `cublasZdgmm`	レイディン直交化の $1/\sqrt{\lambda}$ 列スケーリング(対角行列との積)。DC-LNO 非コリニアではスピン倍化変換行列 $T = \mathrm{diag}(U\cdot s^{-1/2}, U\cdot s^{-1/2})$ の左上ブロック生成にも使用	`Cluster_DFT_Col.c` 、`Band_DFT_Col.c` 、`Divide_Conquer_LNO.c`
`cublasDgeam` / `cublasZgeam`	デバイス内転置(純粋なデータ移動なので後段への入力はビット一致)。DC 法はホスト側 $O(n^2)$ ストライド転置を `DC_GpuSolver_DeviceTranspose` (`Divide_Conquer.c:1175`)に置換。DC-LNO 非コリニアは H' の共役行列生成に使用	`Divide_Conquer.c` 、`Divide_Conquer_LNO.c`
`cublasZgetrfBatched` / `cublasZgetriBatched`	休眠 複素 LU 分解・逆行列(batch=1)。NEGF 表面グリーン関数向けの布石(§ 5.9)	`Lapack_LU_inverse.c:384,400`
`cublasGemmEx` (INT8, COMPUTE_32I)	GEMMul8 内部の法別 INT8 行列積(OpenMX からは直接呼ばない)	`third_party/GEMMul8/src/oz2/common/matmult.hpp`

表 3-1: 使用 cuBLAS ルーチン一覧

3.2 ハンドルと CUDA ストリームのライフサイクル

現行実装の主経路では、**ソルバごとのファイルスコープ static コンテキスト**が cuBLAS/cuSOLVER ハンドルを 遅延初期化で 1 個ずつ保持する(MPI プロセスあたりソルバ種別ごとに 1 個)。デバイスが変わった場合のみ破棄・再生成される。かつて反復ごとに create/destroy していた箇所も、現在はキャッシュに統一されている (例: 非コリニアクラスタは反復あたり 13 回のハンドル生成を `ClusterNonCol_CublasHandle()` の デバイス別キャッシュに置換、`Cluster_DFT_NonCol.c:762` 。DC 非コリニア/DC-LNO 非コリニアの毎原子対角化は `gpusolver_Syevdx_Complex_openacc_cached` がハンドル・ストリーム・ワークスペースを保持、§ 5.2)。

Cluster_DFT_Col.c — コンテキストの遅延初期化(典型形)

```
wait_cudafunc(cudaGetDevice(&current_device));

if (ctx->initialized && ctx->device_id == current_device) return;
if (ctx->initialized) ClusterCol_GpuSolver_Destroy();

wait_cudafunc(cudaStreamCreateWithFlags(&ctx->stream, cudaStreamNonBlocking));
wait_cudafunc(cublasCreate(&ctx->cublas));
wait_cudafunc(cusolverDnCreate(&ctx->gpusolver));
wait_cudafunc(cublasSetStream(ctx->cublas, ctx->stream));
wait_cudafunc(cusolverDnSetStream(ctx->gpusolver, ctx->stream));

ctx->initialized = 1;
ctx->device_id   = current_device;
```

ストリームの扱いはソルバ間で**非対称**である点に注意が必要である。

- **Cluster_DFT_Col / Cluster_DFT_NonCol / Band_DFT_NonCol(DM 用)/ Krylov / DC / DC-LNO:** 専用の non-blocking ストリームを生成し、`cublasSetStream` / `cusolverDnSetStream` で束ね、転送は `cudaMemcpyAsync` + 要所の `cudaStreamSynchronize` で順序を保証する。
- **Band_DFT_Col:** ソルバコンテキスト構造体に stream メンバがなく、`cublasSetStream` も呼ばれない。cuBLAS はデフォルトストリームで動き、転送も同期 `cudaMemcpy` (syevdx ワークスペース照会専用の一時ハンドルのみ一時ストリームを使う)。
- **OpenACC との順序制御:** OpenACC の既定キューとライブラリ用ストリームは束ねていない(`acc_get_cuda_stream` / `async` 節は全コードで不使用)。OpenACC 管理データをライブラリに渡す直前に `#pragma acc wait`、直後に `cudaStreamSynchronize` を置いて明示同期する。

> **注意:** ソルバ間でストリーム前提が異なるため、あるソルバのヘルパ関数を別ソルバに移植する際に 同期規約を混ぜると競合の恐れがある。各コンテキストのハンドルは**プログラム正常終了時に明示破棄されない**が、デバイス常駐の行列バッファ・ワークスペース・キャッシュは、コミット `ae3dd189` 以降 **毎 SCF サイクルの力計算直前に一括返却される**ようになった(`DFT.c:2312-2319` の解放フック群: `Set_Hamiltonian_Release_OpenACC_DeviceCache` / `Divide_Conquer_Release_GPU_SCache` / `Krylov_Release_GPU_KUCache` / `Cluster_DFT_Col_Release_GPU_Solver` / `Cluster_DFT_NonCol_Release_GPU_Solver` / `Mixing_H_Release_GPU`)。これにより力計算の GPU バッチが「真の空き VRAM」を測って発動可否を判断できる。

3.3 デバイスメモリ管理

- **grow-only の永続バッファ:** `if (n <= ctx->matrix_dim) return;` の形で、必要サイズが既存以下なら再利用し、不足時のみ free→malloc する。cuSOLVER ワークスペースも同様。
- **有界リトライの導入:** 必須でない確保、および失敗時に CPU/ELPA フォールバックが用意されている確保は、無限リトライの `wait_cudafunc` ではなく **8 回上限のリトライ付き `TryDeviceMalloc` 系ヘルパ** (`ClusterCol_TryDeviceMalloc`、`ClusterNonCol_TryDeviceMalloc`、`MixH_TryDeviceMalloc` など)を使う。恒久的にメモリが足りない場合に「無言のフリーズ」ではなく失敗を返してフォールバックできる(コミット `ae3dd189` 以降の設計。§ 5.6)。
- **単一 cudaMalloc のアリーナ:** 非コリニアクラスタは対角化サイクル全体のバッファ群を 1 回の `cudaMalloc` で確保し(512 バイト整列の区画割り)、`acc_map_data` で OpenACC 側に見せる(`ClusterNonCol_DenseArena_TryEnsure`、`Cluster_DFT_NonCol.c:1613`)。予約が通れば**サイクル途中で他フェーズがデバイスを埋めても GO 判定が無効化されない**。DC 法の S キャッシュ、Krylov の KU キャッシュ、Mixing_H の履歴リングも同じアリーナ方式。
- **pinned メモリ:** ほぼ不使用。唯一 DC 法がホスト側に `cudaMallocHost` を使う。
- **共有 GPU への配慮:** Band 系の SCF 反復間バッファ常駐は既定 OFF(`OPENMX_BAND_GPU_PERSISTENT`)。コード内コメントは「16 GB を 8 ランクで共有すると GEMMul8 が FP64 フォールバックに追い込まれ、反復あたり 0.2 秒の節約より高くつ

く」と明記する。一方 Set_Hamiltonian の SCF 不変テーブルは、空き VRAM と他フェーズの公開需要を見て**入るときだけ**常駐する(§7.7)。

3.4 ソルバ別 GEMM エンジンの実像

コミット `66ce1e62` 以降、**GPU 上で実行される GEMM は全ソルバとも GEMMul8 ブリッジ経由に統一**されている (cuBLAS FP64 が使われるのはブリッジ内フォールバックと DC 法の多段フォールバックのみ)。ただし**定義だけ存在して呼ばれないヘルパが複数残る**ため、grep の見かけに惑わされないよう、実際に呼ばれる経路(ライブ経路)を表 3-2 に整理する。

ソルバ	ライブ経路の GEMM エンジン	備考
Band_DFT_Col	**GEMMul8**(`openmx_gemmul8Zgemm` × 3、三重積変換と逆変換)	第 1 パス(all_knum≠1)は固有値のみ必要なため `jobz=NOVECTOR` で対角化し逆変換 GEMM を省く(§7.1)
Band_DFT_NonCol	**GEMMul8**(`BandNonCol_GEMMul8Zgemm_OpenACC` 経由 × 6)	永続ワークスペースの cublas ハンドル+ストリームを使用
Cluster_DFT_Col	**GEMMul8**(`ClusterCol_GEMMul8Dgemm_Device`。H′ 構成 × 2 + 逆変換 × 1)	DM/EDM 蓄積は GEMM ではなく OpenACC `deviceptr` カーネル(§7.3)
Cluster_DFT_NonCol	**GEMMul8**(D/Z 合計 12 回の U†HU 変換 + 逆変換)	逆変換は明示 leading dimension 版 `ClusterNonCol_GEMMul8ZgemmLd_OpenACC` で m=MaxN に縮小(`f5de630e`)
Divide_Conquer(DC)	**GEMMul8 → cuBLAS → CPU** の多段(`DC_GpuSolver_TryGpuDgemm`)	GEMMul8 失敗時は以後そのランクで GEMMul8 を恒久無効化。承認済み GEMM 形状以下では preflight を省略(`6f7f31b2`)
Divide_Conquer_LNO(Col)	**GEMMul8**(`openmx_gemmul8Dgemm` × 3)	—
Divide_Conquer_LNO(NonCol)	**GEMMul8**(`openmx_gemmul8Zgemm` × 3: H·T、T†(HT)、逆変換)	`d62cac60` で新設。スピン倍化変換 T は Zdgmm + `cudaMemcpy2DAsync` のブロックコピーで構成(§7.5)
Krylov	**GEMMul8**(`openmx_gemmul8Dgemm`。融合パスでは原子あたり 3 本: H·u1、u1†Hu1、逆変換)	3 本すべてが $m \cdot n \cdot k \geq 5\times10^7$ のときのみ融合パス発動(`3ee45228`、§7.6)
Mixing_H(RMM-DIISH)	GEMM 不使用(OpenACC の Gram/最適残差カーネル)	GPU を使う混合は RMM-DIISH の Tier A のみ(§7.12)
DMmu/CWF/LNO 系 11 ファイル	GPU は対角化のみ(GEMM は従来どおり CPU)	各ファイルの GEMMul8 ヘルパは定義のみ(準備扱い)

表 3-2: ソルバ別 GEMM エンジン(ライブ経路と休眠コードの区別)

3.5 撤去された汎用ラッパと簡約ヘルパの注意

初期開発で使われた自己完結型 GEMM ラッパ `my_cublasDgemm.c` / `my_cublasZgemm.c` (呼び出しごとに `cublasCreate` → `cudaMalloc` / `cudaMemcpy` → GEMM → 破棄)は、長らく**呼び出し箇所 0 件の休眠コード**だったのち、コミット `a7f8cba6` (Remove obsolete cuBLAS wrapper files)でソース・宣言・リンク対象とも**完全に削除**された。古いリビジョンを読む際にのみ登場する名前である。

関連して注意すべき簡約が現行ヘルパにも残る: ソルバ内 GEMM ヘルパの多くは **alpha=1.0 / beta=0.0 固定**であり、lda/ldb の扱いは 2 系統ある。ライブヘルパの大半は転置に応じた lda/ldb 計算に対応済みで、非コリニアクラスタには明示 leading dimension 引数を持つ `ClusterNonCol_GEMMul8ZgemmLd_OpenACC` (`Cluster_DFT_NonCol.c:993`)が追加されたが、正方行列でのみ使われる旧型ヘルパや DMmu/CWF 系の休眠ヘルパには **lda=m・ldb=k のハードコード**が残っており、転置指定時に正しいのは正方行列(m=n=k)の場合だけである。非正方・転置で再利用する場合は ld の扱いを必ず確認すること。

3.6 エラー処理 — wait_cudafunc マクロ

CUDA/cuBLAS/cuSOLVER 呼び出しの大半が次のマクロで包まれる。

openmx_common.h:4135-4162(抜粋)

```
#define GPU_CPU_SWITCH_NUM 1000
#define WAITTIME           (10.0 * 1.0E-6)

#define wait_cudafunc(call)                                                 \
    while (true) {                                                          \
        if (cudaSuccess == call) {                                          \
            break;                                                          \
        }                                                                   \
        /* pop the CUDA error latched by the failed attempt so it cannot be     \
           misattributed by the next cudaGetLastError() caller once a retry     \
           succeeds */                                                      \
        (void)cudaGetLastError();                                           \
        double wait_time    = drand48() * WAITTIME;                         \
        double start_time   = MPI_Wtime();                                  \
        double current_time = start_time;                                   \
        while ((current_time - start_time) < wait_time) {                   \
            current_time = MPI_Wtime();                                     \
        }                                                                   \
    }
```

失敗時は 0〜10 μs のランダム待ち(ビジーウェイト)を挟んで**成功するまで同じ呼び出しを無限に再試行**する。成功コード 0 は `cudaError_t` / `cublasStatus_t` / `cusolverStatus_t` に共通なので 3 種の API を同一マクロで包める。これは複数ランクが同一 GPU を取り合う際の一時的な `cudaMalloc` 失敗を 吸収するための設計である。

コミット `c5eb6845` で、失敗した試行ごとに `cudaGetLastError()` を呼んで **CUDA ランタイムのスレッド別エラーラッチを掃除する**行が追加された。これがないと、リトライで回復した確保失敗の エラーコードがラッチに残り、後続の無関係な `cudaGetLastError()` 利用者(Set_Hamiltonian の CUDA カーネルなど)が 自分の失敗と誤認して abort する「幻の失敗」が起こる(実際に SCF 316 反復目で発症した事例への対処。§7.7)。同じ理由で、DC 法の CPU フォールバック(`DC_GpuSolver_DisableGemmPath(Cuda)`)もフォールバック前に ラッチを pop する。

計算失敗(固有値の収束失敗など)への対応はソルバ層で行われ、経路により `MPI_Abort` / `exit` か CPU フォールバックかが異なる(詳細は §5.4 の表)。

3.7 cuBLAS 使用上の注意点

1. **`wait_cudafunc` は無限リトライ** — 引数の呼び出し式は毎周再評価される。恒久的エラー (不正な leading dimension、デバイス喪失など)を渡すとハング+副作用の反復になる。GPU 実行が「止まったように見える」ときは まずここを疑う。ただし主要な「満杯になり得る」確保は有界リトライの `TryDeviceMalloc` 系+フォールバックに 移行済みで、恒久的な VRAM 不足でハングする箇所は大幅に減った(§5.6)。
2. **簡約ヘルパの ld/alpha/beta 固定** — §3.5 のとおり。一部ヘルパに固定が残り、転置 + 非正方の組み合わせで静かに誤結果になる。
3. **ストリーム前提の非対称性** — Band_DFT_Col はデフォルトストリーム+同期 memcpy、他は専用ストリーム+明示同期。コードを移植・共通化する際は同期規約を確認すること。
4. **エラーラッチの規約** — `cudaGetLastError()` を「自分のカーネルの検査」に使うコードを追加する場合は、直前に stale エラーを pop してから launch する(`set_hamiltonian_gpu.cu` の作法に倣う)。回復可能な失敗を握りつぶす他コンポーネントとの共存が前提のツリーである。

5. **行列サイズ 0 のガード** — GEMMul8 ブリッジは m,n,k ≤ 0 で即成功を返す(全 GEMM がブリッジ経由のため、この一箇所で守られる)。
6. **cuBLAS ワークスペース未設定** — GEMMul8 が推奨する `cublasSetWorkspace`(32 MiB)は どのハンドルにも設定されていない(性能改修の候補。`gemmul8.hpp` のパフォーマンスノート参照)。
7. **列優先規約** — cuBLAS は Fortran BLAS と同じ列優先。OpenMX の実対称行列は対称性を利用して 行優先フラット配列をそのまま渡す箇所がある(転置と等価なため)。エルミート行列は明示的に列優先へ詰め替えるか、`cublasDgeam/Zgeam` によるデバイス内転置で整える(DC 法、DC-LNO 非コリニア)。

4. GEMMul8 の使い方

4.1 GEMMul8 とは — 原理と出自

GEMMul8(GEMMulate)は理化学研究所 計算科学研究センター(RIKEN R-CCS)が開発する「INT8/FP8 行列エンジンを用いた GEMM エミュレーション」ライブラリである(責任開発者: Yuki Uchino 氏、MIT ライセンス)。本実装が取り込んでいるのは **v3.1.1**(上流コミット `6bf409bd…` に固定)。上流リポジトリ: `https://github.com/RIKEN-RCCS/GEMMul8`。

原理は **Ozaki Scheme II**(整数モジュラー技法/中国剰余定理 CRT に基づく浮動小数点行列積エミュレーション)である。

1. **スケーリング+量子化:** FP64 行列 A, B を指数調整し、互いに素な法 p = 256, 255, 253, 251, 247, 241, …(8 bit に収まる整数、最大 20 個)ごとの 剰余をとって `num_moduli` 枚の INT8 行列に変換する。
2. **誤差なし整数行列積:** 法ごとに INT8 GEMM を実行する。実体は cuBLAS の `cublasGemmEx(…, CUDA_R_8I, …, CUBLAS_COMPUTE_32I, …)`、すなわち **INT8 Tensor Core(IMMA)**である。整数演算なので丸め誤差が入らない。
3. **CRT 復元:** 法別の結果から中国剰余定理で整数積を復元し、スケーリングを戻して $\alpha AB + \beta C$ を得る。

`num_moduli`(法の個数)を増やすほど高精度・低速になる。手順が決定的であるため、**同一設定なら結果は bitwise に再現する**(上流 README が明記)。GEMMul8 は、INT8 整数行列エンジンで DGEMM を行う先行研究 ozIMMU(Ootomo, Ozaki, Yokota 2024)系の Ozaki Scheme I を発展させた Scheme II の実装という位置づけである。引用すべき文献は巻末の参考文献を参照。

4.2 統合アーキテクチャ — 2 ファイル方式

OpenMX 本体は NVHPC `mpicc -std=c11` でビルドされる C コードであり、C++ テンプレート API を直接呼べない。また上流のビルドシステムは「ルーチン × 型 × バックエンド」ごとに大量のオブジェクトを生成する。そこで本実装は次の 2 ファイルで統合している。

(a) gemmul8_openmx.cu — 単一翻訳単位での明示的テンプレート実体化

上流のテンプレート**定義**ヘッダ(`gemm_impl.hpp`、`worksize_impl.hpp` と、上流では別オブジェクトに分かれるカーネル定義ヘッダ 7 本)を 1 つの翻訳単位に include し、OpenMX が必要とする **4 シンボルだけ**を明示的実体化する(67 行の小さなファイル)。

gemmul8_openmx.cu:31-44(double 版。cuDoubleComplex 版と workSize 2 種も同様)

```
template std::vector<double> gemm<double, Backend::INT8, double, double>(
    cublasHandle_t,
    cublasOperation_t, cublasOperation_t,
    size_t, size_t, size_t,
    const double *,
    const double *const, size_t,
    const double *const, size_t,
    const double *,
    double *const, size_t,
    int, bool,
    void *const,
    void *const,
    void *const,
    bool, bool, bool, bool);
```

実体化されるのは `gemm<double,INT8>`、`gemm<cuDoubleComplex,INT8>`、`workSize<false,INT8,Func::gemm>`、`workSize<true,INT8,Func::gemm>` の 4 本。このオブジェクトだけを `ar` で固めて `third_party/GEMMul8/lib/libgemmul8.a` を自

前生成する (上流の同名ライブラリとは中身が異なる)。この方式は上流の内部ヘッダ構成に依存するため、**上流コミットのピン留めが必須**である(Makefile のコメントに明記。§ 4.7)。

(b) gemmul8_bridge.cu — extern "C" ブリッジ

C 側から呼べる 5 関数を輸出する(プロトタイプは `openmx_common.h:4164-4173`)。

- `cublasStatus_t openmx_gemmul8Dgemm(handle, transa, transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)` — **cublasDgemm と完全同型**のシグネチャ(alpha/beta・ld とも任意)
- `openmx_gemmul8Zgemm(…)` — 複素版(同型)
- `size_t openmx_gemmul8DWorkspaceSize(m, n, k)` / `openmx_gemmul8ZWorkspaceSize(m, n, k)` — **ワークスペース必要量の照会 API**。現在有効な num_moduli 設定で `gemmul8::workSize` を評価して返す (環境変数で当該精度が無効化されていれば 0)。各ソルバの VRAM 見積り(Band の k ターン計画、DC-LNO の グループ適合判定など)が GEMM ワークスペース分を正確に算入するために使う(コミット `58ea6471`, `d62cac60` で追加)
- `openmx_gemmul8ReleaseWorkspaces()` — 全ワークスペースの解放

ブリッジは (i) 環境変数の解釈、(ii) ワークスペースの照会・確保・キャッシュ、(iii) 確保できない場合の native cuBLAS へのフォールバック、(iv) `gemmul8::gemm` 呼び出し、を担う。

gemmul8_bridge.cu(Dgemm 本体の骨格。Zgemm も同構造)

```
if (gemmul8_disabled("OPENMX_GEMMUL8_DISABLE_D", "GEMMUL8_DISABLE_D")) {
    report.reason = "environment disable";
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}

cublasStatus_t status =
    ensure_workspace<false>(handle, static_cast<size_t>(m), static_cast<size_t>(n), static_cast<size_t>(k),
                            num_moduli, &work, &report);
if (status == CUBLAS_STATUS_ALLOC_FAILED) {
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}
if (status != CUBLAS_STATUS_SUCCESS) {
    return status;
}

(void)gemmul8::gemm<double, gemmul8::Backend::INT8>(handle, gemmul8_transa, gemmul8_transb, ..., num_moduli, fastmode, work);
```

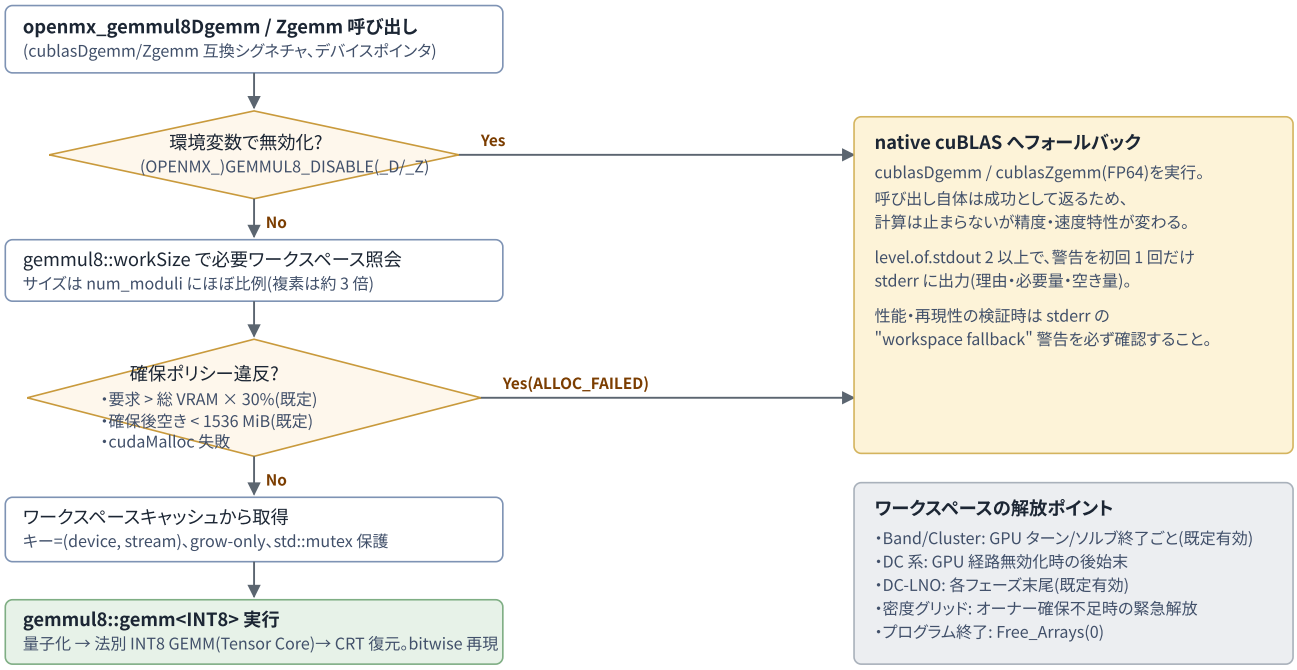


図 3: GEMMul8 ブリッジのディスパッチフロー(gemmul8_bridge.cu)。

4.3 呼び出しパラメータと精度設定

パラメータ	既定値	環境変数(OPENMX_ 名が優先)	意味・挙動
`num_moduli`	**15**	`(OPENMX_)GEMMUL8_NUM_MOD_D` / `_Z`	法の個数(精度)。有効範囲 2〜20、範囲外は黙って 15 に戻る。D/Z 独立に設定可
`fastmode`	**false**(accurate)	`(OPENMX_)GEMMUL8_FASTMODE_D` / `_Z` ("1" で有効)	高速スケーリング経路。精度が下がる
無効化	off	`(OPENMX_)GEMMUL8_DISABLE`、`_D` / `_Z`	native cuBLAS FP64 に固定
WS 上限比率	**30%**	`(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT`	要求が総 VRAM のこの割合を超えたら拒否(Band_DFT_NonCol は未設定時に 50 を setenv する)
確保後最低空き	**1536 MiB**	`(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB`	確保後の空き VRAM がこの値を割るなら拒否

- 実数(D)経路は `CUBLAS_OP_C` を `CUBLAS_OP_T` に静かに正規化する(実数では共役転置=転置)。
- `m,n,k ≤ 0` は何もせず成功を返す。
- OpenMX 側に GEMMul8 の結果を FP64 直接計算と比較・検証するコードは存在しない。フォールバックはすべて メモリ/環境変数条件によるもので、精度条件によるものではない。
- skip-scaling 等の上流拡張機能は未使用。フェーズ別時間の戻り値も捨てられる。

4.4 ワークスペース管理

- **照会:** 呼び出しごとに `gemmul8::workSize<is_complex,INT8>(m,n,k,num_moduli)` で必要量を計算。ソルバ側からは `openmx_gemmul8D/ZWorkspaceSize` で事前照会でき、各経路の VRAM 見積りに算入される。
- **キャッシュ:** `(device, cudaStream_t)` をキーとする `std::unordered_map` でプロセス全体共有、`std::mutex` 保護、grow-only。cuBLAS ハンドルのストリームが キーになるため、**ハンドル(ストリーム)ごとに別領域**になる点に注意。
- **サイズ感:** A 側 = num_moduli 枚の INT8 行列+シフトベクトル、B 側同様、C 側 = (num_moduli−1) 枚の中間行列 + 32 MiB の内部 BLAS 領域 + INT32 アキュムレータ。複素はさらに約 3 倍。n ≈ 1 万規模で**約 1 GiB/ランク**。

- **解放:** `openmx_gemmul8ReleaseWorkspaces()` のみ。呼び出し箇所は (1) `Free_Arrays(0)`(プログラム終了時)、(2) Band_DFT_Col の GPU ターン終了ごと(`OPENMX_BAND_GEMMUL8_TURN_RELEASE=0` でオプトアウト)、(3) Cluster_DFT_Col/NonCol のソルブ終了ごと(`OPENMX_CLUSTER_GEMMUL8_TURN_RELEASE`、未設定時は BAND 設定を継承)、(4) DC 法の GPU 経路無効化時、(5) DC-LNO の各フェーズ末尾(`OPENMX_DCLNO_GPU_TURN_RELEASE`)、(6) 密度グリッド GPU サービスがオーナー確保に失敗しそうなときの緊急解放。

4.5 複素数(Z)対応

OpenMX 側では複素行列を実行列に分解せず、`cuDoubleComplex` のまま `gemm<cuDoubleComplex, INT8>` に渡す。GEMMul8 内部のネイティブ複素パスは、入力 1 個あたり低精度表現を 3 セット持ち、1 法あたり**実 INT8 GEMM 3 本**で複素積を構成する(4M 法ではなく 3M/Karatsuba 系。理論的背景は CRT に基づく複素行列積エミュレーションの論文、参考文献参照)。このため複素のワークスペースと演算量は実数の約 3 倍になる。

4.6 どの GEMM が GEMMul8 で実行されるか

ブリッジ内に行列サイズによる振り分けは**ない**(GEMMul8 が既定、フォールバックはメモリ/環境変数条件のみ)。コミット `66ce1e62` 以降は **GPU 上の全 GEMM がブリッジを経由する**(表 3-2)。代表例は Band_DFT_Col の一般化固有値問題変換 $H' = \tilde{U}^\dagger H \tilde{U}$ である(ここが最大の GEMM 負荷)。

Band_DFT_Col.c — 三重積変換(2 本目までを抜粋。逆変換 1 本が続く)

```
wait_cudafunc(openmx_gemmul8Zgemm(ctx->cublas, CUBLAS_OP_N, CUBLAS_OP_N, n, n, n, &alpha,
                                  (cuDoubleComplex *)ctx->d_H, n, (cuDoubleComplex *)ctx->d_S, n, &beta,
                                  (cuDoubleComplex *)ctx->d_tmp, n));

wait_cudafunc(openmx_gemmul8Zgemm(ctx->cublas, CUBLAS_OP_C, CUBLAS_OP_N, n, n, n, &alpha,
                                  (cuDoubleComplex *)ctx->d_S, n, (cuDoubleComplex *)ctx->d_tmp, n, &beta,
                                  (cuDoubleComplex *)ctx->d_H, n));
```

4.7 ビルド

- **取得:** `make` が git submodule ディレクトリで `git fetch --depth 1 origin $(GEMMUL8_COMMIT)` → `checkout` を実行し、コミット `6bf409bd…`(v3.1.1)に固定する。チェックアウトが無い新規クローンでも `gemmul8.hpp` の order-only ルール経由で自動取得される(コミット `9fe8ce13`)。ピン留めの理由は Makefile コメントに明記:「gemmul8_openmx.cu が GEMMul8 内部を直接実体化するため、チェックアウトは内部レイアウトが一致するコミットに固定し続けなければならない」。
- **コンパイル:** NVHPC 同梱 CUDA 13.1 の nvcc、ホストコンパイラ `-ccbin nvc++`、`-std=c++20 -O3 -diag-suppress 177 -DGPU_ARCH=$(arch) -gencode arch=compute_$(arch),code=sm_$(arch)`。アーキは `nvidia-smi --query-gpu=compute_cap` から自動検出(本開発環境では compute_120/sm_120)。`GEMMUL8_GPU_ARCH=90` のように手動指定も可能。
- **環境サニタイズ:** `NVCC_GEMMUL8_ENV` が `CPATH` 等を空にして NVHPC のヘッダ・フラグ混入を遮断。
- **リンク追加:** `-lcublasLt -lcuda -lnvidia-ml -lstdc++`。

4.8 GEMMul8 使用上の注意点

1. **メモリ使用量** — ワークスペースは num_moduli にほぼ比例、複素は約 3 倍、n ≈ 1 万で約 1 GiB/ランク。複数ランク共有 GPU では「30% 上限・1536 MiB 予約・ターン毎解放」の 3 段構えで枯渇を防いでいる。さらに各ソルバの 並列度プランナが `openmx_gemmul8D/ZWorkspaceSize` でワークスペース分を事前算入するため、既定値を変える場合は見積りとの整合を保つこと。

2. **静かな FP64 フォールバック** — ワークスペース拒否時は警告 1 回だけで cuBLAS FP64 に切り替わり、**速度と数値結果(ビットパターン)の両方が変わる**。「なぜか遅い/結果が揺れる」ときは `level.of.stdout 2`(または `OPENMX_GEMMUL8_VERBOSE=1`)で実行して stderr の "GEMMul8 workspace fallback" を確認する。決定的な再現が必要な検証では `OPENMX_GEMMUL8_DISABLE=1` で全無効化するか、メモリに十分な余裕を持たせる。
3. **精度設定** — 既定 15 moduli・accurate モード。範囲外の指定は黙って 15 に戻る。fastmode は高速だが スケーリング精度が落ちる。D と Z は独立に調整できる。
4. **初回呼び出しコスト** — 初回およびサイズ増加時に GiB 級の `cudaMalloc` が同期的に走る。ベンチマークは 2 回目以降の反復で評価すること。
5. **ビルドのハマりどころ** — (a) コミットピンを外すと内部ヘッダ構成の変更で `gemmul8_openmx.cu` が壊れる。(b) NVHPC 環境変数(CPATH 等)を掃除しないと nvcc が NVHPC ヘッダを拾って失敗し得る。(c) `-lcublasLt`・`-lstdc++` のリンク忘れ。(d) アーキ自動検出は `nvidia-smi` が使える環境前提。(e) 単一アーキ `-gencode` のため別世代 GPU ではバイナリ非互換(要再ビルド)。
6. **上流 API 追従** — 旧 API から現行構造(884a287)への移行(コミット `cb4d348d`)で `gemmul8_openmx.cu` は 149 行 → 67 行に簡素化された。ブリッジの公開シグネチャは不変で、C 側への影響はない。今後の上流更新時も「単一 TU が include する定義ヘッダの構成」を最初に確認するとよい。
7. **ハードウェア要件** — INT8 Tensor Core(IMMA)必須。C++20 対応 nvcc(CUDA 12 系以降)が事実上必要 (検証済みは CUDA 13.1 のみ)。

5. cuSOLVER(gpusolver 層)の使い方

5.1 層の構成と命名の意図

固有値計算は cuSOLVER を **gpusolver** というベンダー中立名のラッパ層経由で使う。層は次の 3 階層で構成される。

1. **汎用ラッパ関数**(1 呼び出し完結型): `gpusolver_Syevd.c` / `gpusolver_Syevdx.c` の 6 関数。ホスト配列版と、OpenACC デバイス常駐配列を直接処理する `_openacc` 版、さらにハンドル・ワークスペースを 呼び出し間でキャッシュする `_cached` 版がある(表 5-1)。冒頭のライセンスヘッダが示すとおり、NVIDIA CUDA Library Samples の syevd/syevdx サンプルを改変した出自を持つ。
2. **共通ユーティリティヘッダ**: `openmx_gpusolver_dense_utils.h` (79 行)。`OpenMX_GpuSolver_DenseDsyevx/DenseZheevx` の 1-based/0-based 版 4 関数を static 定義し、`info != 0` なら `MPI_Abort` する。分極・DMmu・CWF・LNO 系の 12 ファイルが include する。
3. **ソルバ内常駐コンテキスト**: `BandColGpuSolverCtx`、`ClusterColGpuSolverCtx`、`DCGpuSolverCtx`、`DCLNO_GpuSolverZCtx`、Krylov のスレッド別ワークスペースプールなど。ハンドル・デバイスバッファをキャッシュし、cuSOLVER API を直接呼ぶ(第 3 章と共通の構造)。

バックエンドの選択: 関数名と入力キーワードは両バックエンドで gpusolver を使う。CMake がコンパイル前にソース集合を選び、実行ファイルごとにバックエンドを固定する(§9.7)。以下の cuSOLVER の型と API は CUDA ビルドの説明であり、HIP では hipSOLVER と型別 API への互換ラッパを使う(§11.5)。入力キーワードの旧名 `cusolver` は警告付きの非推奨エイリアスとして残る。

関数(gpusolver_Syevd(x).c)	cuSOLVER API	データ所在	現在の呼び出し元
`gpusolver_Syevd(A,W,m)`	Xsyevd(全固有値)	ホスト⇄デバイス転送内蔵	休眠 呼び出し元なし
`gpusolver_Syevdx(A,W,m,MaxN)`	Xsyevdx(下から MaxN 本)	同上	`Eigen_lapack.c`、dense_utils 経由多数
`gpusolver_Syevdx_openacc(…)`	Xsyevdx	OpenACC 常駐(present + host_data)	`Eigen_lapack.c`
`gpusolver_Syevdx_Complex(…)`	Xsyevdx(CUDA_C_64F)	ホスト⇄デバイス転送内蔵	`EigenBand_lapack.c`、dense_utils 経由多数
`gpusolver_Syevdx_Complex_openacc(…)`	Xsyevdx(CUDA_C_64F)	OpenACC 常駐	`EigenBand_lapack.c`
`gpusolver_Syevdx_Complex_openacc_cached(…)`	Xsyevdx(CUDA_C_64F)	OpenACC 常駐	DC 非コリニア・DC-LNO 非コリニアの毎原子対角化(コミット `6f7f31b2` で新設。static コンテキストにハンドル・ストリーム・grow-only ワークスペース・d_info を保持し、原子ごとの create/destroy を排除)

表 5-1: 汎用ラッパの公開 API

5.2 使用 API の詳細

- すべて **generic(X)API・64-bit 整数版**を使用: `cusolverDnXsyevd(_bufferSize)` / `cusolverDnXsyevdx(_bufferSize)`。legacy API(`cusolverDnDsyevd` 等)や `Zheevd` 系の別名 API は一切使わず、複素も Xsyevdx に `CUDA_C_64F` を渡す統一形。一般化固有値ソルバ `cusolverDnXsygvd` も使わない(§5.3)。
- 共通パラメータ: `params = NULL`、`uplo = CUBLAS_FILL_MODE_LOWER`、範囲指定は `CUSOLVER_EIG_RANGE_I` (il=1, iu=MaxN)で、`m == MaxN` のとき `RANGE_ALL` に切り替える実装と常に RANGE_I の実装が混在する。jobz は原則

`CUSOLVER_EIG_MODE_VECTOR` だが、Band(Col)の第 1 パス(固有値のみ必要)は `CUSOLVER_EIG_MODE_NOVECTOR` で解いて逆変換 GEMM ごと省略する (`BandCol_EigenvaluesOnlyNoVectors()` 、`OPENMX_BAND_SYEVDX_NOVECTOR=0` で無効化)。

- ハンドル: 汎用ラッパは呼び出しごとに `cusolverDnCreate` → non-blocking ストリーム生成 → 計算 → 全解放。`_cached` 版と常駐コンテキストはキャッシュする(§ 3.2)。
- ワークスペース: 全経路で `_bufferSize` によりデバイス/ホスト両方の必要量を照会してから確保する。常駐側は grow-only キャッシュ。さらに並列度プランナも **実際に `_bufferSize` を照会して** syevdx ワークスペース分を VRAM 見積りに算入する(照会失敗時は $4n^2$ 複素要素で代用。§ 5.5)。

gpusolver_Syevdx.c — 中核の呼び出し形(64-bit generic API)

```
wait_cudafunc(cusolverDnXsyevdx_bufferSize(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu,
                                           1L, MaxN, &h_meig, CUDA_R_64F, d_W, CUDA_R_64F,
                                           &workspaceInBytesOnDevice, &workspaceInBytesOnHost));

wait_cudafunc(cudaMalloc((void **)(&d_work), workspaceInBytesOnDevice));
h_work = (workspaceInBytesOnHost == 0) ? NULL : malloc(workspaceInBytesOnHost);
...
wait_cudafunc(cusolverDnXsyevdx(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu, 1L, MaxN,
                                &h_meig, CUDA_R_64F, d_W, CUDA_R_64F, d_work, workspaceInBytesOnDevice, h_work,
                                workspaceInBytesOnHost, d_info));
```

5.3 一般化固有値問題 Hc = εSc の処理

cuSOLVER の一般化ソルバは使わず、OpenMX 従来の **S の固有分解によるレイディン(対称)直交化**を各経路が GPU 上で自前実装する。Cholesky 分解は一切使わない。手順は共通して次の 6 ステップである。

1. S を Xsyevdx(全固有値)で対角化 → 固有ベクトル U、固有値 λ
2. 小さい/負の固有値の処置(**経路により異なる**: Band/Cluster は 1×10^{-10} に切上げ、DC は `OLP_eigen_cut` 未満を切り捨て、DC-LNO は `fabs`)
3. U の列を $1/\sqrt{\lambda}$ でスケール(`cublasDdgmm/Zdgmm`)→ $\tilde{U}$。DC-LNO 非コリニアはさらに `cudaMemcpy2DAsync` のブロックコピーでスピン倍化変換 T = diag($\tilde{U}$, $\tilde{U}$) を構成する
4. $H' = \tilde{U}^{\dagger} H \tilde{U}$ を GEMM 2 回で構成(GEMMul8。表 3-2)
5. H' を Xsyevdx(il=1..MaxN)で対角化 → 固有値 ε、固有ベクトル z
6. 逆変換 $c = \tilde{U}z$ を GEMM で実行(固有値のみ必要な場面では省略)

Cluster_DFT_Col.c — S 対角化と $1/\sqrt{\lambda}$ スケーリング(GPU 上のレイディン直交化)

```
if (rebuild || !(ctx->transformed_s_valid && ctx->transformed_s_dim==n)){
    ClusterCol_GpuSolver_EigenDevice(ctx->d_S,n,n,ko0+1);

    for (int l=1; l<=n; l++){
        if (ko0[l]<1.0e-10) ko0[l] = 1.0e-10;
        ko0[l] = 1.0/sqrt(ko0[l]);
    }

    wait_cudafunc(cudaMemcpyAsync(ctx->d_W,ko0+1,sizeof(double)*(size_t)n,cudaMemcpyHostToDevice,ctx->stream));

    old_s = ctx->d_S;
    new_s = ctx->d_tmp;
    wait_cudafunc(cublasDdgmm(ctx->cublas,CUBLAS_SIDE_RIGHT,n,n,
                              old_s,n,ctx->d_W,1,new_s,n));
```

注意(数値的挙動の差): S の小さい固有値の処置(切上げ / 切捨て / fabs)が経路間で統一されていないため、悪条件の重なり行列を持つ系では、同じ物理系でもソルバ経路によって数値挙動が微妙に異なり得る。

部分固有値数 MaxN の決め方: Band(Col)は `MaxN = (TZ-charge)/2 + max(0.4·Ne/2, 100)`、Cluster は SCF 第 1 反復で全数、以後 `(TZ-charge)/2 + max(0.1n〜0.2n, 400)`。DC-LNO は SCF 2 反復まで全数、以後 `HO_TC + 100`。占有帯+余裕分だけを解くことで syevdx の計算量を抑えている。

5.4 devInfo の検査と失敗時処理(経路別)

`d_info` は全経路でデバイスに確保し、計算後にホストへコピーして検査するが、**失敗時の方針は経路ごとに異なる**。なお VRAM 不足による発動不可は info 失敗より前の段階 (事前見積り/実予約)で検出され、ELPA/CPU にフォールバックする (§5.6)。ここでの表は「GPU 経路に入った後の計算失敗」の扱いである。

経路	info 検査	失敗時の挙動
汎用ラッパ(gpusolver_Syevdx 等)	呼び出し元に返すのみ	呼び出し元依存
dense_utils ヘッダ経由(11 ファイル)	あり	即 `MPI_Abort`
`Eigen_lapack.c`	**info<0 のみ** exit(10)	info>0(収束失敗)は素通り。ただし上位の `Eigen_lapack()` に固有ベクトル直交性検査つきリトライがあり、失敗時はソルバを CPU 系へ巡回して再計算(最大 3 回)
`EigenBand_lapack.c`	あり	CPU の Householder 法 `Eigen_HH` へフォールバック
Band_DFT_Col / Cluster_DFT_Col / Cluster_DFT_NonCol	あり(Band は h_meig も)	即 abort(exit / MPI_Abort)。ただし VRAM 起因の発動不可は事前に ELPA へ逃げるため、ここに来るのは数値的失敗のみ
Divide_Conquer(DC)	あり(h_meig も)	**sticky 無効化フラグ**を立てて以後そのランクは CPU の DC ソルバへ。GEMMul8 ワークスペースも解放
Divide_Conquer_LNO(Col/NonCol)	あり	exit / abort(フォールバックなし。ただし発動前にデバイス共有ランク群の合算メモリ判定があり、不足時は最初から CPU 固有値ソルバを使う)
Krylov	あり	CPU LAPACK(`Eigen_lapack2`)へその場でフォールバック(融合パスでは結果を再アップロードして続行)

表 5-2: devInfo 検査と失敗時処理の経路差

5.5 適応並列度制御 — UUID 共有群とプレフィックススキャン

第 2 版時点の並列度制御は「固定グループ幅(既定 4)の GPU ターン直列化」と「固定はしご(32→16→⋯→1)」だったが、現在は**全経路が実測ベースの適応制御**に統一されている(コミット `58ea6471`, `7c414f91`, `48921b36`, `99f4014b`, `6b84a1f9`)。共通の骨格は次のとおり。

1. **共有群の検出:** `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` でノード内コミュニケータを作り、`cudaGetDeviceProperties` の **GPU UUID**(16 バイト)を Allgather して、同一物理 GPU を共有する ランク群のコミュニケータ(device_comm)に分割する。`cudaSetDevice` の番号ではなく UUID で照合するため、`CUDA_VISIBLE_DEVICES` の設定差があっても正しく共有を検出できる。
2. **空き VRAM の合意:** 計測前に `acc_wait_all(); cudaDeviceSynchronize(); acc_clear_freelists();` で 各プロセスの OpenACC フリーリストを CUDA に返却してから `cudaMemGetInfo` を全員で実行し、`MPI_Allreduce(MIN)` でグループ共通の空き(group_free)を得る。
3. **per-rank 所要量モデル:** 密行列・ベクトル・syevdx ワークスペース(**`_bufferSize` の実照会値**)・GEMMul8 ワークスペース(`openmx_gemmul8D/ZWorkspaceSize`)・疎→密 構築/DM 転送域・ランタイム オーバーヘッド(既定 256 MiB、`OPENMX_BAND_GPU_RANK_OVERHEAD_MB`)を合算する。
4. **プレフィックススキャン:** 各ランクの所要量を Allgather して降順ソートし、上位 c ランクの合計 peak が `peak ≤ group_free かつ reserve ≤ group_free − peak` を満たす**最大の c** を選ぶ (reserve = max(総 VRAM の 1/10, 1536 MiB)、`OPENMX_BAND_GPU_RESERVE_MB` で引き上げ可)。初期候補はデバイス共有ランク数(旧版の固定はしごと暗黙の上限 32 は撤廃され、環境変数は「明示上限」の意味だけになった)。

5. **合意と縮退:** 結果を `MPI_Allreduce(MIN)` で全ランク一致に揃える。全ランクが同時に収まる場合は ターン分割そのものを畳み込み(バリアと GEMMul8 解放点が消える)、1 ランクも収まらない場合は ELPA フォールバック(§5.6)。
6. **需要の公開:** フル並列時の必要総量を `OpenMX_GpuPhaseNeed_Register("band_col" など)` で公開し、Set_Hamiltonian の常駐キャッシュがこの分を空けて動く(§7.7)。

5.5.1 Band(Col)— BandCol_AutoGpuTurnLimit

所要量モデルは `BandCol_GpuTurnRequiredBytes(n, maxn, size_H1)`(`Band_DFT_Col.c:429`): 密行列 3 枚($3 \cdot n^2 \cdot 16$ B)+ ベクトル + syevdx ワークスペース((n,n)/(n,maxn) の 2 形状を実照会してメモ化)+ GEMMul8 Zgemm ワークスペース + max(構築転送域, DM 転送域) + オーバーヘッド。決定された並列度は次の 1 行で報告される(値が変わったときだけ、デバイス代表ランクが出力):

```
<Band_DFT_Col> GPU device %d: %d k-owner rank(s), GPU concurrency=%d, turn group=%d,
    per-rank=%.3f GiB, peak=%.3f GiB, free=%.3f GiB, reserve=%.3f GiB
```

`all_knum==1` 側の GPU ターン直列化(`OPENMX_GPUSOLVER_SERIAL_GPU_TURNS`、既定 1)は温存されており、コード内コメントは「8 オーナーランク/1 GPU では並行投入より約 27% 高速」のまま。明示上限は `OPENMX_BAND_GPU_MAX_CONCURRENT_K` → `OPENMX_BAND_GPU_TURN_GROUP` → `OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE` の優先順で読まれる。

5.5.2 Band(NonCol)— ステージ別の 3 モード見積り

非コリニアバンドは 1 ターンの中でメモリピークが大きく変わるため、見積りを **固有値のみ(EIGVALS)/固有ベクトル(EIGVECS)/ 密 度 行 列 (DM)** の 3 モードに分けて別々に並列度を決める(`BandNonCol_RootDenseDeviceBytes` / `BandNonCol_RootDenseDMDeviceBytes`、`Band_DFT_NonCol.c:803,871`)。DM 蓄積の所要量はソルブ段よりはるかに小さいため、たとえば「ソルブは 2 ランクずつ、DM 蓄積は 8 ランク同時」のような 運転が可能になった(コミット `7c414f91`)。報告行はモード名付き:

```
<Band_DFT_NonCol> GPU device %d: %d k-owner rank(s), GPU concurrency=%d (eigenvalues only |
    eigenvectors | density matrix), turn group=%d, per-rank=%.3f GiB, ...
```

複数 k 世界の同時実行可否も同じ枠組みで判定され、収まらなければ「Serializing non-collinear dense GpuSolver k-worlds」と表示して直列化する。また `BandNonCol_SetDenseGemmul8Defaults` が `OPENMX_GEMMUL8_MAX_WORKSPACE_PERCENT` 未設定時に 50 を setenv する。

5.5.3 クラスタ(Col/NonCol)— 実予約方式

クラスタ経路は k ターンを持たない(root-dense の単一オーナー)ため、並列度制御の代わりに **「見積り」ではなく「実予約」**で発動可否を決める。オーナーランクが行列バッファと syevdx ワークスペース(`_bufferSize` の実報告値)を **8 回上限のリトライ付き cudaMalloc で実際に確保**し、さらに `OPENMX_CLUSTER_GPU_DIAG_RESERVE_MB`(既定 1024 MiB)が空きに残ることを要求する(`ClusterCol_OwnerReserveProbe`)。プローブは精錬 FP32 固有値ソルバ(`OPENMX_EIGEN_REFINE`、§10.2)の n^2 の基底も数える。密行列バッファと予約分は収まるが基底は収まらないときは、ELPA へ落ちる代わりに GPU 経路を保ち、そのサイクルを FP64 固有値ソルバで解く(1 行 `… holds its GPU diagonalization buffers but not the … basis of the refined eigensolver …: FP64 eigensolver for this SCF cycle` が出る。16 GB の RTX 5080 で 18 ランクの C1000 が この場合に当たる: 予約後の空き 1257 MiB に対し、予約分 1024 MiB+基底 1289 MiB)。実予約方式に変えた背景には、「syevdx ワークスペースの見積り $2n^2$ doubles に対し実際は約 $4n^2$ doubles だった」という実測がある(コミット `cff323e6`)。非コリニアは対角化サイクル全体を**単一 cudaMalloc のアリーナ**として予約し、`acc_map_data` で OpenACC に見せるため、予約成立後は他フェーズの動きに影響されない(`ClusterNonCol_DenseArena_TryEnsure`)。

スピン分極(SpinP_switch==1)で 2 つのスピンワールドのオーナーが同時に収まらない場合は、**スピンワールド 0 のオーナーが両スピンを直列に解く**第 3 の運転モードがある(コミット `bf08a9d8`、`OPENMX_CLUSTER_GPU_DIAG_SERIAL` 既定 1、2 で直列

強制)。このときリモートスピンの固有値・固有ベクトルは MPI(タグ 997/998、64M 要素ずつ分割)で相手オーナーへ送られる。それも不可なら ELPA へ(§5.6)。

5.5.4 DC-LNO — デバイス共有ランク群の合算判定

DC-LNO の direct per-rank dispatch は全ランクがソルババッファを常駐させるため、`DCLNO_GpuGroupMemoryFits`(`Divide_Conquer_LNO.c:1012`)が「7·elem·Max_Msize² + GEMMul8 ワークスペース + 256 MiB」をランク合算(Allreduce SUM)し、空き MIN と比較して不足なら**グループ全体で CPU 固有値ソルバに切り替える**(18 ランク NC 実行が 16 GiB カードに 26.1 GiB を要求して 40 分スピンした事例への対処)。通知は代表ランクが 1 回:

```
<DC-LNO> GPU device %d: %d rank(s) would need %.3f GiB (free %.3f GiB, reserve %.3f GiB);
    using the CPU eigensolver instead.
```

5.6 実行時 ELPA フォールバック

第 2 版までは「GPU 経路に入ってからの VRAM 枯渇は abort かハング」だったが、コミット `ae3dd189` / `cff323e6` / `3ef5347a` / `a4200a04` で **バンド・クラスタの全 4 経路に実行時 ELPA/ScaLAPACK フォールバック**が入った。共通仕様:

- 判定関数:`BandCol_GpuDenseFits` / `BandNonCol_GpuDiagFits`(1 ターン分の見積り)、`ClusterCol_GpuDiagFits` / `ClusterNonCol_GpuDiagFits`(実予約)。
- 判定(verdict)は `mpi_comm_level1` の `MPI_Allreduce(MIN)` による**集団合意**。GPU/ELPA の分岐は集団通信の構造を変えるため、全ランクが必ず同じ側に落ちる。
- verdict は static にキャッシュされ、SCF リスタートか行列サイズ変化まで再判定しない(sticky)。
- フォールバック時は予約済みバッファを全解放してから従来の ELPA/ScaLAPACK コードへ落ちる。通知は 1 回だけ:

```
<Cluster_DFT_Col> Falling back to the ELPA/ScaLAPACK diagonalization. Force the GPU path with
    OPENMX_CLUSTER_GPU_DIAG=1 or lower OPENMX_CLUSTER_GPU_DIAG_RESERVE_MB.

<Band_DFT_Col> A k-owner rank cannot fit even one k-point's dense GPU diagonalization
    (%.1f MiB needed per rank, %.1f MiB free); falling back to the ScaLAPACK/ELPA diagonalization.
    Force the GPU path with OPENMX_BAND_GPU_DIAG=1 or lower OPENMX_BAND_GPU_RESERVE_MB.
```

- 強制ノブ:`OPENMX_BAND_GPU_DIAG=1/0`、`OPENMX_CLUSTER_GPU_DIAG=1/0`(1 = 判定を無視して GPU 強制〔予約失敗時は明確な診断つき abort〕、0 = 常に ELPA)。
- 必須確保が本当に尽きた場合の abort は、無言のハングではなく所要量・空き量つきの診断になる:`"Cluster_DFT_Col.c: out of GPU memory while allocating %s (%.1f MiB requested, %.1f MiB free of %.1f MiB). Reduce the MPI ranks sharing the device or set OPENMX_CLUSTER_GPU_DIAG=0 ..."`

> **設計の意図:** 旧実装では root-dense の行列確保が無限リトライ(`wait_cudafunc(cudaMalloc(…))`)だったため、満杯のデバイスでオーナーがスピンし、他の全ランクが MPI_Bcast で待つ「ノード全体の無言フリーズ」が起きた(n=13,000・np18・16 GiB GPU で実測)。有界リトライ+実予約+集団合意+ELPA フォールバックはこのクラスの 障害を仕様上排除する(コミット `ae3dd189`)。

5.7 MPI ランク → GPU デバイス割当

- **collective 版**(`set_cuda_default_device_from_local_rank.c`):`MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` でノード内 local rank を求め、`local_rank % deviceCount` を `cudaSetDevice`。全ランクが揃って呼ぶ必要がある(片側だけ呼ぶとデッドロック)。

- **noncollective 版**: SCF 内ホットパス用。環境変数 `OMPI_COMM_WORLD_LOCAL_RANK` → `MV2_COMM_WORLD_LOCAL_RANK` → `SLURM_LOCALID` → `PMI_LOCAL_RANK` の順で local rank を取得。どれも無ければ大域 rank で代用(この場合マルチノードでは割当が偏り得る)。
- **OpenACC 側も対で設定** (`set_openacc_device_from_local_rank.c`): 同じ規則で `acc_set_device_num` 。**CUDA と OpenACC の両ランタイムに同一デバイスを設定することが必須**という設計。
- 割当後の「実際にどのランクがどの物理 GPU を共有しているか」は、各フェーズが UUID の Allgather で自律的に検出する(§5.5)。

5.8 LU_Zinverse_GPU — NEGF 向けの休眠実装

> **休眠コード:** 複素 LU 逆行列の GPU 版 `LU_Zinverse_GPU` (`Lapack_LU_inverse.c:347-444`)は、cuSOLVER ではなく **cuBLAS のバッチ API** `cublasZgetrfBatched` / `cublasZgetriBatched` (batch=1)で実装されている。A は OpenACC `deviceptr` 、特異行列検出時は abort、結果は `acc kernels` で書き戻す。しかし公開関数 `Lapack_LU_Zinverse` からの呼び出しは `/* LU_Zinverse_GPU(n, A, handle); */` とコメントアウトされており、既定は従来の CPU 実装 (LAPACK zgetrf/zgetri)のまま。呼び出し元は NEGF の表面グリーン関数計算 (`TRAN_Calc_SurfGreen.c` ほか)であり、**将来 NEGF 系の複素逆行列を GPU 化するための布石**である。

5.9 cuSOLVER(gpusolver 層)使用上の注意点

1. **ワークスペース量の見積り** — n×n 複素行列 1 本で $16n^2$ バイト、syevdx ワークスペースは見積りでなく `_bufferSize` の実照会値を使うこと(実測で見積り $2n^2$ に対し実際 $4n^2$ だった例がある)。各プランナのモデル(§5.5)を流用するのが確実。
2. **`wait_cudafunc` の無限ループリスク** — §3.7 と同じ。恒久的エラーではハングする。「満杯になり得る」確保には `TryDeviceMalloc` 系(8 回上限)を使うのが現行の作法。
3. **devInfo 検査の不均一** — 表 5-2 のとおり、収束失敗からの回復可能性が経路により異なる。Band/Cluster/DC-LNO/dense_utils 系は「数値的失敗=即終了」である(VRAM 不足は事前に ELPA へ逃げる)。
4. **フォールバックの階層** — (a) デバイス不在: 起動時に全体が ELPA2 へ降格。(b) VRAM 不足: 経路別の fits 判定で ELPA/CPU へ(§5.6)。(c) 数値的失敗: 表 5-2。この 3 層を混同しないこと。
5. **対称性と 0/1-based の変換** — 全呼び出しが uplo=LOWER の対称/エルミート前提。固有値配列の 0/1-based ずらし(`W[l]=W[l-1]`)が各所で必要で、dense_utils が吸収している。
6. **S の条件数処理が経路で不統一** — §5.3 の警告参照。
7. **再現性** — 変換 GEMM は GEMMul8 で実行されるため丸め挙動が cuBLAS FP64 と異なり、メモリ状況による フォールバックで実行ごとに経路が変わり得る(対策は §4.8-2)。加えて GPU⇄ELPA のフォールバックでも 結果のビットパターンは変わる。verdict は sticky なので 1 回の SCF 内では揺れない。
8. **fits 判定は集団操作** — 判定関数の内部で `MPI_Comm_split_type` / `Allreduce` / `MPI_Barrier` を呼ぶため、**全ランクが同じ回数だけ到達する必要がある**。経路を改修する際に 片側のランクだけ判定をスキップさせるとデッドロックする。

6. OpenACC と生 CUDA の使い分け

6.1 全体像

ループ並列のオフロードは NVHPC の OpenACC(`-acc -Minfo=accel`)で記述されている。要点:

- **GPU 専用の `#ifdef` ガードは存在しない**。CUDA ヘッダ(`cublas_v2.h` 等)は `openmx_common.h` で無条件 include されるため、**このツリーは CPU 専用にはビルドできない** (CPU 用はオリジナルまたは `makefile.cpu_intel.bak` を使う)。ゲートはすべて実行時フラグである。
- `-gpu=ccXX` 指定はなく、ターゲット CC はビルドマシンの GPU 自動検出に依存する。**managed memory は不使用**。`async` 節・`acc_get_cuda_stream` も全コードで不使用。
- `#pragma acc` を持つファイルは約 30。最厚は `Divide_Conquer.c` (66 箇所)、`Band_DFT_NonCol.c` (61)、`Force.c` (49)、`Band_DFT_Col.c` (46)、`Cluster_DFT_NonCol.c` (43)、`Set_ProExpn_VNA.c` (21)。**`Krylov.c` は OpenACC ゼロ**(`-acc` なしでコンパイルされ、生 CUDA のみ)。
- 基本形は `parallel loop gang` + `loop vector reduction`。密度行列構築・格子トレースなど 再現性が重要な縮約は `loop seq` (逐次)や「float 積 → double 加算」のキャスト位置まで含めて CPU と加算順序を一致させる。

6.2 データ常駐の 4 パターン

1. **呼び出し毎の一時 data 領域(copyin/copyout)** — `Total_Energy_GPU`、`Set_Nonlocal` など。OpenMX の多次元ポインタ配列はデバイスで辿れないため、**必ずホスト側で 1 次元フラット配列に詰め替えてから転送する**設計で統一されている。
2. **enter data / exit data + update による SCF スコープ常駐** — Band_DFT_NonCol の root-dense 経路、Set_Hamiltonian の SCF 不変テーブル(present_or_copyin 意味論により、常駐済みなら転送されない。§7.7)。
3. **生 `cudaMalloc` + `deviceptr` ブリッジ** — `Band_DFT_Col.c` / `Cluster_DFT_Col.c` は static コンテキストの `d_H/d_S` (cudaMalloc 領域)に対し、OpenACC 側から `#pragma acc parallel loop deviceptr(d_H)` + `acc atomic update` で 疎→密の scatter-add を実行する。
4. **単一確保のアリーナ + `acc_map_data` / `deviceptr`** — 現行実装で最も新しいパターン。複数のバッファを 512 バイト整列で 1 つの `cudaMalloc` / `acc_malloc` 領域に区画割りし、OpenACC には `acc_map_data` (Cluster_DFT_NonCol のアリーナ)または同名シャドウポインタ+ `deviceptr` 節(Force のバッチ、Mixing_H の履歴リング)で見せる。確保が 1 回になることで (a) 予約の成否が原子的に決まり、(b) 共有デバイスでの部分確保・部分失敗という中間状態が消える。

ライブラリへのポインタ受け渡しは `host_data use_device` で行う。`present` で常駐を表明したうえで、ブロック内では配列名がデバイスアドレスに置換され、cuSOLVER/cuBLAS/GEMMul8 にそのまま渡せる(H2D/D2H 転送なし)。

gpusolver_Syevdx.c — OpenACC 常駐配列を cuSOLVER に直接渡す典型形

```
#pragma acc data      present(A[0 : m * m])
#pragma acc data      present(W[0 : MaxN])
#pragma acc host_data use_device(A, W)
    {
        cusolverDnHandle_t cusolverH = NULL;
        wait_cudafunc(cusolverDnCreate(&cusolverH));
        cudaStream_t stream = NULL;
        wait_cudafunc(cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking));
        wait_cudafunc(cusolverDnSetStream(cusolverH, stream));
```

> **注意(present エラー):** `_openacc` 系ラッパは呼び出し側が data 領域を確立していることを 前提とする。確立せずに呼ぶと NVHPC ランタイムの FATAL(present table lookup failed)で落ちる。現行コードは「呼び出し側が必ず enter data を先行させる」規約で統一されている。

6.3 同期の設計と OpenACC メモリプールの扱い

OpenACC の既定キューとライブラリ用 CUDA ストリームは接続されていない。順序保証は (1) ライブラリ呼び出し直前の `#pragma acc wait`、(2) ライブラリ用ストリームの `cudaStreamSynchronize`、(3) 同期 `cudaMemcpy`、の明示同期のみで行う。性能よりも正しさと単純さを優先した設計である。

共有 GPU 運用で重要なのが **NVHPC の OpenACC メモリプール(フリーリスト)の性質**である。`exit data delete` や `acc_free` で解放したブロックはプロセス私有のプールに戻るだけで CUDA には返らず、しかも**同一サイズの要求にしか再利用されない**。この 2 つの性質が共有デバイスで実害を生む: (a) 他ランクから見ると「解放したはずのメモリが空きに現れない」、(b) サイズが微妙に異なる確保を繰り返すと プール内にブロックが座礁して実質リークになる(Force3 のチャンクバッチで実際に OOM を起こした。コミット `28361f7e`)。このため現行コードは、(i) 空き VRAM を測る前に必ず `acc_wait_all(); cudaDeviceSynchronize(); acc_clear_freelists();` でプールを CUDA に返却し、(ii) サイズが変動する確保は per-chunk 最大値で 1 回だけ確保して再利用する、という作法で統一されている。唯一の例外として、Mixing_H の preflight は**意図的に `acc_clear_freelists()` を呼ばない** (SCF サイクル中にプールをドレインすると常駐 GPU モジュールの確保動態が乱れ、約 30 反復後の Set_Hamiltonian で OOM を誘発した実測に基づく。コード内コメントに明記)。

6.4 set_hamiltonian_gpu.cu — 結線済みの生 CUDA カーネル

現行ツリーで唯一の「.c から呼ばれる生 CUDA カーネル」が `set_hamiltonian_gpu.cu`(194 行、コミット `95fa51ed` で追加)である。Set_Hamiltonian の dVH+Vxc+VNA 格子積分の内核 `matrix_elements_kernel` 1 本と、extern "C" ラッパ `Set_Hamiltonian_Cuda_MatrixElements()` を含む。設計上の要点:

- 1 ブロック行 = 1 原子ペア、block=256 スレッド。共有メモリに「重み付き軌道タイル」(double、GridVol·Vpot·Orbs0 を前計算)と「orbs1 タイル」(float)の 2 枚を置き、グリッド点を 32 点刻みのタイルで回して積和する。float→double のキャスト位置・積和順序は CPU/OpenACC 経路と一致(ビット同一)。
- 戻り値は 4 値: 0=成功、1=共有メモリ不足でカーネル非対応(タイル幅を 32→1 まで半減して試行した後) → OpenACC カーネルへ、2=一時的失敗だがコンテキスト健全(stale エラー対策後の再分類)→ 同一バッチを OpenACC カーネルでやり直し、負値=致命的 → abort。
- 起動前に `cudaGetLastError()` で他コンポーネントが残した stale エラーを掃除する (§3.6 のエラーラッチ規約の実装例。コミット `c5eb6845`)。
- ビルドは GEMMul8 と同じ nvcc(`NVCC_GEMMUL8_FLAGS`)でコンパイルされ、`OBJS` にリンクされる。`OPENMX_SETHAM_CUDA_KERNEL=0` で無効化でき、その場合は同一計算の OpenACC カーネルが走る。

> **撤去された休眠カーネル集:** 第 2 版で「検証済み・未結線」として記載した `openmx_cuda_kernels.cu/.h` (git 未追跡・ビルド対象外の生 CUDA カーネル集)は、現行ツリーでは**ディスクからも削除済み**である。当時のカーネルのうち Set_Hamiltonian 系の発想は `set_hamiltonian_gpu.cu` として結線された形になった。

6.5 NVHPC 固有の注意 — ミスコンパイル回避の実例

NVHPC 26.3 の OpenACC(`-acc`)には特定コードを壊す既知の不具合があり、Makefile は**ファイル単位で `-acc` を外す**ことで回避している(理由コメント付き)。これは NVHPC のバージョンを変える際に必ず再検証すべきポイントである。

ファイル	コンパイル指定	Makefile コメントの理由(要約)
`Occupation_Number_LDA_U.c` / `Eff_Hub_Pot.c` / `Total_Energy.c`	`CC_NOACC` (-O1, -acc なし)	NVHPC 26.3 + -acc が LDA+U Type2/cFLL の占有数・ポテンシャル経路をミスコンパイルする
`zero_cfrac.c`	`CC_NOACC`	-acc が DSYGV を失敗させ、ON2 法の極を壊す
`RestartFileDFT.c`	`CC_NOACC`	過分割 MPI 実行のリスタート I/O で segfault し得る
`MD_pac.c` / `Generate_Wannier.c`	`CC_NOACC`	SCF 後の MD 制御 / Wannier 生成経路で segfault し得る
`Krylov.c`	`CC_NOACC_O3` (-O3, -acc なし)	GPU 化は生 CUDA API のみで行う(OpenACC 不使用)
`Band_DFT_NonCol.c` / `Divide_Conquer.c`	`CC3` (-O2)	最適化レベル調整
`truncation.c` / `OutData.c`	`CC2` (-O1)	同上
`DFTD3vdW_init.c`	`CC_DFTD3` (-O0)	同上

表 6-1: ファイル別コンパイル調整(NVHPC 26.3 不具合回避を含む)

このほか、`Total_Energy.c` を `-acc` なしでビルドする必要から、OpenACC カーネル部分だけを 別翻訳単位 `Total_Energy_GPU.c` (`-acc` 付きでビルド)に分離するという構成が生まれている(§7.10)。また `Force.c` には旧 NVHPC の OpenMP バグ回避として `#if NVHPC_VERSION >= 250000` の 条件付きで OpenMP 並列を再有効化する箇所がある。`Divide_Conquer.c` の `if (use_dc_openacc)` ガードには「// compiler's bug」というコメントが残る。

7. 計算経路別の GPU 実装詳細

7.1 バンド計算(コリニア)— Band_DFT_Col.c

最大規模の改修対象(対オリジナル +4,724/−2,544 行)。GPU 密行列経路の条件は `scf_eigen_lib_flag==GPUSOLVER && n ≥ 1000` に加え、コミット `a4200a04` 以降は **`BandCol_GpuDenseFits` による 1 ターン分の VRAM 見積り**が成立すること(不成立なら ELPA/ScaLAPACK へ。§5.6)。特徴:

- **密行列構築キャッシュ:** 疎行列(H, S)から k 点ごとの密行列を組み立てる部分を OpenACC 化し、ペア構造の「fingerprint」付きキャッシュ(`entries` / `phase_r` / `phase_i`)を `enter data` で常駐させる。位相のみ `update device` で更新。scatter-add は `deviceptr` + `acc atomic update`。Set_Hamiltonian が構築した packed H/S キャッシュ(§7.7)から直接行列を集約する。
- **対角化と変換:** S 変換行列(Ũ)は初回に計算してホスト側 `transformed_s` に退避し、SCF 反復間で再利用。H′ の構成は GEMMul8 Zgemm × 3(§4.6)、対角化は `cusolverDnXsyevdx`。固有ベクトルは DM フェーズまでデバイス常駐。
- **固有値のみの第 1 パス:** `all_knum != 1` の第 1 対角化ループ(化学ポテンシャル決定用に固有値だけが要る)は `jobz=CUSOLVER_EIG_MODE_NOVECTOR` で解き、固有ベクトル書き出しと逆変換 GEMM を丸ごと省く(`BandCol_GpuSolver_SolveEigenvaluesDeviceOnly`。`OPENMX_BAND_SYEVDX_NOVECTOR=0` で旧動作。コミット `58ea6471`)。この経路は `na_rows==n && na_cols==n` を要求する。
- **適応並列度:** §5.5.1 のとおり。k 点ウェーブの初期値は「そのデバイスを共有する k オーナーランク数」(コミット `48921b36`)で、VRAM に収まらないときだけ縮退する。全ランクが収まる場合はターン分割・途中バリア・GEMMul8 解放点が消える。決定はフル並列時の必要量として `OpenMX_GpuPhaseNeed_Register("band_col", …)` で公開され、Set_Hamiltonian の常駐キャッシュが道を空ける(§7.7)。
- **DM フェーズ:** OpenACC の `gang` + `loop seq` 縮約で、加算順序を CPU と一致させる。
- SCF 反復間のデバイスバッファ常駐は既定 OFF(`OPENMX_BAND_GPU_PERSISTENT`)、ターンごとの GEMMul8 ワークスペース解放は既定 ON(`OPENMX_BAND_GEMMUL8_TURN_RELEASE`)。

7.2 バンド計算(非コリニア)— Band_DFT_NonCol.c

- OpenACC 常駐型(`enter data` 中心)の root-dense 経路。位相付き密行列構築(scatter-add)、S 対角化+S^−1/2、2n×2n スピノル H2/S2 の組立、U†HU(GEMMul8 Zgemm)、固有ベクトル逆変換、DM/EDM(`loop seq`)までを GPU で実行。
- **ステージ別適応並列度(§5.5.2):** 固有値のみ/固有ベクトル/DM の 3 モードで別々に VRAM を見積り、それぞれ独立に同時ランク数を決める(コミット `7c414f91`, `99f4014b`)。DM 蓄積専用の見積り `BandNonCol_RootDenseDMDeviceBytes` はソルブ段よりはるかに小さく、「ソルブ 2 並列・DM 8 並列」のような 運転になる。フェーズ需要は `"band_noncol_ev" / "band_noncol_vec" / "band_noncol_dm"` の名前で公開。
- **ELPA フォールバック:** `BandNonCol_GpuDiagFits`(1 ランク分すら収まらない場合)。強制は `OPENMX_BAND_GPU_DIAG`(§5.6)。
- OpenACC キューとの順序は `#pragma acc wait` → `openmx_gemmul8Zgemm` → `cudaStreamSynchronize` で明示制御。

7.3 クラスタ計算 — Cluster_DFT_Col.c / Cluster_DFT_NonCol.c

クラスタ経路は第 2 版以降で最も密度の高い改修を受けた(Col: 対オリジナル +1,534 行、NonCol: +2,424 行)。

Col — root-dense 経路の常駐化と GPU DM

- **発動判定は実予約(§5.5.3、§5.6):** `ClusterCol_GpuDiagFits` が 3 値を返す — 0 = ELPA フォールバック、1 = 並行 GPU(各スピンワールドが自スピンを解く)、2 = 直列化 GPU (スピンワールド 0 のオーナーが両スピンを解き、リモートスピンの結果を MPI で送る。コミット `bf08a9d8`)。直列化モードは packed Set_Hamiltonian キャッシュが前提(非 packed の行列集約は level1 全体の集団通信のため 片オーナーだけでは呼べない)。
- **DM/EDM の GPU 蓄積(コミット `75fc30d3`):** `ClusterCol_AccumulateDM_OpenACC` が 131,072 エントリ単位の OpenACC gang ループで DM1/EDM1(partial charge 時は PDM1 も)を蓄積する。状態内の総和順序は CPU ループと同一(`loop seq`)。`OPENMX_CLUSTER_GPU_DM=0` で CPU ループへ。2026-10-13 からは密な求解を持つランクだけでなく全ランクが自分の状態スライスをデバイスで蓄積する(他ランクはホストの パネルを半減リトライでステージし、最小ブロックも入らなければ原子でスレッド分割した CPU ループへ)。`OPENMX_CLUSTER_GPU_DM_ALL=0` でオーナーだけの蓄積に戻る(C1000・6 ランク: 対角化段階 113 → 78 s、密度行列のホスト時間 37 → 6 s)。疎→密 index は幾何指紋(FNV-1a)付きでキャッシュされ、スピン×反復ごとの再構築・再アップロードを排除。
- **固有ベクトルのデバイス常駐(コミット `527ed524`):** ソルブ直後の固有ベクトルパネルを スピン別に D2D コピーで stash し(`ClusterCol_StashDeviceEvec`)、DM 蓄積カーネルが `deviceptr` で直接読む(EVec1 の D2H → 再 pack → H2D 往復を排除)。stash の確保は `TryDeviceMalloc` で、失敗時は黙ってスキップ(最適化専用)。非オーナーランクの固有ベクトルスライスの ステージングは、収まるまで状態ブロックを半減(下限 128)し、最小でも不可なら CPU ループで続行する。
- ソルブ終了ごとに GEMMul8 ワークスペースを解放(`OPENMX_CLUSTER_GEMMUL8_TURN_RELEASE`、未設定時は BAND 設定を継承)。旧経路 `ClusterCol_GpuSolverDensePath` (CPU dgemm 使用)と ホストバッファ経路は削除された。

NonCol — 単一アリーナと反復間キャッシュ

- **単一 cudaMalloc アリーナ(コミット `3ef5347a`):** S、Ss2、作業行列 7 枚、Hs2、密固有ベクトル、疎→密 index/staging を 512 B 整列で 1 領域に区画割りして予約し、`acc_map_data` で OpenACC に見せる。予約不成立なら ELPA へ(§5.6)。
- **反復あたりオーバーヘッド削減(コミット `a9b6fa31`, `f5de630e`):** cublas ハンドルのデバイス別キャッシュ(旧: 反復あたり 13 回 create/destroy)、変換済み S とスピノル展開 Ss2 の デバイス常駐(216 原子で反復あたり約 570 MiB の再アップロードを削減)、zheevdx ワークスペースの永続コンテキスト化、ホスト側スクラッチ約 1.6 GiB の反復間再利用、逆変換 GEMM の m を n2 から MaxN に縮小 (明示 leading dimension 版へルパ `ClusterNonCol_GEMMul8ZgemmLd_OpenACC`)。
- 密固有ベクトル `dense_evec` はデバイス常駐キャッシュされ、後段へは `DFT.c` から `Cluster_DFT_NonCol_ScatterGpuSolverCachedEVec` で配布。メモリ圧迫時は `Cluster_DFT_NonCol_DemoteGpuSolverCachedEVec` がホストへ退避させる(密度グリッド GPU サービスが オーナー確保に失敗しそうなときに呼ばれる。§7.9)。
- MaxN(部分固有値数)は SCF 第 1 反復のみ全数、以後は占有数+マージン(§5.3)。

7.4 DC 法 — Divide_Conquer.c

- 原子ごとの部分クラスタ行列(局所次元 NUM)を各ランクが独立に処理する。GPU 化条件は コリニアは `NUM ≥ DC_GPU_Threshold()` (既定 400、`OPENMX_DC_GPU_THRESHOLD` で変更可)、非コリニアは `2·NUM ≥ DC_NonCol_GPU_Threshold()` (既定 300、`OPENMX_DC_NONCOL_GPU_THRESHOLD`。CPU の zheev がコリニアの dsyevx よりはるかに遅いため、GPU がより小さい行列から優位になる)。
- **Col 側は生 CUDA 実装:** `DCGpuSolverCtx` に cudaMalloc バッファ+pinned ホストバッファ (`cudaMallocHost`、本実装唯一の使用箇所)。S 対角化(SCF 2 反復目まで)→ H′ 構成 (`DC_GpuSolver_TryGpuDgemm`: GEMMul8 → 失敗時 cublasDgemm → それも失敗なら CPU)→ 活性部分空間を `cudaMemcpy2DAsync` でデバイス内切り出し → Xsyevdx。
- **S12 パネルのデバイス常駐キャッシュ(S-cache、コミット `6f7f31b2`):** 変換済み重なり行列 S12 = U·λ^−1/2 は SCF 3 反復目以降不変であることを利用し、per-rank の単一 cudaMalloc アリーナに常駐させる。採用は「予算 =(空き − 予約

4096 MiB)/ ノード内ランク数」の範囲で貪欲に(他フェーズを追い出さない polite 方式)。ヒット時は「ホスト S_DC 再充填 → 再パック → PCIe アップロード」を丸ごとスキップし、GEMM が常駐コピーを直接読む。CPU フォールバックが必要になった時だけ `DCCol_FillSDCFromS12` でホスト側を遅延復元する。`OPENMX_DC_GPU_S_CACHE=0` で無効化。採用状況は `<DC> GPU S-cache: rank %d keeps %d of %d cluster overlaps on the device (%.1f MiB)` と報告される。

- **ホスト転置の廃止:** ストライドアクセスの O(n^2) ホスト転置を、行連続 memcpy のパック+ `cublasDgeam` によるデバイス内転置(`DC_GpuSolver_DeviceTranspose`)に置換。geam は純粋なデータ移動なので後段への入力はビット一致。
- **preflight の省略:** 一度承認された GEMM 形状以下の解では `cudaMemGetInfo` preflight をスキップ (GEMMul8/cuBLAS のワークスペースは grow-only で、小さい形状が新規確保を起こさないため)。
- **多層の安全装置:** 確保前の VRAM preflight(`GPUSOLVER_MIN_FREE_AFTER_MB` 既定 1536 MiB)、ホスト malloc 失敗検出、固有値分解失敗検出 — いずれかで **sticky 無効化フラグ**を立て、以後そのランクは CPU の DC ソルバで継続する (GEMMul8 ワークスペースも解放)。CPU フォールバック前には CUDA エラーラッチを掃除する(§3.6)。
- **NonCol 側は OpenACC 実装:** S_DC/H_DC/C を連続ストア+行ポインタ化して data 節を 1 ブロック転送にまとめ、対角化は `gpusolver_Syevdx_Complex_openacc_cached` (ハンドル・ワークスペースを原子間で再利用)経由。64 原子 Si・16 SCF で対角化バケットが 78.4 s → 66.4 s(コミットメッセージの実測)。

7.5 DC-LNO 法 — Divide_Conquer_LNO.c

- 現行は **direct per-rank dispatch**: 全ランクが `cudaSetDevice` を実行し、自分の局所問題(コリニア: NUM ≥ 800 = `GPU_CPU_SWITCH_NUM2` 、`OPENMX_DCLNO_GPU_THRESHOLD` で変更可。非コリニア:スピノル次元 ≥ 400 = `DCLNO_NONCOL_GPU_CPU_SWITCH_NUM` 、`OPENMX_DCLNO_NONCOL_GPU_THRESHOLD` で変更可 — CPU の zheev が大幅に遅いため別々に実測して分離した)を 直接 GPU に投入する。旧方式の GPU プロキシ機構(rank 0 が他ランクの対角化を MPI タグ 41001-41004 で代行)は `DCLNO_ENABLE_SHARED_GPU_PROXY 0` で温存。
- **非コリニアも GPU 化(コミット `d62cac60`):** `DCLNO_Solve_NonCol_GpuSolver` が クラスタごとの一般化固有値問題全体をデバイスで実行する。手順: NUMH×NUMH の S を zheevdx で対角化 → `ko = 1/√\|λ\|` → `cublasZdgmm` + `cudaMemcpy2DAsync` のブロックコピーで スピン倍化変換 T = diag(Ũ, Ũ) を構成 → H′ = T † HT を GEMMul8 Zgemm 2 本 → `cublasZgeam` で 共役行列を作成(CPU フローが対角化するのと同じ行列にして数値互換を確保)→ NUM2 本を zheevdx → 逆変換を転置形 Zgemm 1 本で residue ステージのレイアウトのまま書き出し。有効化は `use_gpu_accel_nc && Eigen_MPI_flag==0 && NUM ≥ しきい値` 。
- **デバイス共有ランク群の合算メモリ判定(§5.5.4):** `DCLNO_GpuGroupMemoryFits` が不成立なら グループ全体で CPU 固有値ソルバへ(コリニア・非コリニア両方)。
- **フェーズ末尾のメモリ返却:** `DCLNO_GpuTurnRelease()` (既定 1、`OPENMX_DCLNO_GPU_TURN_RELEASE=0` でオプトアウト)が各フェーズ末尾でソルババッファと GEMMul8 ワークスペースを解放し、グリッド/力フェーズにメモリを返す(ハンドルは維持)。
- ノード内 GPU 台数は `scf.Gpu.Num` (既定 30 ≒ 実質無制限)で上限化。数値的失敗時のフォールバックはなく abort(表 5-2)。216 原子 Si(クラスタ次元 ~2222)の NC 実行は np=2 で約 13 分(CPU 解は 1 時間超)。

7.6 Krylov 法 — Krylov.c

- **OpenACC 不使用・生 CUDA のみ** (`-acc` なしの `CC_NOACC_O3` でコンパイル)。OpenMP スレッドごとの `Krylov_GPU_Workspace` プール(d_A/d_B/d_C/d_W/d_work + 専用 non-blocking ストリーム)を持つ。
- **原子別パイプラインの融合(コミット `3ee45228`):** 従来は原子ごとに「基底再パック&アップロード × 3 + GEMM ごとの H2D/D2H 往復 + ホスト対角化」だったが、`Krylov_ProjectedSolve_GPU` が 1 回のデバイス訪問に融合した: ① 基底 d_KU を KU キャッシュから取得(ミス時のみ再パック&アップロード)、② H_DC を 1 回だけアップロード、③ C = H·u1(GEMMul8)、④ u1 † Hu1(GEMMul8、m3×m3 だけダウンロード)、⑤ ホストで ZeroNum 補正と EC_matrix 加算

(CPU コードと同一)、⑥ `cusolverDnXsyevd` で対角化(固有ベクトルパネルは 逆変換が読むレイアウトのままデバイスに残す。失敗時は LAPACK にフォールバックして再アップロード)、⑦ 逆変換 c = u1·b(GEMMul8)→ ダウンロード。

- **発動条件:** 3 つの GEMM すべてが `m·n·k ≥ 5×10⁷` (`OPENMX_KRYLOV_GPU_DGEMM_MIN_FLOPS`)を 満たすこと。これにより「per-call パスと同じ GEMM だけが GPU に行く」条件が保たれ、GEMM のディスパッチは旧コードと **ビット一致**する。対角化のデバイス実行は `Msize3 ≥ 300` (`OPENMX_KRYLOV_GPU_EIGEN_MIN` 。cusolver と LAPACK の固有ベクトルは最下位ビットで異なるため、この条件を満たす計算は CPU 対角化とビット同一には ならない — クロスオーバー実測では Uele の一致は Msize3=455 まで $\le 2\times10^{-9}$、Msize3=715 で 2×10^{-6})。融合パス自体は `OPENMX_KRYLOV_GPU_FUSED=0` で無効化でき、その場合は従来の per-call シーケンスが走る。
- **KU キャッシュ:** 再パック済み Krylov 基底パネル(SCF 1 反復目に生成、以後不変)を per-rank アリーナに デバイス常駐させる。採用は DC の S-cache と同じ polite 方式 (予算 =(空き − 4096 MiB)/ ノード内ランク数、 `OPENMX_KRYLOV_GPU_KU_RESERVE_MB`)。 `OPENMX_KRYLOV_GPU_KU_CACHE=0` で無効化。採用状況は `<Krylov> GPU KU-cache: rank %d keeps %d of %d basis panels on the device (%.1f MiB)` と報告。 解放は力計算直前の `Krylov_Release_GPU_KUCache()` 。
- S 変換は Krylov 基底構成(CPU の `Inverse_S_by_Cholesky`)で処理済みのため、GPU 側は標準固有値問題のみを扱う。 `OPENMX_KRYLOV_GPU_PROFILE=1` で融合パスの内訳 (`KRYFUSE n=… ku=… g1=… eig=…`)が出力される。

7.7 行列要素構築 — Set_Hamiltonian.c / set_hamiltonian_gpu.cu / Set_Nonlocal.c / Set_OLP_Kin.c

Set_Hamiltonian.c(対オリジナル +2,972/−1,149 行)+ set_hamiltonian_gpu.cu(194 行)

第 2 版時点では「格子積分の OpenACC 版は実装済みだが実測で遅く恒久無効」だったが、コミット `95fa51ed` 以降、dVH+Vxc+VNA の格子積分(`Calc_MatrixElements_dVH_Vxc_VNA`)は **既定で GPU 実行**になった(`OPENMX_SETHAM_MATRIX_GPU` 、既定 1)。転換を支えたのは次の 4 機構である。

1. **SCF 不変キャッシュ:** ペアメタデータ・グリッド索引(nolg_MN/nolg_Nc)・軌道値バッファ (orbs0buf/orbs1buf、float)を SCF を通じて 1 回だけ構築するホスト側キャッシュ (`SetHamiltonianMatrixElementsCache`)。毎反復転送されるのは H ブロック(hbuf)と ランク局所の Vpot スライスだけになった。
2. **オンデバイス Vpot ギャザー(コミット `bff0af37`):** 旧実装のホスト側「spin×重なり点数」展開を廃し、 `Vpot_Grid` のランク局所スライスをそのまま転送してカーネル内で間接参照する。丸め順序は 旧展開と同一でビット同一。
3. **専用 CUDA カーネル(§6.4):** 共有メモリに「GridVol·Vpot·Orbs0 の重み付きタイル(double)」と「orbs1 タイル(float)」を置くタイル型カーネル `matrix_elements_kernel` 。 `OPENMX_SETHAM_CUDA_KERNEL=0` で OpenACC カーネルに切替可能(数値は同一)。
4. **適応 GPU ターン計画(`Set_Hamiltonian_CreateGpuTurnPlan`):** §5.5 と同じ UUID 共有群+降順プレフィックススキャンで同時実行ランク数を決める。既定は**ハイブリッド波**: 第 1 波のランクが GPU で計算する間、残りのランクは同時に CPU で計算する (`OPENMX_SETHAM_GPU_SERIAL_WAVES=1` で全波 GPU 直列に変更可)。既定では H の局所セグメントを持つ**全ランクが GPU 作業候補**(旧 k オーナー限定ポリシーは `OPENMX_SETHAM_GPU_KOWNER_ONLY=1` で復元)。

常駐キャッシュと「需要への譲歩」(コミット `bff0af37` , `69dde552`): そのデバイスを共有する全ランクが単一波で走れる場合に限り、SCF 不変テーブル (orbs1/orbs0/nolg/meta の 4 クラス、サイズ降順) をデバイス常駐させる(`OPENMX_SETHAM_GPU_RESIDENT` 、既定 1)。入場予算は「空き −(公開済みフェーズ需要)− フロア」で、フロアは対角化需要が公開済みなら 4 GiB、未公開なら 16 GiB (`OPENMX_SETHAM_GPU_RESIDENT_FLOOR_MB` で上書き)。バンド系ソルバが `OpenMX_GpuPhaseNeed_Register("band_…")` で公開した需要を毎回照会し、空きが需要を下回ると**同一 SCF 反復の対角化の直前に自主退去**して、その MD ステップの残りは transient 運転に切り替わる。 `copyin` は present_or_copyin 意味論なので、常駐の有無・部分常駐に かかわらず数値はビット同一である。関連の stdout:

```
<Set_Hamiltonian>  Hamiltonian matrix for VNA+dVH+Vxc (GPU-accelerated)...
<Set_Hamiltonian> GPU device %d: %d Hamiltonian rank(s), GPU concurrency=%d,
    serialized waves|CPU fallback ranks=%d, peak=%.3f GiB, free=%.3f GiB, reserve=%.3f GiB
<Set_Hamiltonian> GPU resident cache: %d rank(s) hold %.3f GiB (orbs1+orbs0+nolg+meta) on device %d across SCF iterations
<Set_Hamiltonian> GPU resident cache released: a GPU phase needs %.3f GiB on device %d; staying transient for this MD step
```

幻の失敗の修正(コミット `c5eb6845`): 満杯に近いデバイスでは、正常に回復した他コンポーネントの 確保失敗(GEMMul8 の FP64 フォールバック、`wait_cudafunc` のリトライなど)が CUDA の スレッド別エラーラッチに残り、CUDA カーネルの検査コードが自分の失敗と誤認して abort していた (SCF 316 反復目で「CUDA matrix-elements kernel failed with status -2」)。現行は、カーネル起動前に stale エラーを pop し、失敗時もコンテキストが健全なら「回復可能(status 2)」に分類して **同一バッチを OpenACC カーネルでやり直す**(部分更新された可能性のあるデバイス H ブロックは ホストコピーから復元)。abort するのはコンテキスト汚染型のみ。

- **base 結合(H = F_Kin・H0 + …)の GPU 版**は引き続きオプトイン(`OPENMX_SETHAM_BASE_GPU`、実測 10 ms vs CPU 2 ms のため)。
- **packed H/S キャッシュ** (GPUSOLVER 用に H と S を 1 次元 packed 形式で保持する API 群、`Set_Hamiltonian_Build_GpuSolver_HS_Cache` ほか)は従来どおり Band/Cluster ソルバの密行列組立を支える。クラスタ 2 スピン直列化モード(§7.3)はこのキャッシュが前提。
- **共有テーブルの公開:** 密度グリッドの分散求積(§7.9)が同じ軌道値テーブルを読めるよう、読み取り専用ビュー `Set_Hamiltonian_GetMatrixElementsTables()` と、テーブルを構築せずに 存在だけ確認する `Set_Hamiltonian_MatrixElementsTables_Ready()` (数 GiB の遅延構築を誘発しない プローブ。コミット `ae3dd189`)を提供する。
- プロファイル: `OPENMX_BAND_PROFILE=1` で `SETHPROF` / `SETHMEPROF` 行を出力。

Set_Nonlocal.c(対オリジナル +1,738/−305 行)

第 2 版時点は「動径 k 積分ループの OpenACC 化」だったが、コミット `211f0684` で **非局所射影子積分全体のバッチ GPU 化** に置き換わった(`SetNonlocal_CalcDSNL_OpenACC`)。構成は 3 フェーズ:

1. **ホスト準備:** 種ごとの動径テーブル(基底/射影子の Fourier 変換)と共通 k グリッドを OpenMP で構築し、バッチの単位となる「行」=(原子ペア, 微分方向 so, 角運動量 LL, 基底×射影子コンボ)を列挙する。Wigner-3j 階乗和の数千万回再評価を避けるため **Gaunt 係数を事前表化**する(選択則 m = M0−M1 で軸を削減)。
2. **デバイス実行:** 球ベッセル関数表を**デバイス上で生成**(ホスト実装と同じ下降漸化式・リスケーリング・ 正規化の忠実移植 — 旧実装の「約 21 秒の直列ホスト Bessel 生成+チャンク転送」を除去)し、台形則の動径積分を gang×vector reduction でバッチ縮約する。球ベッセル表はチャンクあたり 96 MiB の デバイス予算に収まるようペアを分割。
3. **ホスト仕上げ:** 球面調和・Gaunt・複素→実変換は CPU パス同様の構造で OpenMP ペア並列。

フォールバックは `OPENMX_SETNL_OPENACC=0` (または非 GPUSOLVER)のみで、メモリ量による動的 フォールバックはない。実測 (216 原子 Si NC、np=2): DS_NL ステージ 23.7 s → 0.66 s。メタンの CPU 比較で最初の 8 SCF ステップがビット同一。あわせて固定サイズのスタック配列を オーバーフロー検査付きのヒープ確保に置き換える堅牢化も維持されている。

Set_OLP_Kin.c(+528/−180 行)と Spherical_Bessel.c

- 重なり・運動エネルギー行列要素の動径積分オフロードは、データ転送律速で CPU より遅い実測 (コメント: 約 1.2 s vs 0.3 s)のため `OPENMX_OLPKIN_RADIAL_GPU=1` の**オプトイン**のまま。さらにスレッド数 1(`Nthrds==1`)を要求する。
- CPU 側の実質的な高速化として、ペア距離 r の **bit パターン完全一致をキーとする球ベッセル関数テーブルキャッシュ** (結果は bitwise 同一、シングルスレッド時のみ)と、`Spherical_Bessel.c` のホットパスの 毎呼び出し malloc/free 除去は従来どおり。

7.8 VNA 射影子 — Set_ProExpn_VNA.c

第 2 版時点では「一度 GPU 化されたのち CPU 版へ戻され、オリジナルと差分なし」だったが、コミット `77d2a22c` で **+2,029 行の本格的な GPU バッチ化**が入った(216 原子 NC で 160.9 s → 4.4 s)。共通ゲートは GPUSOLVER かつ `OPENMX_SETPRO_GPU`(集団判定、既定 1)。3 つの独立なバッチからなり、それぞれ独立にメモリ判定して不可なら従来 OpenMP ループに落ちる:

1. **DS_VNA 構築バッチ:** 一中心積分 <φ|P_VNA> の全ペアを 256 ペアずつのチャンクで処理。4 カーネル構成 — ① 球ベッセル漸化のデバイス実行(`Spherical_Bessel2` の忠実移植)、② Gauss-Legendre 求積和、③ Gaunt × Y_lm 重み付き (LL,m) 和、④ 複素→実変換と float ストア。階乗ベースの Gaunt 係数と long double の複素球面調和は**ホストで表引き化**し、デバイスは参照のみ。求積は要素単位で CPU と同一の累積順を保持。メモリ条件は(空き − 256 MiB)/ノード内ランク数 > 1 GiB。
2. **HVNA トレースバッチ:** HVNA[Mc][j] = dmp·Σ ene·DS_VNA·DS_VNA のエネルギー重み付き float 内積を、MPI パイプラインで受信した halo ブロックのアーカイブ後に全 (Mc, j) 対を 1 カーネルで評価。ダンピングと格納はホスト(CPU ループと完全同順)。
3. **Set_VNA2 / Set_VNA3 バッチ:** <Φ|VNA> と <VNA|Φ> は「PAO 積側の種と方向ベクトルの符号を 入れ替えた鏡像」として同一カーネル群(mirror 引数)で処理。512 ペアずつのチャンク。

メモリ不足時の通知は `level.of.stdout` 2 でノードごと 1 回(コミット `b569f272`。level 3 では各ランクが 自身の数値を出力、§10.3): `Set_ProExpn_VNA GPU: not enough free device memory; CPU fallback.` DFT.c のステージバナーは `<MD=…> Calculation of the VNA projector matrix (GPU-accelerated)` となる (コミット `30148e1e`)。

これらの判定がノード内ランクで分け合う空き容量は、GPU 経路の他のすべてのデバイスメモリ予算と同じく `OpenMX_GpuMemGetInfo()`、すなわち NVML が報告する物理的な空き容量で頭打ちにした `cudaMemGetInfo()` から取る(§10.2 `OPENMX_GPU_MEMINFO_NVML`)。この頭打ちは、WSL2 上の 12 GB RTX 4070 Ti で 12 ランクがここで止まった事例から入った: WDDM ドライバは 12 個のコンテキストで残り 0.6 GiB の状態でも各ランクに「空き 10.8 GiB」を見せ、ランクあたり 836 MiB の DS_VNA バッチがシステムメモリへ 退避して、1 バッチが 0.4 秒から数分までばらついた。物理的な値を使うとバッチは 128 ペア(423 MiB)に縮み 2.7 秒で終わる。固定の 512 MiB で判定していた HVNA2/HVNA3 バッチも同じ方式になった: データ領域の実測見積り (`SetPro_VNA23_DeviceBytes`、Si s2p2d1 の 512 ペアで 36 MiB)をランクの取り分と比べ、収まるまでペアバッチを半減する。`OPENMX_SETPRO_VERBOSE=1` でこれらの判定値と各バッチの所要時間がランクごとに出る。

7.9 密度グリッド — Set_Density_Grid.c + Set_Density_Grid_GPU.c

電子密度のグリッド構築(ρ(r) = Σ DM·φφ)が新規ファイル `Set_Density_Grid_GPU.c`(1,366 行)で GPU 化された。**3 段のフォールバック連鎖**で動く(`Set_Density_Grid.c` のコメントに明記): **分散 GPU 求積 → 単一ランク GPU サービス → CPU/OpenMP ループ**。共通の有効化条件は GPUSOLVER かつ `Cnt_switch==0`(`OPENMX_DENSITY_GRID_GPU=0` で全体無効)。分散 GPU 求積は Cluster/Band ソルバに加えて 2026-10-09 から DC・Krylov・DC-LNO でも使う(どのソルバも Set_Density_Grid にFNAN 近接までの同じ DM を渡すので求積はソルバに依らない)。単一ランク GPU サービス(全格子の軌道値テーブルを 1 台のデバイスに作る)はCluster/Band のまま。

(a) 分散 GPU 求積(コミット `d14ef297`)

- 各ランクが自分の原子の原子球密度 `Tmp_Den_Grid` をデバイスで積分し、既存の A→B 通信に流す (オーナー直列化なし)。出力点ごとの CSR を CPU ループと同じ (h_AN, Nog) 順で辿り、加算順序も同一。
- **Set_Hamiltonian のテーブルを共有:** 軌道値テーブル (orbs0/orbs1)・グリッド索引は `Set_Hamiltonian_GetMatrixElementsTables()` の読み取り専用ビューで参照し、常駐済みなら `copyin`(present_or_copyin)はアタッチのみ — 毎反復転送されるのは CSR・密度行列・結果だけになる。

- モード(`OPENMX_DENSITY_GRID_GPU_LOCAL`): 0=無効、1(既定)=**共有テーブルが全クラス常駐のときだけ** 関与(毎回ステージングするとオーナーサービスより遅いため)、2=常に関与。モード 1 では `Set_Hamiltonian_MatrixElementsTables_Ready` の Allreduce(MIN) で「全ランクがテーブル構築済み」の ときだけ進む(CPU 実行ランクに数 GiB のホストテーブルを作らせないため。コミット `ae3dd189`)。さらにノード共有クォータ(空き − 256 MiB を ノード内ランク数で割った値)に収まることも要求。
- 参加決定は全ランクの Allreduce(MIN) で合意し、初回に `<Set_Density_Grid> distributed GPU quadrature: every rank integrates its own atoms` と表示(`level.of.stdout` 2 以上)。自身の一時需要は `OpenMX_GpuPhaseNeed_Register("density_grid_local", …)` で公開する。
- **その場計算のランクと作業領域(2026-10-11):** 常駐軌道テーブルのないランクは、バッチごとに デバイスで軌道を評価する。中央原子の軌道タイル全体が入るときは、複数の原子対で再利用する。対が作業領域を超えるときは、重なり格子点を分割し、その点だけで両原子の軌道を評価する。元の格子点順序を保ち、同じ H ブロック・密度格子点に加算する。行列カーネルの回復可能な失敗では、失敗した分割部分だけを CPU で計算する。分割軌道の評価と OpenACC の密度フォールバックは格子点を 複数の GPU gang に分配し、1 原子対でも GPU 全体を使う。したがって原子球全体のタイルを収める必要はない (Eu: 41 軌道、8.0 bohr。自身との対全体は 500 MiB を超えることがある)。
- 密度の既定作業領域は 128 MiB。大きい原子に対しては、空きメモリがあれば従来どおり準備段階で 全対を 1 バッチに収める大きさまで拡張する。これにより Eubcc8-Band の既存の高速化 (2041 MiB・1 バッチ・密度 1 ステップ 0.11 s)を維持する。拡張しても対全体が入らなければ、格子点を分割して GPU 計算を続ける。明示した `OPENMX_SETDEN_ONTHEFLY_MB` は上限を保ち、その範囲内で分割する。固定の密度行列・出力領域と実行時の余裕は別途予算化し、48 MiB という最低目安が 実際の空きメモリの取り分を超えることはない。
- それでも作業領域不足で参加できないランクは CPU ループを取り、残りのランクは GPU を続ける。全ランクが作業領域不足で辞退しても、単一 GPU に集約せず分散計算を維持する。分散実装が使えない `Cnt_kind` 1 や、ローカル求積を明示的に無効化した場合には、従来どおり単一ランク GPU サービスを利用できる。
- `level.of.stdout` 2 では、作業領域不足・デバイス失敗を状況が変わったときに Host_ID が表示する。全ランク辞退時も表示し、level 3 では各ランクの必要量と利用可能な容量も示す。通常実行や設計どおりのサービス経路は表示しない。

(b) 単一ランク GPU サービス(コミット `bff93d08`)

- オーナー(Host_ID)が幾何ごとに 1 回、全ランクの原子中心軌道グリッドと重なり位相表を `MPI_Gatherv` で収集してキャッシュ。以後の SCF 反復では **CDM だけ**を gather し、オーナーが正規グリッド全体の密度を CSR カーネルで構築して `MPI_Scatterv` で配布する。
- オーナーの確保が不足しそうなときは段階的に道を空ける: まず全ランクで GEMMul8 ワークスペース解放+ OpenACC プール返却、次に非コリニアクラスタの固有ベクトルキャッシュのホスト退避(`Cluster_DFT_NonCol_DemoteGpuSolverCachedEVec`)。それでも不足なら `Set_Density_Grid GPU owner needs … ; CPU fallback.` を出してそのエポックは CPU で計算。
- 既定ではデバイス側を計算直後に解放する(`OPENMX_DENSITY_GRID_GPU_PERSISTENT=0`。GEMMul8 の `cudaMemGetInfo` 判定を狂わせないため)。予約は `OPENMX_DENSITY_GRID_GPU_RESERVE_MB`(既定 256 MiB)。

7.10 全エネルギー — Total_Energy.c + Total_Energy_GPU.c

ファイル分離の理由: NVHPC 26.3 の `-acc` が LDA+U 経路をミスコンパイルするため、`Total_Energy.c` 本体は `-acc` なし(`CC_NOACC`)でビルドせざるを得ない。そこで OpenACC カーネルだけを別翻訳単位 `Total_Energy_GPU.c`(`-acc` 付き、現在 1,064 行)に分離した。

ゲートは `TotalEnergyUseOpenACC()`。第 2 版時点は「非共線(SpinP_switch==3)では無効」だったが、コミット `19b9b69c` で **NC 制限が撤廃**され、現在は GPUSOLVER なら常時有効(`OPENMX_TOTALENERGY_GPU=0` で全カーネル一括無効化)。GPU 化された格子縮約は 5 種:

1. **EXC/EH1 格子積分**(Ena・Eef・EH1・EXC を単一 `parallel loop reduction` で縮約)
2. **双極子モーメント 6 成分**(格子座標は GN→(n1,n2,n3) をデバイス内で再構成)
3. **CWF 二重計上補正**(dcEH1/dcEXC)
4. **EH0 二中心項のバッチ計算** — 全原子ペアを一括配列化し、species ごとの球面格子・VPS メッシュ・V_H 原子テーブルをフラット化して転送。`gang`(ペア)×`vector reduction`(格子点)で エネルギーと r 微分(力 #6/#7 相当)を同時計算。スプライン補間は `#pragma acc routine seq` の デバイス関数として実装。
5. **Exc0 細メッシュ補正のバッチ(新設、コミット `19b9b69c`)** — `Exc0_correction_flag==1` のとき、従来ホスト直列だった Lebedev 球面格子×粗動径メッシュの 補正計算を 2 パスの GPU バッチに置換(`TotalEnergy_Exc0_Batch_OpenACC`): ① 全ローカル原子×動径×Lebedev 点をフラット化してエネルギーを縮約しつつ点ごとの力プレファクタを デバイス常駐で保存、② (原子, 近接原子) ペアカーネルでプレファクタ×密度勾配を縮約して力 #8 相当を得る。XC 汎関数(Ceperley-Alder)とスプライン補間(KumoF/Dr_KumoF)はデバイス関数として再実装。

Total_Energy_GPU.c — 格子縮約の典型形

```
#pragma acc data copyin(Density_Grid_B0[0:grid_count], Density_Grid_B1[0:grid_count], \
                        ADensity_Grid_B[0:grid_count], PCCDensity_Grid_B0[0:grid_count], \
                        PCCDensity_Grid_B1[0:grid_count], dVHart_Grid_B[0:grid_count], \
                        Vxc_Grid_B0[0:grid_count], Vxc_Grid_B1[0:grid_count], \
                        RefVxc_Grid_B[0:grid_count], VNA_Grid_B[0:vna_grid_count], \
                        VEF_Grid_B[0:vef_grid_count])
  {
#pragma acc parallel loop reduction(+:local_Ena,local_Eef,local_EH1,local_EXC0,local_EXC1)
    for (BN=0; BN<grid_count; BN++){
```

7.11 力 — Force.c

力計算は第 2 版以降に最も大きく変わった領域である(対オリジナル +15,123/−11,241 行)。第 2 版時点は「#2/#4/#5 の OpenACC 縮約、#4B は VRAM 枯渇回避のため CPU 維持」だったが、現在は **#3・#4B・HNL を含む主要トレースがすべて GPU バッチ化**され、代わりに「共有デバイスで絶対に abort しない」ためのアリーナ+ピア待ち機構が整備された。親ゲートは `OPENMX_FORCE_GPU`(集団判定、既定 1)。

ステージ	実装	ゲート/フォールバック
#1(重なり補正)	ホストのみ	—
#2(運動エネルギー)	パック+GPU リダクション(`Force2_Kinetic_OpenACC`)	SO/LDA+U/拘束/Zeeman なしのとき。確保失敗時はその場でホスト計算
#3(dn/dx·Vpot 格子トレース)	チャンクバッチ `Force3_GpuTrace`: 1 ペア = 1 gang、重なり点で vector 縮約。**dOrbitals はデバイスで再計算**(実球面調和 L0≤3 の明示式+動径スプライン。約 4.5 GB/rank の PCIe 転送を排除)	予算判定(常駐分+192 MiB が(空き−予備)/ノード内ランク数に収まる。予備は 256 MiB+ノード内ランクあたり 16 MiB で、Force 段階の他の予算も同じ)。dOrbitals 再計算のみの不成立はホスト計算 dchi の転送に縮退(`OPENMX_FORCE3_GPU_DORB`)
#4(ProExpn_VNA==0)	dVNA 格子項の GPU リダクション(`Force4_OpenACC`)	確保失敗時はホスト計算
#4B(射影 VNA)	**GPU 内積バッチ**(コミット `7d38382d`): DS_VNA の 4 方向を float でデバイス常駐(direction-major)、MPI パイプラインで受けた halo/case-2 ブロックをアーカイブし、case-1/case-2 の融合トレースを各 1 カーネルで実行。float 積 → double 加算のキャスト位置まで CPU を再現。ダンピング尾部はホスト	`OPENMX_FORCE4B_GPU`(既定 1)+メモリ判定。不成立時は**融合 CPU トレース**(`OPENMX_FORCE4B_FUSED`、コミット `fe68cca6`)、それも無効なら従来 dHVNA 経路。特殊対(q_AN==0 等)は常にホスト

ステージ	実装	ゲート/フォールバック
#5(重なり×EDM)	GPU リダクション(`Force5_OpenACC`)	確保失敗時はホスト計算
HNL(非局所)	GPU 内積バッチ(case-1/case-2)。DS_NL は double のまま常駐。case-2 は全対を GPU が担当	`OPENMX_FORCEHNL_GPU`(既定 1)。スカラー相対論トレースのみ(SO/LDA+U/拘束/Zeeman 付き NC、j 依存 VPS は CPU)
#6/#7(EH0)、#8(Exc0)	Total_Energy の GPU バッチと同時計算(§7.10)	`OPENMX_TOTALENERGY_GPU`

表 7-1: 力計算ステージ別の GPU 化状況

アリーナ+ピア待ち機構(コミット `206b7e06`, `d0e1d17a`): 共有デバイスではランク間のステージ進行がずれるため、「バッチ開始時に見た空きメモリ」は当てにならず、遅れたランクの `acc data copyin` が満杯のデバイスに当たると OpenACC ランタイムが実行全体を abort させていた。現行実装はタイミング回避ではなく**致命窓そのものを除去**した:

- 全確保を 512 B 整列の**単一アリーナ**にまとめ(`Force_gpu_arena_off`)、カーネルは同名シャドウ ポインタ+`deviceptr` で読む(カーネル本体=数値は不変)。
- ホストフォールバックがある確保(リダクションバッファ)は `Force_gpu_arena_try` — 失敗したら NULL を返し、その場でホスト計算(通知はランクごと 1 回: `Force: a %.1f MiB GPU reduction buffer does not fit …; using the host for this reduction.`)。
- 必須確保(バッチ本体)は `Force_gpu_arena_wait` — 50 ms 刻みでピアランクの解放を待ち (2 秒で警告 1 回)、180 秒待っても不足なら所要量・空き量つきの診断で MPI_Abort。ピアの一時確保は必ず解放されるので前進保証があり、1 領域 = 1 アリーナなので待機中のランクは 何も保持せず相互デッドロックしない。
- **力ステージに「abort し得るデバイス確保」は存在しない**(コミット `d0e1d17a` で最後の 2 箇所の data 節確保もアリーナ化)。Force() 入口と大アリーナ解放後には OpenACC プールを CUDA に返却 (`Force_gpu_pool_flush`)。
- チャンクバッファは per-chunk 最大値で 1 回だけ確保して再利用(NVHPC OpenACC プールの「同一サイズのみ再利用」問題への対処。§6.3、コミット `28361f7e`)。

そのほか: Force4B/HNL/ProExpn の「メモリ不足で CPU フォールバック」通知は `level.of.stdout` 2 でノードごと 1 回 (コミット `b569f272`。level 3 では各ランクの数値)。`OPENMX_FORCE_PROFILE=1` でステージ別の `[Force profile]` 行(max/avg 秒)が出る。コミット `098adddc` は Force3/4B のステージングを「ホスト常駐ミラー方式」から「デバイス常駐+1 原子ステージ方式」に改め、ランク数倍で効いていたホストメモリ消費(数 GB/rank)と予算計上漏れを修正した。各ステージのバナーには GPU 実行時に `(GPU reduction)` / `(GPU-accelerated)` が付く(§10.3)。

7.12 電荷/DM 混合の高速化 — Mixing_H.c ほか 6 ファイル

SCF の混合(`scf.Mixing.Type`)は第 2 版まで完全に CPU のままだったが、コミット `94b6ca8c` / `6e94a9d3` で全スキームに高速パスが入った。有効化条件はすべて **GPUSOLVER のときのみ**(`OPENMX_MIX_FAST=0` で一括無効化)。レガシーコードは else 分岐に 無傷で残り、Simple mixing と NEGF 変種は対象外である。

RMM-DIISH(Pulay_Mixing_H)— 2 層構造(コミット `94b6ca8c`、Mixing_H.c +1,350 行)

動機: 残差 Gram 行列のためにスパース H 構造を $\dim^2$ 回掃引し、**行列要素ごとに 1 回の MPI_Allreduce** を 発行していた (History 50 の Fe fcc 5×5×5・np18 で DFT 時間 7,352 s 中 2,415 s)。`MixH_GpuPreflight` (集団判定、指紋が変わらない限り sticky)が 2 層のどちらかを選ぶ:

- **Tier A(GPU 常駐、`Pulay_Mixing_H_GPU`):** per-rank の単一 cudaMalloc アリーナに 残差スロットのリング・メトリック重み・最適残差・Gram ペアバッファを配置。履歴シフトは **論理→物理スロットマップの回転(コピー 0 回)**、ホスト側の残差履歴もポインタ回転+「新残差をスロット 0 とフラット像に同時計算」で常に整合。重み付き Gram 三角全体を 1 つの

OpenACC カーネルで計算し、**MPI_Allreduce は 1 回**。メトリック・α 階段・nan 検査は レガシーと同一。デバイス共有ランク群のアリーナ合計が空きに収まるときだけ発動。

- **Tier B(増分 CPU、`Pulay_Mixing_H_Inc`):** Gram 行列を(原子, 軌道行)ブロックの メトリック非依存部分和に厳密分解し、ポインタ回転で旧エントリがビット単位で有効なままスライドすることを利用して、継続反復では**新しい 1 行だけ**を O(dim·L) で計算(レガシーは O(dim²·L))。デバイスメモリ不要。
- 制御: `OPENMX_MIXH_GPU`(未設定=自動、0=レガシー、1=Tier A 強制、2=Tier B 強制)、`OPENMX_MIXH_GPU_RESERVE_MB`、`OPENMX_MIXH_GPU_VERBOSE`。不適合時は 1 回だけ「using the incremental CPU Pulay mixing instead」と通知。preflight は**意図的に `acc_clear_freelists()` を呼ばない**(§ 6.3)。
- 実測: Fe fcc 5×5×5・100 反復で Mixing_DM 504.6 s → 47.3 s(10.7 倍)、フル実行の総 wall −25%。収束 Utot は全表示桁で一致。

残りのスキーム(コミット `6e94a9d3`)— GPU は使わない CPU 高速パス

スキーム(ファイル)	高速化の内容
RMM-DIISK(`DIIS_Mixing_Rhok.c`)/ RMM-DIISV(`Mixing_V.c`)	**増分 Gram:** 格納済み残差列は事前重み付きでイテレーション間はスライドするだけなので、Gram 行列を対角方向に 1 つスライドし、新残差行だけを dgemv で計算して**短い Allreduce 1 回**。履歴シフトはポインタ回転+復元コピー 1 回(レガシーとビット一致)。継続が切れたら(SCF 巻き戻し等)レガシー計算で再シード
RMM-DIIS(`DIIS_Mixing_DM.c`)/ GR-Pulay(`GR_Pulay_DM.c`)	**バッチ Gram:** 全ペアの Gram を 1 回のスパース構造掃引(OpenMP)+ **1 回の Allreduce** で計算(レガシーはペアごとに 1 掃引+1 Allreduce、History 50 なら最大 1,275 回/反復)。最適残差と混合掃引も 1 パスに融合。残差履歴はポインタ回転、**DM/iDM の履歴シフトは意図的にレガシーの内容コピーのまま**(他の GPU モジュールが DM バッファに紐づくデバイス側状態を保持し得るため、DM のスロットポインタは回転してはならない — コード内コメントに明記)
Kerker(`Kerker_Mixing_Rhok.c` / `Mixing_V.c`)	履歴シフトのみ高速化(回転+memmove)。RMM-DIISK/V の StartPulay 前反復で毎回走るため無視できない

検証(216 原子 Si・np8): Kerker は全出力行ビット一致、RMM-DIISK は最終 Uele ビット一致、他スキームも 1e-10〜1e-12 で一致。Fe fcc 5×5×5(rmm-diisk・40 反復・np18)で Mixing_DM 33.5 s → 20.0 s。

7.13 派生ソルバ群 — 分極・DMmu・CWF・LNO(定型パターン 11 ファイル)

以下の 11 ファイルには同一の定型パターンが適用されている: `openmx_gpusolver_dense_utils.h` の include、GPUSOLVER 要求時のデバイス割当、static コンテキスト、そして **ELPA2 分岐の直前**への `scf_eigen_lib_flag==GPUSOLVER && GPU_CPU_SWITCH_NUM<=n && na_rows==n && na_cols==n` 分岐の挿入(`OpenMX_GpuSolver_DenseDsyevx/DenseZheevx` による対角化)。条件を満たさなければ従来の ELPA/ScaLAPACK に流れる。

ファイル	GPU 対角化の挿入箇所
`Band_DFT_Col_CWF.c`	Zheevx × 3(S・H・CWF バンド)
`Band_DFT_Col_DMmu.c`	Zheevx × 4
`Band_DFT_NonCol_CWF.c`	Zheevx × 3
`Band_DFT_NonCol_DMmu.c`	Zheevx × 4
`Cluster_DFT_Col_CWF.c` / `Cluster_DFT_Col_DMmu.c` / `Cluster_DFT_Col_LNO.c`	Dsyevx × 2 ずつ
`Cluster_DFT_NonCol_CWF.c` / `Cluster_DFT_NonCol_DMmu.c`	Dsyevx × 1 + Zheevx × 1 ずつ
`Electric_Polarization_BandCol.c`	Dsyevx × 1 + Zheevx × 4(DM・重なり・H・ABC)
`Electric_Polarization_BandNonCol.c`	Zheevx × 5(ELPA1 側フォールバック条件の調整も含む)

表 7-2: 定型 GPUSOLVER 分岐が挿入されたファイル群

7.14 汎用固有値ルーチン — Eigen_lapack.c / EigenBand_lapack.c

- `Eigen_lapack()`(実対称)は `flag != ELPA1/2 && n ≥ 1000` で `Eigen_gpusolver_x`(→ `gpusolver_Syevdx`)へ分岐。**Eigen_lapack を呼ぶ 20 以上のファイル(軌道最適化・DC・LNO など)が自動的に GPU 化の恩恵を受ける**。上位に固有ベクトル直交性検査つきのソルバ巡回リトライがある。
- `EigenBand_lapack()`(エルミート)は N ≥ 1000 で `gpusolver_zheevx` へ。失敗時は CPU の Householder 法 `Eigen_HH` にフォールバック。
- 両者とも OpenACC デバイス常駐配列を直接処理する `_openacc` エントリを持ち、非共線 DC の OpenACC 経路などから使われる。2 次元配列⇔1 次元デバイス配列の詰め替えも device 上で行う。

8. GPU 化以外の変更点

8.1 OpenMP バグ修正 — private 指定漏れによるヒープ破壊の解消

オリジナル OpenMX 4.0 の `Stress.c` と `Total_Energy.c` には、`#pragma omp parallel` の `private()` 節にループ変数 `i` が入っておらず、**暗黙 shared の `i` を複数スレッドが同時に読み書きする競合**があった。あるスレッドがループ境界を判定した直後に別スレッドが `i` を進めると配列の範囲外アクセスが起こり、ヒープ破壊(malloc メタデータの破損)として顕在化する。本 GPU 版の開発中に runtest のクラッシュ原因として特定され、修正された(コミット `d451eeb8`)。

Stress.c — 修正内容(private リスト末尾への `,i` 追加。Total_Energy.c も同形)

```
/* 修正前(オリジナル)*/
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(中略),sumt1,sumt2,sumt3,sumt4,sumt5)

/* 修正後(GPU 版)*/
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(中略),sumt1,sumt2,sumt3,sumt4,sumt5,i)
```

- `Stress.c`: 競合箇所は並列リージョン内の `for (i=0; i<9; i++){ sumt1[i]=0.0; … }` (原子ループ内で毎回実行)。このファイルの差分はこの 1 行のみ。
- `Total_Energy.c`: `Calc_EXC_EH1()` の Lebedev 球面グリッド積分並列区間。競合ループは `for (i=0; i<(FNAN[Gc_AN]+1); i++)` で `Dr[i][ir][ia]` 等を添字アクセスするため、競合時に確保域外を指し得る(ヒープ破壊経路)。

> **意義:** この 2 件はオリジナル OpenMX 4.0 にも存在するバグであり、GPU と無関係に OpenMP スレッド並列(スレッド数 ≥ 2)で発症し得る。CPU 版 OpenMX を使う場合にも適用価値がある修正である。

8.2 メモリ管理・堅牢化

ファイル	修正内容
`openmx.c`	起動直後に `Raise_Stack_Limit()` を呼び、スタックのソフトリミットをハードリミットまで自動で引き上げる(`getrlimit/setrlimit(RLIMIT_STACK)`)。`ulimit -s` 未設定環境での大きなスタック配列によるクラッシュ対策
`truncation.c` / `Free_Arrays.c`	`VNA_Grid` / `VNA_Grid_B` / `VEF_Grid` / `VEF_Grid_B` の free 後に NULL 代入を追加(二重 free 防止)。truncation は Set_Hamiltonian / Set_Density_Grid の GPU キャッシュ無効化フックも呼ぶ(幾何変更で常駐テーブルが陳腐化するため)
`Set_Nonlocal.c`	固定サイズ配列をオーバーフロー検査付きヒープ確保に置換(§ 7.7)
`Free_Arrays.c`	終了時(wherefrom==0)に GPU キャッシュの無効化(`Set_Density_Grid_GPU_Invalidate` / `Set_Hamiltonian_Invalidate_OpenACC_MatrixElements_Cache`)と `openmx_gemmul8ReleaseWorkspaces()` を呼ぶ

openmx.c — Raise_Stack_Limit(全文)

```
static void Raise_Stack_Limit(void)
{
#ifdef RLIMIT_STACK
  struct rlimit limit;

  if (getrlimit(RLIMIT_STACK,&limit)!=0) return;
  if (limit.rlim_cur==RLIM_INFINITY) return;
  if (limit.rlim_max==0 || limit.rlim_cur==limit.rlim_max) return;

  limit.rlim_cur = limit.rlim_max;
  setrlimit(RLIMIT_STACK,&limit);
#endif
}
```

8.3 その他の修正

ファイル	修正内容
`Runtest.c`	(1) 入力ファイル判定を `strstr` による部分一致から末尾一致 `has_dat_suffix()` に厳密化(`foo.dat~` 等の誤検出防止)。(2) 各テスト実行前に旧 `<System.Name>.out` を削除し、前回結果の混入を防止(MPI_Barrier 付き)
`Spherical_Bessel.c`	ホットパスの呼び出し毎 malloc/free(漸化式係数配列)を除去し、係数をインライン計算化(数式は同一)
`openmx_common.h`	インクルードガードを追加(オリジナルには無かった)。バージョン文字列を "4.0" → "4.0 GPU" に変更
`Input_std.c`	GPU 関連キーワードの追加(第 10 章)
`mpao.c`	`#include <ctype.h>` の 1 行差のみ(ビルド対象外のユーティリティで影響なし)

9. ビルドシステム

§9.1〜9.3・§9.5〜9.6 はネイティブ CUDA ビルドの説明と過去の例を残す。現行の CUDA／HIP ビルドは §9.7 を起点とし、どちらも §9.4 の共通 ELPA を使用する。

9.1 ツールチェーンの全面刷新

オリジナルの `makefile`(Intel oneAPI + MKL)は `Makefile`(NVHPC + CUDA + 自前 FFTW + GEMMul8)に 全面書き換えされた。`makefile.cpu_intel.bak` は当初は原本の同一コピーだったが、現在は ELPA 部分を更新している(§9.4)。

項目	オリジナル(makefile)	GPU 版(Makefile)
C コンパイラ	`mpiicx -O3 -xHOST -qopenmp`	NVHPC 同梱 OpenMPI の `mpicc`(nvc)+ `-acc -Minfo=accel -std=c11 -mtune=native -march=native -fopenmp`
Fortran	`mpiifx`	`mpif90`(nvfortran) `-O2 -fopenmp`
BLAS/LAPACK	MKL(静的)	NVHPC 同梱の静的 `liblapack_lp64.a` / `libblas_lp64.a`
ScaLAPACK	MKL ScaLAPACK	HPC-X OpenMPI 側の `libscalapack_lp64.a`
FFT	MKL FFTW ラッパ	自前ビルドの静的 FFTW 3.3.11(third_party)
GPU ライブラリ	—	`-lcudart -lcusolver -lcublas -lcublasLt -lcuda -lnvidia-ml -lnvJitLink -cuda` + `libgemmul8.a`
CUDA C++	—	nvcc で `gemmul8_openmx.cu` / `set_hamiltonian_gpu.cu` をコンパイル(ホストコンパイラ nvc++)
現在の SDK 既定値	Intel oneAPI	`NVHPC_ROOT ?= /opt/nvidia/hpc_sdk/Linux_x86_64/26.9`、`NVHPC_CUDA_VERSION ?= 13.4`(変数で上書き可)

ビルド環境の隔離: `Makefile` 冒頭で `PATH` を NVHPC のみに固定し、`CPATH` / `LD_LIBRARY_PATH` / `CC` など 30 個超の環境変数を `unexport` する。ユーザー環境の CFLAGS 等は効かない(意図的)。

Makefile — コンパイラ定義(抜粋)

```
COMMON_INCLUDES = $(FFTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include -I$(NVHPC_CUDA_TARGET)/include -I$(NVHPC_CUDA_HOME)/include
COMMON_CFLAGS   = -w -Minfo=accel -acc -std=c11 -mtune=native -march=native -fopenmp $(COMMON_INCLUDES)

CC  = $(MPICC) $(COMMON_CFLAGS) -O3
CC2 = $(MPICC) $(COMMON_CFLAGS) -O1
CC3 = $(MPICC) $(COMMON_CFLAGS) -O2
CC_NOACC = $(filter-out -Minfo=accel -acc,$(CC2))
CC_NOACC_O3 = $(filter-out -Minfo=accel -acc,$(CC))
CC_DFTD3 = $(MPICC) -w -O0 -std=c11 -fopenmp $(COMMON_INCLUDES)
FC = $(MPIF90) -O2 -mtune=native -march=native -fopenmp $(FFTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include
```

オリジナルは全ファイル `$(CC)` 一本だったのに対し、GPU 版は**ファイル別の最適化・OpenACC 調整**を行う (表 6-1)。OpenACC のターゲットアーキ指定(`-gpu=ccXX`)はなく、ビルドマシンの GPU を自動検出する。クロスターゲットしたい場合は `COMMON_CFLAGS` に追加する。

9.2 リンク構成

Makefile(抜粋)

```
NVPL_SCALAPACK_LIB ?= $(NVHPC_MPI_LIBDIR)/libscalapack_lp64.a
NVPL_LAPACK_LIB    ?= $(NVHPC_COMPILER_LIB)/liblapack_lp64.a
NVPL_BLAS_LIB      ?= $(NVHPC_COMPILER_LIB)/libblas_lp64.a
NVPL_LIBS          ?= -Wl,--start-group $(NVPL_SCALAPACK_LIB) $(NVPL_LAPACK_LIB) $(NVPL_BLAS_LIB) -Wl,--end-group
CUDA_LIBS          ?= ... -Wl,--no-as-needed -lnvJitLink -Wl,--as-needed -lcudart -lcusolver -lcublas -cuda
LIB = $(FFTW3_LIBS) $(NVPL_LIBS) $(MPI_FORTRAN_LIBS) -pgf90libs -mp -lpthread -lm -ldl $(CUDA_LIBS)
...
LIB += -lcublasLt -lcuda -lnvidia-ml -lstdc++
```

リンク行は `$(CC) $(OBJS) $(GEMMUL8_LIB) $(LIB) -lm -o openmx` 。OBJS への追加分：`Total_Energy_GPU.o` 、`Set_Density_Grid_GPU.o` 、`set_hamiltonian_gpu.o` 、`gemmul8_bridge.o` 、`gpusolver_Syevd.o gpusolver_Syevdx.o set_cuda_default_device_from_local_rank.o set_openacc_device_from_local_rank.o` 、`OutData_Binary.o` (呼び出し元のないデッドリンク)。かつて含まれていた `my_cublasDgemm.o my_cublasZgemm.o` は `a7f8cba6` で削除された。

9.3 third_party の自動ビルドと「make 一発」保証

既定ターゲットが `openmx: $(FFTW3_LIB) $(GEMMUL8_LIB) $(OBJS)` のため、素の `make` で FFTW3 → GEMMul8 → 本体の順に全自動でビルドされる。コミット `9fe8ce13` で **クリーンツリーからの単一 `make` (-j 並列可)で完走する**ことが保証された。修正は 4 点:

1. **FFTW3 ソースの自動クローン:** `third_party/fftw3` が無い場合、従来はエラーで停止していたが、現在は `git clone --branch fftw-3.3.11 --depth 1` を自動実行する(GEMMul8 と同じ扱い)。git ソースには生成済み codelets が無いため、番兵ファイルを確認してリリース tarball (`fftw-3.3.11.tar.gz` 、リポジトリにもコミット済み)から `rsync --ignore-existing` で補完 → autoreconf → out-of-tree configure(`--enable-static --disable-shared` 、コンパイラは nvc/nvc++/nvfortran)→ `make install` の既存チェーンに接続する。
2. **FFTW3_USERS の補完:** `fftw3.h` を include する `Cluster_DFT_Col.o` が order-only 依存(`| $(FFTW3_LIB)`)のリストに入っておらず、-j ビルドで FFTW の install 前に コンパイルが走って最初の 1〜2 回の make が失敗していた。現在は `FFTW3_USERS = Poisson.o Stress.o Poisson_ESM.o TRAN_Poisson.o TRAN_CDen_Main.o TRAN_Set_Electrode_Grid.o Cluster_DFT_Col.o` 。
3. **当時の Fortran モジュール依存の修正:** コミット `9fe8ce13` で、当時の埋め込み ELPA に不足していた `use` 依存を追加し、`.mod` ファイル生成前にコンパイルが始まるレースを解消した。この埋め込み版は現在は削除済み(§9.4)。現在の `openmx_elpa_cpu.o` と GPU ブリッジはともに 新 ELPA のインストール完了スタンプに依存し、ELPA 内部の Fortran モジュール依存はそのビルドシステムが管理する。
4. **gemmul8.hpp の order-only ルール:** GEMMul8 チェックアウトが無い新規クローンで「No rule to make target」で死ぬのを防ぐ空レシピルールを追加。

Makefile — GEMMul8 の取得と単一 TU ビルド

```
$(GEMMUL8_DIR)/Makefile:
        mkdir -p $(GEMMUL8_REPO_DIR)
        cd $(GEMMUL8_REPO_DIR) && \
          { [ -d .git ] || git init -q .; } && \
          { git remote get-url origin >/dev/null 2>&1 || git remote add origin $(GEMMUL8_REPO_URL); } && \
          git fetch --depth 1 origin $(GEMMUL8_COMMIT) && \
          git checkout -q $(GEMMUL8_COMMIT)
$(GEMMUL8_OPENMX_OBJ): gemmul8_openmx.cu $(GEMMUL8_DIR)/Makefile
        $(NVCC_GEMMUL8_ENV) $(NVCC) $(NVCC_GEMMUL8_FLAGS) -c gemmul8_openmx.cu -o $(GEMMUL8_OPENMX_OBJ)
$(GEMMUL8_LIB): $(GEMMUL8_OPENMX_OBJ)
        mkdir -p $(GEMMUL8_DIR)/lib
        ar rcs $(GEMMUL8_LIB) $(GEMMUL8_OPENMX_OBJ)
```

検証(コミットメッセージ): `make clean` 後、単一の `make -j16 openmx` が RTX 5080 機で 177 秒で完走。2 回目の make は no-op。コンパイルフラグは不変のため生成バイナリも不変。`clean` は `fftw3-clean` と `gemmul8-clean` を含むため、`make clean` 後は third_party の再ビルドが必要になる。

9.4 ELPA 2026.02.002 の共通利用

旧 `elpa-2018.05.001/` ディレクトリと、旧実装 `elpa1.f90` / `solve_evp_real.f90` / `solve_evp_complex.f90` は削除した。CPU 経路と GPU 経路は、`third_party/dist/` のリリースアーカイブからビルドした **同一の ELPA 2026.02.002 ライブラリ**、`$(GPUSOLVER2_PREFIX)/lib/libelpa.a` をリンクする。従来のシンボル隔離用オブジェクトの生成・リンクも廃止した。

`openmx_elpa_cpu.c` は既存の Fortran 呼び出しインターフェースを維持し、新しい C API を使う。`scf.eigen.lib elpa1` は 1stage CPU ソルバ、`elpa2` は generic の実・複素カーネルを 使う 2stage CPU ソルバを選び、ELPA の GPU オプションは明示的に無効化する。`gpusolver2` は `elpa_cosma_bridge.c` から同じライブラリを使い、ビルド時に選んだ CUDA または HIP の GPU バックエンドを有効にする。CPU ELPA は引き続き **GPU メモリ不足時の実行時フォールバック先(§5.6)**になる。

9.5 派生 Makefile

- **`makefile.cpu_intel.bak`** — 元はオリジナルの CPU/Intel makefile のバイト同一コピー。現在の ELPA 部分は共通の新ライブラリと CPU アダプターを使う構成に更新済み(§9.4)。
- **`Makefile.bunshiken`** — 共用計算機(モジュール環境: `nvhpc/25.9-nompi` + `openmpi/5.0.8` + `gcc-toolset/15`)向けの変種。NVHPC 25.9 + 外部 OpenMPI、CUDA 13.0、nvcc ホストコンパイラが g++、GEMMul8 が旧レイアウト(コミット固定なし)、オブジェクト名が gpusolver リネーム前の `cusolver_*.o` のまま、という点で**主 Makefile より一世代以上古い**。9.3 の「make 一発」修正も未反映。他環境へ移植する際の参考例として扱うこと。

9.6 ビルド手順と注意

1. `git clone --recursive` (submodule: fftw3, GEMMul8)。submodule を忘れても `make` が両方とも自動 clone/fetch する(9.3)。
2. `source/` で `make` (既定ゴール openmx、`-j` 可)。FFTW3 の取得・補完→configure→install、GEMMul8 の取得→コンパイル→ar、CUDA カーネルの nvcc コンパイル、本体のコンパイル→リンクまで一括実行される。
3. `make install` で `../work/openmx` にコピー。ユーティリティ群は `make all`。

ビルド時の注意: (1) NVHPC のバージョンを変えると §6.5 のミスコンパイル回避策の再検証が必要。(2) SDK 更新時は MPI の選択を確認し、必要に応じて `NVHPC_MPI_BIN` と `NVHPC_MPI_LIBDIR` を上書きする。(3) GEMMul8 のアーキは `make GEMMUL8_GPU_ARCH=90` のように上書き可能(既定は nvidia-smi 自動検出)。(4) オリジナルにあった API ライブラリ(libopenmx_api)・elpa_col などの補助ターゲットは削除されている。

9.7 CMake による CUDA / HIP 統合ビルド

統合リポジトリは両バックエンドを含む。CMake 3.20 以上・Python 3.8 以上・GNU make が分離した ビルドを準備し、既存 Makefile のファイル別コンパイラ・オフロード設定を利用する。`tools/prepare_build.py` が選択したソースを `build/<backend>/native/source/` にリンクする。共通ソースは `source/`、HIP の置換ソースと互換ヘッダは `source/backends/hip/` にあり、`sources.json` が選択を記録する。変更される依存ソース・オブジェクト・インストール先はビルドごとに 分離される。元の `openmx4.0_amdgpu` チェックアウトは不要。

設定	内容
`OPENMX_GPU_BACKEND`	`CUDA` (既定)または `HIP`。別々のビルドディレクトリを使う。

設定	内容
`OPENMX_GPU_ARCH`	CUDA: `120`(RTX 5080)、`80`(A100)、`90`(H100)など。HIP: `gfx942`(MI300A)、`gfx1200`(RX 9060 XT)など。
`OPENMX_BUILD_JOBS`	内部 make の並列数。既定は `4`。外側の `cmake --build -j` では変わらない。
`OPENMX_MAKE_OPTIONS`	Makefile へ渡す `NAME=value` をセミコロンで区切り、シェルで引用符を付ける。

CUDA の例 — リポジトリのルートで実行

```
git submodule update --init --recursive
cmake -S . -B build/cuda \
  -DOPENMX_GPU_BACKEND=CUDA -DOPENMX_GPU_ARCH=120 \
  -DOPENMX_BUILD_JOBS=4
cmake --build build/cuda
```

環境に合わせて `NVHPC_ROOT`、`NVHPC_CUDA_VERSION`、`NVHPC_CUDA_HOME`、`NVHPC_MATH_DIR`、MPI のパスを `OPENMX_MAKE_OPTIONS` で上書きする。CUDA の MAGMA を有効にする場合は `WITH_MAGMA=1` を追加する。従来の `make -C source -j4 openmx` も利用できる。

HIP の例 — MPI とライブラリのパスは実際の環境に置き換える

```
git submodule update --init --recursive source/third_party/GEMMul8
cmake -S . -B build/hip \
  -DOPENMX_GPU_BACKEND=HIP -DOPENMX_GPU_ARCH=gfx942 \
  -DOPENMX_BUILD_JOBS=4 \
  '-DOPENMX_MAKE_OPTIONS=ROCM=/opt/rocm-7.2.0;MPI_HOME=/path/to/openmpi-rocmflang;AOCL_HOME=/path/to/aocl/aocc;AOCC_LIB=/path/
to/aocc/lib'
cmake --build build/hip
```

HIP は ROCm clang/clang++、hipcc、flang、OpenMP オフロードと、AOCL の BLIS/libFLAME、ScaLAPACK、FFTW3、Utilities を使う。MPI の Fortran モジュールは ROCm flang と互換である必要があり、gfortran 用の `mpi.mod` は使用できない。`AOCC_LIB` は AOCL が依存する Fortran ランタイムを指す。Makefile は `LIBRARY_PATH` の先頭に `ROCM_LLVM_LIB` を置き、Intel/AOCC などの 別の OpenMP ランタイムより、選択した ROCm のランタイムを優先する。

初回は GEMMul8、パッチ適用済み MAGMA(HIP)、ELPA/COSMA も構築する。ELPA スタックには autoconf、automake、libtoolize、m4、unzip も必要。HIP の構築スクリプトは ROCm flang の `type(c_ptr)` 相互運用問題を避けるため、ELPA のホスト／デバイス割当ラッパの出力先を `integer(c_intptr_t)` 経由に修正する。元の同梱アーカイブは変えず、ビルド内のソースに適用する。依存物の構築に既存の CMake 3.24 以上を使う場合は `GPUSOLVER2_CMAKE=/absolute/path/to/cmake` を指定できる。

実行ファイルは `build/cuda/openmx` と `build/hip/openmx`。各ビルドの `selected-sources.json` でソース選択を確認できる。`cmake --install build/hip --prefix /path/to/install` は `bin/openmx` のみをインストールする。実行時ライブラリの配置を維持し、DFT データを別途用意すること。入力の `DATA.PATH` は作業ディレクトリ 基準なので、既存入力は `work/` から実行するか `DFT_DATA19` の絶対パスを指定する。設定の詳細は `docs/GPU_BACKENDS.md` を参照。

9.8 回帰試験と記録済みの検証結果

`scf.eigen.lib elpa1` / `elpa2` は両ビルドで CPU 実行を選ぶ。`OPENMX_GPU=0` は既定の GPU 選択を ELPA2 に切り替える。CUDA または HIP 実行ファイルの中で CPU 経路を使うため、そのビルドの実行時ライブラリは引き続き必要。

```
ctest --test-dir build/cuda --output-on-failure
ctest --test-dir build/hip --output-on-failure
python3 tests/run_elpa_cpu_mpi.py \
  --native-source build/cuda/native/source
```

アダプター試験は LAPACK を参照し、実数／複素数、一段階／二段階法、部分固有ベクトル、長方形のプロセス グリッド、MPI 部分コミュニケータ、空のローカル行・列を確認する。HIP では `build/hip/native/source` を指定する。標準 CPU 試験では毎回未使用の出力ディレクトリを使う。

```
python3 tools/run_gpu_suites.py --binary build/cuda/openmx \
  --output build/cpu-elpa-S --suite S --ranks 18 --threads 1 \
  --env OPENMX_GPU=0 --together
python3 tools/compare_gpu_suites.py --reference build/cpu-elpa-S

python3 tools/run_gpu_suites.py --binary build/cuda/openmx \
  --output build/cpu-elpa-L --suite L --ranks 18 --threads 1 \
  --env OPENMX_GPU=0 --together
python3 tools/compare_gpu_suites.py --reference build/cpu-elpa-L
```

S は `-runtest`、L は `-runtestL` に対応する。参照比較は全エネルギーと記録された全原子の 力の全成分を確認する。HIP を調べる場合は実行ファイルを置き換える。上記は再現手順であり、新しいベンチマーク結果ではない。

記録された変更	正当性の確認
`bdcba87d`: ELPA 共通化	CPU S: 14/14、L: 16/16、ELPA1 明示: 4/4。MPI アダプター: 6 ランクで各ランク 20 ケース。
`57a3a3fc`: GPU 非表示時の HIP 後処理	16 MPI × 1 スレッドで S 14/14、L 16/16。エネルギー・力は参照値と 1e-7 以内で一致。HIP CTest: 9/9。

これらは記載したコミットの検証記録である。`docs/GPU_BACKENDS.md` の AMD GPU 検証は Radeon RX 9060 XT 1 台、ROCm 7.2.0、OpenMPI 5.0.9、AOCL 5.2.0 で実施した。CPU/GPU 比較、S 全例、L3 代表例、強制 GPU 対角化、ELPA/COSMA、一部の容量復旧経路を含む。MI300A 実機、物理マルチ GPU、複数ノード、GPU での L/L2/L3 全例の実行はこの検証に含まれない。

10. 実行時制御リファレンス

10.1 入力ファイルキーワード

GPU 機能に関して追加・拡張された入力キーワードは、GPU 診断行をどこまで出力するかも決めるようになった OpenMX 本来の `level.of.stdout`(§10.3)を除けば、次の 2 つだけである(それ以外の調整はすべて環境変数)。

Input_std.c — 追加部分(全文)

```
/* default=gpusolver; input_string2int() returns i_vec[0] when the keyword is absent ...
   "cusolver" is a deprecated alias of "gpusolver"; ... */
s_vec[0]="gpusolver"; s_vec[1]="lapack"; s_vec[2]="elpa1"; s_vec[3]="elpa2"; s_vec[4]="cusolver";
i_vec[0]=GPUSOLVER;   i_vec[1]=0;        i_vec[2]=ELPA1;   i_vec[3]=ELPA2;   i_vec[4]=-GPUSOLVER;
input_string2int("scf.eigen.lib", &scf_eigen_lib_flag, 5, s_vec,i_vec);
if (scf_eigen_lib_flag==-GPUSOLVER){
  if (myid==Host_ID){
    printf("Warning: scf.eigen.lib=cusolver is deprecated; use scf.eigen.lib=gpusolver instead.\n");
  }
  scf_eigen_lib_flag = GPUSOLVER;
}
scf_eigen_lib_flag_input = scf_eigen_lib_flag;

/* use GPU? (added by H.Kawai, February 2024) */
input_int("scf.Gpu.Num", &SCF_Gpu_Num, 30);
```

キーワード	型	既定値	意味
`scf.eigen.lib`	文字列選択	`gpusolver`	従来の `elpa2` / `lapack` / `elpa1` に **`gpusolver`**(GPU 経路一式の有効化)を追加し、これが既定値。したがって CPU 側の固有値ソルバを明示指定しない限り GPU 経路が走る。`cusolver` は警告付きの非推奨エイリアス。GPU 不在時は自動的に ELPA2 に降格。**`gpusolver2`** はさらに、コリニア/ノンコリニアの**クラスター計算本流の対角化**を ELPA 2026.02.002(CUDA または HIP の GPU カーネル)+ COSMA(pdgemm)で全ランク・全 GPU に分散する(Pegasus 級マルチノード向け)。クラスター以外の計算では入力エラーで停止し、クラスター対角化以外の動作は `gpusolver` と完全に同一
`scf.fp32.enable`	論理値	`on`	`off` で FP64 演算を強制する。FP32 固有ベクトル精緻化、FP32・二重単精度のグリッド積分、二重単精度の軌道評価を無効化し、VNA 行列・力・応力の縮約にある従来の float 積も FP64 に昇格する。FP32/DF を要求する環境変数より優先。GEMMul8 は `scf.gemmul8.enable` と専用環境変数で独立に制御し、cuSOLVER の FP64 固定小数点エミュレーションも独立。既存の float テーブル保存形式は変更しない。既定の `on` は従来の精度選択を許可する。
`scf.Gpu.Num`	int	30	ノード内で使用する GPU 台数の上限。すべての rank→GPU 割り当て(GPUSOLVER 中核初期化、Band/Cluster ソルバ経路の local-rank デバイスヘルパー、DC-LNO プロキシ)に適用。`scf.Gpu.Num 1` でノードの GPU 搭載数によらず 1 台のみ使用(デバッグ用途。既定 30 は実質「検出された全 GPU を使う」)。0 以下や検出数以上の値は無視
`level.of.stdout`	int	1	OpenMX 本来の標準出力量(−1〜3)を GPU 診断行にも適用する(§10.3)。1 はエラー、GPU が使えないこと、GPU 固有値ソルバを断念したことだけを出力し、2 でデバイス計画とバナー、最終ステップの FP64 再求解、個々のステージ・求解のフォールバックと再試行を加え、3 でさらに SCF ステップごと・k 点ごと・ランクごとの詳細を加える

10.2 環境変数一覧

GPU 実行のチューニングは環境変数で行う設計である(入力ファイルを変えずにジョブスクリプト側で調整できる)。`OPENMX_` 接頭辞版と `GEMMUL8_` 接頭辞版がある変数は OPENMX_ 版が優先される。**ほぼすべての GPU 経路が既定で有効**であり、環境変数は主に「無効化」「メモリ予約の調整」「強制」に使う。

プログラムが読む名前はすべて `source/openmx_env_registry.h` に登録されており、以下の表はこのレジストリと `doc/environment_variables.json` から `tools/env_registry_doc.py` が生成する(両者がずれるとテストが失敗する)。レジストリモジュール `source/openmx_env.c` は `MPI_Init` の直後に環境を一度だけ 読み、rank 0 の値を全ランクへ配布する。そのため、ランチャーが一部のランクにしか渡さなかった変数でもランク間で 食い違わない(最後のグループのランチャー由来の名前だけは例外)。ビルドのどの設定にも使われない `OPENMX_` で始まる名前と、種類に合わない値は、黙って無視されるのではなく rank 0 が起動時に 1 度警告し、`level.of.stdout 2` では入力読み込み後に有効な設定を一覧する。解釈はすべての読み手で共通である: フラグは数値(非零で有効)、`on` / `yes` / `true` 、`off` / `no` / `false` を受け付け、`auto`・空の値・未設定は既定値を選ぶ。整数と MiB 単位の大きさは十進の整数全体でなければならない。

環境変数	既定値	意味
デバイス割り当てとデバイスメモリの会計(set_cuda_default_device_from_local_rank.c)		
`OPENMX_GPU_STACK_BYTES`	OpenMX 自身のカーネルの最大スタックフレーム(リンク時にリンク済みバイナリを cuobjdump で測る。現在のビルドでは 1000 バイト)	デバイス初期化時に設定するスレッドあたりのカーネルスタック上限(`cudaLimitStackSize` 、`OpenMX_GpuPresizeKernelStack`)。ドライバ既定の 1 KiB を超えるときだけ設定し、ドライバはその時点、つまりデバイスが空のうちにコンテキストのローカルメモリプールを確保するので、フレームの大きいカーネルの初回起動時に確保が走らなくなる。その起動時の増分(RTX 5080 で 1224 バイトのフレームにつきコンテキストあたり約 26 MiB。(フレーム − 1024)×デバイスの全常駐スレッド数)は、デバイスを予備領域まで埋める Force 段階のアリーナと競合し、16 GB で 18 ランクのとき負けていた(`Force3_GpuTrace` の `cuLaunchKernel` が out of memory、C1000。この dOrbitals カーネルがそのフレームを持っていた。球面調和関数の表を平坦化して 936 バイトにしたので、現在この処理は何もしない)。Makefile はリンク後のバイナリからこの値を読み(オブジェクトのカーネルのみ。ELPA の 1328 バイトのものなど、リンクしたライブラリのカーネルは数えない)、一度だけ再リンクする。0 で無効。上限を上げた場合のコストはランクあたり(上限 − 1024)×常駐スレッド数のデバイスメモリ。
`OPENMX_GPU_MEMINFO_NVML`	1	GPU 経路のデバイスメモリ予算はすべて `OpenMX_GpuMemGetInfo()` から始まる。これは `cudaMemGetInfo()` の空き容量を、NVML が報告する物理的な空き容量で頭打ちにしたもの。WDDM 系プラットフォーム(WSL2)では CUDA ランタイムがプロセスごとの見え方を返し、他ランクのコンテキストや配列が空き容量に反映されず、物理メモリを超える確保も失敗せずにシステムメモリへ退避して数十分の一の速度になる: 12 GB の RTX 4070 Ti で 12 ランクが各自「空き 10.8 GiB」(実際はコンテキストの後に 0.6 GiB)と見て DS_VNA バッチに 836 MiB ずつ確保し、Set_ProExpn_VNA が数分間止まった。Linux では両者は一致する。両者が 256 MiB 以上違うとき標準出力に 1 回だけ通知する(level.of.stdout 2 以上)。0 で素の `cudaMemGetInfo()`
`OPENMX_GPU`	1	GPU 経路の親スイッチ。**0** にすると `scf.eigen.lib=gpusolver` でも全ランクがホストコードで動く(デバイス探索、マルチ GPU プール、容量ワーカーがすべて参照する)。
`OPENMX_GPU_PROBE_RESERVE_MB`	256	各ランクが自分のコンテキストを作った直後、GPU を使うと判定するために残っていなければならない空きデバイスメモリ。デバイスモジュールのロード(確保できないと失敗ではなく異常終了する)とランタイムの一時領域の余裕。

環境変数	既定値	意味
`OPENMX_GPU_RANK_MAP`	block	ノード内のランクとデバイスの対応。既定ではノード内の連続したランクがブロック単位で 1 台を共有する（1 台あたり ceil(ノード内ランク数 / デバイス数)）。`m` で始まる値（modulo）にすると `local_rank % デバイス数` で割り当てる。
GEMMul8(gemmul8_bridge.cu)		
`(OPENMX_)GEMMUL8_DISABLE` / `_D` / `_Z`	off	GEMMul8 を無効化し cuBLAS FP64 に固定(全体 / 実数のみ / 複素のみ)
`(OPENMX_)GEMMUL8_NUM_MOD_D` / `_Z`	15	法の個数(精度)。範囲 2〜20、範囲外は 15 に戻る
`(OPENMX_)GEMMUL8_FASTMODE_D` / `_Z`	0	高速スケーリングモード(精度低下)
`(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT`	30	ワークスペースの総 VRAM 比上限(Band_DFT_NonCol は未設定時 50 を setenv)
`(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB`	1536	ワークスペース確保後に残すべき最低空き VRAM
`(OPENMX_)GEMMUL8_MAX_WORKSPACE_MB`	256	GEMMul8 作業領域のランクあたり上限（MiB）。これより大きな積は cuBLAS FP64 へ退避せずにブロック分割して GEMMul8 で計算する（AMD 版から移植）。0 で上限なし、小さすぎる値は実行可能なブロックサイズまで引き上げる。
`OPENMX_GEMMUL8_LOCAL_RANKS`	ランチャーの値	作業領域の合計予算を見積もるときの、1 台の GPU を共有するランク数。既定ではランチャーの値（`OMPI_COMM_WORLD_LOCAL_SIZE`、`SLURM_NTASKS_PER_NODE`）を使い、ランク配置が不均一なら切り上げる。
`OPENMX_GEMMUL8_A100_FORWARD_MIN_N`	6144	フル A100（sm_80）では、この次元以上の実数正方積はネイティブ FP64 デバイスでも全作業領域の Ozaki 演算を使い、それ未満はネイティブ FP64 を使う（実測の交点。0 で規則を無効化。MIG・A30・GeForce・複素積には外挿しない）。
`OPENMX_GEMMUL8_FORWARD` / `_NUM_MOD` / `_FASTMODE` / `_REUSE` / `_UNBLOCKED`	実数積と同じ / 15 / 0 / 0 / 1	前向き積（ブリッジの forward 段を通る cuBLAS 形式の呼び出し）の演算。未設定 = 実数積と同じ設定、`fp64` = cuBLAS FP64、`gemmul8` = 専用の法の数（`_NUM_MOD`）、高速スケーリング（`_FASTMODE`）、準備済みオペランドの呼び出し間再利用（`_REUSE`、unblocked を含意）、非ブロック積（`_UNBLOCKED`）を持つ GEMMul8。
`OPENMX_GEMMUL8_TIMING` / `_FORWARD_TIMING`	0 / 0	一般の GEMM と前向き GEMM のプロセスごとの統計（呼び出し数、秒、GFlop、GEMMul8 と cuBLAS の内訳、法の数、スケーリング）を実行終了時に出力する（ランク表示は `OMPI_COMM_WORLD_RANK` か `PMI_RANK`）。
`OPENMX_GPU_VERBOSE` / `OPENMX_GEMM_VERBOSE` / `(OPENMX_)GEMMUL8_VERBOSE`	0	GEMMul8 ブリッジの詳細ログ（作業領域の判断、退避、forward 段の要約）。4 つの名前をこの順に読み、後のものが前のものを上書きする。
バンド計算(Band_DFT_Col.c / Band_DFT_NonCol.c)		
`OPENMX_BAND_GPU_DIAG`	auto	**1**=見積り判定を無視して GPU 密行列経路を強制、**0**=常に ELPA/ScaLAPACK(§5.6)
`OPENMX_BAND_GPU_SWITCH_NUM`	800	コリニアバンド密行列経路の GPU/CPU 切替次元(専用ノブのない経路は `GPU_CPU_SWITCH_NUM`=1000 のまま)。RTX 5080 を 8 ランク共有した実測クロスオーバーは n ≈ 300〜400
`OPENMX_BAND_NONCOL_GPU_SWITCH_NUM`	800	ノンコリニアバンド密行列経路の同ノブ。2n と比較(実測クロスオーバーは 2n ≈ 250〜400)
`OPENMX_BAND_GPU_DM`	1	**0**=コリニアバンドの ScaLAPACK/ELPA フォールバック時の DM 蓄積を CPU ループに戻す。既定ではフォールバックでも DM は GPU 上で行う(フルソルブが載らなくても固有ベクトルパネルは載るため)
`OPENMX_BAND_NONCOL_GPU_DM`	1	**0**=ノンコリニアの ScaLAPACK/ELPA フォールバック multi-k ループの k 点ごと DM 蓄積を CPU ループに戻す(k-dense GPU フローは元々デバイス上で DM を蓄積する)

環境変数	既定値	意味
`OPENMX_BAND_GPU_RESERVE_MB`	max(総 VRAM/10, 1536)	並列度決定・fits 判定で空けておく予約量の引き上げ
`OPENMX_BAND_GPU_RANK_OVERHEAD_MB`	256	per-rank 所要量見積りに加算するランタイムオーバーヘッド
`OPENMX_BAND_SYEVDX_NOVECTOR`	1	固有値のみ必要な第 1 パスを NOVECTOR で解く(0 で旧動作)
`OPENMX_BAND_GPU_MAX_CONCURRENT_K` / `OPENMX_BAND_GPU_TURN_GROUP` / `OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE`	制限なし	同時実行ランク数の**明示上限**(この優先順で最初に見つかったもの)。旧版の「初期値」の意味は廃止され、実際の並列度は常に適応スキャンで決まる
`OPENMX_GPUSOLVER_SERIAL_GPU_TURNS`	1	all_knum==1 経路の GPU ターン直列化(0 で並行投入)
`OPENMX_BAND_GPU_PERSISTENT` / `_MIN_FREE_MB`	0 / 2048	SCF 反復間のデバイスバッファ常駐(共有 GPU では非推奨)
`OPENMX_BAND_GEMMUL8_TURN_RELEASE`	1	ターン毎の GEMMul8 ワークスペース解放(0 で保持)
`OPENMX_BAND_PROFILE`	0	性能計測カウンタの表示。ランクごとの段階行 SETHPROF(Set_Hamiltonian)、SETHOTFPROF / SETHOTFDENPROF(その場計算のバッチ: 詰め替え・評価・カーネル)、SETNLPROF(Set_Nonlocal: DS_NL・受信原子の積・局所の積)、CLUSTERDMPROF(クラスターソルバの密度行列の集計)、CLUSTERDISTPROF(固有ベクトルの配布。密な求解を持たないランクでは求解待ちを含む)、FORCEHNL2PROF(Force_HNL 第 2 ケースのバッチ: 構築・デバイス・後処理)、truncation と Total_Energy の区間タイマー、SETVPOTPROF(Set_Vpot: xc、Vxc の D→B コピー、VNA、組み立て、混合 FFT、B→C 再分配)、SETDENPROF(Set_Density_Grid: 判定・求積・A→B・重ね合わせ・B→D)も共用
`OPENMX_BAND_COL_KCACHE_MB` / `OPENMX_BAND_NONCOL_KCACHE_MB`	1024 / 1024	GPU バンド経路の k 点ごとの稠密固有ベクトルパネルのホストキャッシュ(ランクあたり MiB。`/proc/meminfo` のランクあたり利用可能メモリでも制限)。0 でキャッシュ無効。
`OPENMX_BAND_COL_OVERLAP_CACHE_MB` / `OPENMX_BAND_NONCOL_OVERLAP_CACHE_MB`	512 / 512	SCF ステップ間で再利用する k 点の重なり変換(とその固有値)のホストキャッシュ(ランクあたり MiB。0 で無効)。
`OPENMX_BAND_NONCOL_PACK_CACHE_MB`	128	非共線バンド: 1 SCF ステップ内の k グループ間で、各稠密 k 所有ランクが保持するパック済み実空間行列のホスト複製の上限(ランクあたり MiB。全所有ランクが収まるときだけ保持)。
`OPENMX_BAND_REFINE_BASIS_MB`	1024	精密化固有値ソルバーのウォームスタート基底を k 点スロットごとに保持するホスト予算(ランクあたり MiB。共線・非共線バンド)。0 でウォームスタートのキャッシュ無効。
`OPENMX_BAND_CACHE_TRACE` / `OPENMX_BAND_NONCOL_CACHE_TRACE`	0 / 0	k 点キャッシュのヒット数と大きさをランクと SCF ステップごとに 1 行出力する(非共線では `BAND_NONCOL_CACHE` 行)。
`OPENMX_BAND_NONCOL_INTERLEAVE_K`	auto	非共線バンド: 稠密 k 点を解く順序を所有ランク間で交互に並べる。0 で k の自然順。
クラスタ計算(Cluster_DFT_Col.c / Cluster_DFT_NonCol.c)		
`OPENMX_CLUSTER_GPU_DIAG`	auto	**1**=実予約失敗でも GPU 強制(失敗時は診断つき abort)、**0**=常に ELPA(§5.6)
`OPENMX_CLUSTER_GPU_SWITCH_NUM`	400	コリニアクラスタ密行列経路の GPU/CPU 切替次元(実測では n=240 まで全サイズで GPU が勝利)
`OPENMX_CLUSTER_NONCOL_GPU_SWITCH_NUM`	400	ノンコリニアクラスタ密行列経路の同ノブ。2n と比較(実測では 2n=480 まで GPU が勝利)
`OPENMX_CLUSTER_GPU_DIAG_RESERVE_MB`	1024	予約プローブ後に残すべき空き VRAM
`OPENMX_CLUSTER_GPU_DIAG_SERIAL`	1	2 スピンが同居できないときの直列化モード(0=無効、2=直列強制)
`OPENMX_CLUSTER_GPU_DM`	1	DM/EDM の GPU 蓄積(0 で CPU ループ)

環境変数	既定値	意味
`OPENMX_CLUSTER_GPU_DM_ALL`	1	クラスターの密度行列を全ランクがデバイスで蓄積する(自分の状態スライスを半減リトライでステージ)。0 で密な求解を持つランクだけ、他は CPU ループ(原子でスレッド分割)
`OPENMX_CLUSTER_GEMMUL8_TURN_RELEASE`	1	ソルブ毎の GEMMul8 ワークスペース解放(未設定時は BAND 設定を継承)
`OPENMX_CLUSTER_GPU_DM_GEMM`	1	非共線クラスタの密な経路の密度行列を、占有状態の積 $W^{H}W$(GEMMul8)1 回と疎な成分の取り出しで作り、全ランクの総和の代わりにルートのバッファを配る。**0** で従来の成分ごとの状態和(RTX 5080 の sidia333 で 1 ステップ 432 ms)
`OPENMX_CLUSTER_PROFILE`	0	クラスター稠密 GPU ソルバーの段階別プロファイル(重なり、収集、前後の変換、固有値分解、解放、密度行列)をランクごとに出力する。各段階の前後でデバイスを同期するため、プロファイルなしの実行時間とは異なる。
`OPENMX_CLUSTER_GPU_RETAIN_SCRATCH`	auto	共線クラスター: 密度行列の完成後に稠密作業領域(固有ベクトルの退避、一時配列)をデバイスに残すか。未設定 = デバイスの空きが半分未満なら解放、**1** = 常に保持、**0** = 常に解放(変換済み重なり行列は常に保持)。
`OPENMX_GEMM_SAMPLE_DIR` / `_ITERS`	未設定(オフ) / 1	診断用: 変換済み重なり X(共線)と固有値ソルバーが受け取る変換済みハミルトニアン(`X_g<geometry>.bin`、`Hs2_scf<step>.bin`、列優先バイナリ)を、`_ITERS` に列挙した SCF ステップ(カンマ区切り)について既存のディレクトリへ書き出す。`tests/eigen_precision_probe.cu` の入力になる。
精密化 FP32 固有値ソルバー(eigen_refine_gpu.c)		
`OPENMX_EIGEN_REFINE`	FP32:FP64 の性能比が 8 以上の GPU(GeForce、RTX PRO 6000 Blackwell など)で、そのソルバーの種類の GEMMul8(共線は実数、非共線は複素)が有効なら 2、それ以外は 0	クラスタの密な固有値計算(共線はスピンごと、非共線)とバンド法(1 ランク 1 k 点: 全 k 点の推定値を集めてから補正。1 ランクに複数 k 点: 1 回目のパスは前ステップの化学ポテンシャルで補正して基底をホストに保持(ランクあたり `OPENMX_BAND_REFINE_BASIS_MB` MB、既定 1024)、2 回目のパスはその基底からこのステップの化学ポテンシャルで補正): K > 0 で n 個(非共線は 2n 個)の固有対を FP32 で解き、必要な MaxN 個を荻田・相島法で K 回補正(積は GEMMul8、モジュール `eigen_refine_gpu.c`)、部分占有のクラスタはレイリー・リッツで回転。SCF の 2 ステップ目以降は、前のステップの固有ベクトルが新しいハミルトニアンに対して残差 2e-5 以内ならそこから補正を始める(`OPENMX_EIGEN_REFINE_WARM`)。SCF を終えるステップは FP64 でもう一度解き、`scf.maxIter` の最後のステップは FP64。スピン分極では 2 つのスピンワールドのオーナーが固有値の推定値を交換して化学ポテンシャルを求める(バンド法では全 k 点の推定値を集めてから補正する)(1 ランクで両スピンを解く場合、直列スピンモード、XANES、空軌道・空状態の指定時は FP64 のまま)。**0** でオフ。RTX 5080(18 ランク)の非共線 sidia333(2n = 5616)で SCF ステップ数は同じまま 261.3 秒が 197.9 秒。バンド法は必要な maxn 列を補正し、残りの n − maxn 列を毎ステップ補正済みの列に対して回転させる(`OPENMX_EIGEN_REFINE_COMPLEMENT`、既定 1): 放置すると、その列が持つ FP32 程度の占有部分空間成分のせいで補正列の不動点が占有部分空間から 1e-8 ほどずれ、SCF 反復が 2〜5 回増え、ウォームスタートに偏りが出ていた。k 点オーナーが GPU の 1 ターン群に入りきらないときは、1 ランク 1 k 点の解も複数 k 点経路と同じ 2 パス(前ステップの化学ポテンシャルで補正して基底をホストへ、そこから 2 パス目)に分けるので、16 GB のカードでも補正される。診断: `OPENMX_EIGEN_REFINE_CHECK=1` で補正列の残差と直交性を占有クラスごとに FP64 の積で表示、`OPENMX_EIGEN_REFINE_ALL=1` で全列を補正

環境変数	既定値	意味
`OPENMX_EIGEN_REFINE_UNTIL`	0(オフ)	この値を表示上の NormRD が下回ると、その SCF サイクルの残りを FP64 の固有値計算に戻す。既定ではオフで、補正ステップで SCF が終わった場合はそのステップのハミルトニアンを FP64 でもう一度解き(固有値計算 1 回、密度行列なし)、SCF 後のエネルギー密度行列と力はその固有ベクトルから作る 反復補正の積・固有値・最終固有ベクトルが NaN/Inf の場合、保存した FP64 行列を上書きする前に補正を棄却し、FP64 経路へ復帰する。正常時の演算と精度基準は変更しない。
`OPENMX_EIGEN_REFINE_WARM`	1	補正つき固有値計算のウォームスタート。新しいハミルトニアンに対する残差が 2e-5 以下なら、FP32 で解き直さずに前のステップの固有ベクトルから補正を始める(A100 の sidia333 で固有値計算の段階が 1 ステップ 706 → 140 ms、22 ステップ中 20)。**0** で毎ステップ FP32 で解く。バンド法でもウォームスタートする(`OPENMX_EIGEN_REFINE_WARM_BAND` 、既定 1)。 `OPENMX_EIGEN_REFINE_WARM_PERIOD` =N で N ステップごとに FP32 で解き直す(既定 0 = 解き直さない: 補正しない列を補正済みの列に対して回転させるようにしてから、ウォームスタートを続けても sidia333 で FP64 と 2e-12 Ha で一致する)
`OPENMX_EIGEN_REFINE_STOP_GUARD`	100	補正 FP32 ステップで SCF の停止判定が通ったとき、表示される NormRD がこの倍率 × `scf.criterion` 以下であることも要求する。OpenMX の dUele だけの判定は偶然で通ることがある(RTX 5080 の sidia333_col_cluster: 15 ステップ目で dUele 7e-12、NormRD 4.3e-8 のまま 5 回早く停止し 2e-9 Ha ずれた)。0 でオフ。また、バンド法は SCF サイクルの終わりにウォームスタート用の基底を解放する(非共線バンドの基底が残ったままだと 16 GB のカードで Force3 のメモリが足りなかった)
`OPENMX_EIGEN_REFINE_UNBLOCKED`	1	補正の積を、GPU に余裕があれば分割なしの GEMMul8 作業領域で計算する(256 MiB 上限の分割計算より約 1.7 倍速い)。**0** で分割計算
`OPENMX_EIGEN_FP32_TWO_STAGE`	1	補正つき固有値計算の FP32 全解法に 2 段アルゴリズムの cusolverDnXsyevd(CUSOLVER_ALG_2。cuSOLVER 12.3.4 以降。RTX 5080 の n = 5616 で 638 ms→323 ms)を使う。**0** = 1 段の cusolverDnXsyevdx。2026-10-09 以降、2 段解法が収束しなかったとき(info > 0。RTX 5080 で散発的、例: 2808 次元のバンド問題で info=297)は、FP64 行列から変換し直して 1 段アルゴリズムで一度だけやり直し、それでも失敗したときに FP64 へ落ちる。以前は 1 回の失敗で SCF サイクルの残りが FP64 になっていた(sidia333 band_on の D で Diagonalization 68 秒→95 秒)
`OPENMX_EIGEN_REFINE_WARM_BAND`	1	精密化固有値ソルバーのウォームスタートをバンドソルバー(k 点スロットごと)でも使う。0 でクラスターソルバーのみ。
`OPENMX_EIGEN_REFINE_WARM_PERIOD`	0	バンドのウォームスタート: この SCF ステップ数ごとにウォームスタートの代わりにコールドな FP32 分解をやり直す(0 = しない)。
`OPENMX_EIGEN_REFINE_WARM_DELTA_FLOOR`	1	ウォームスタート時もコールドな FP32 分解のクラスター閾値 delta を下限として使う(ウォームスタートの残差ははるかに小さく、クラスター連鎖が消えてしまうため)。0 で下限なし。
`OPENMX_EIGEN_REFINE_ALL` / `_COMPLEMENT`	0 / 1	精密化ブロックの診断用: `_ALL=1` で占有ブロックではなくすべての固有ベクトル列を精密化し、 `_COMPLEMENT=0` で精密化する列の補空間を受け持つ第 3 の積ブロックを使わない。
`OPENMX_EIGEN_REFINE_CHECK`	0	診断用: Rayleigh-Ritz 段の後で精密化したベクトルの残差 $\|A x - \lambda x\| / \|x\|$ と直交性を計算して出力する。
cuSOLVER の方針(openmx_cusolver_compat.c、gpusolver_Syevdx.c)		

環境変数	既定値	意味
`OPENMX_CUSOLVER_EMULATION`	1	コードが作るすべての cuSOLVER ハンドルで FP64 の固有値解法を cuSOLVER 12.2+ の固定小数点 FP64 エミュレーション (CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH、performant 戦略、動的仮数制御)で実行する。GeForce で FP64 参照 R0 が現実的な時間で済むのはこのため。**0** = ネイティブ FP64。エミュレーションの解法はまれに収束しない(info > 0: RTX 5080 の sidia333 で、バンド Col info=395、バンド NonCol info=126。2026-10-08 に約 700 回の解法中 2 回)が、ソルバーは info ≠ 0 で中止する。そこで 2026-10-08 以降、cusolverDnXsyevd / cusolverDnXsyevdx の互換ラッパーは、エミュレーションハンドルでの FP64 解法ごとに行列の写し(デバイス上、足りなければホストメモリ)を取り、info > 0 なら行列を戻してネイティブ FP64 でもう一度解き(メッセージ "solving again in native FP64")、ハンドルをエミュレーションに戻す。作業領域の問い合わせは両モードの大きい方を返すので再試行は収まる
`OPENMX_CUSOLVER_ALGORITHM`	auto	cuSOLVER 12.3.4 以降の syevd アルゴリズム: `legacy` / `0` = params オブジェクトなし(旧インターフェース)、`auto` = 1 段/2 段の切り替えを cuSOLVER に任せる(`_TWO_STAGE_MIN_N` 未設定時の既定)、`one-stage` / `1` 、`two-stage` / `2` 。
`OPENMX_CUSOLVER_TWO_STAGE_MIN_N`	未設定 (cuSOLVER の判断)	手動の切り替え点: この次元以上の行列は 2 段 syevd、それ未満は 1 段を使う(不正な値はメッセージを出して無視)。
`OPENMX_CUSOLVER_CACHE_MB`	64	ホスト配列ソルバー呼び出し間で保持する cuSOLVER 作業領域のランクあたり上限(MiB)。それより大きな分解は解放し、バンド・クラスターのアリーナや力のバッファの余地を残す。
`OPENMX_CUSOLVER_VERBOSE`	0	cuSOLVER の方針(エミュレーションの可否、アルゴリズムの選択)とクラスター作業領域解放のメッセージ。
`OPENMX_CUSOLVER_RETRY_TEST`	0	テスト用: エミュレーション分解のたびにネイティブ FP64 への再試行を実行する。
DC 法(Divide_Conquer.c)		
`OPENMX_DC_GPU_THRESHOLD`	400	GPU 化する局所クラスタ次元 NUM の下限(コリニア)
`OPENMX_DC_NONCOL_GPU_THRESHOLD`	300	GPU 化するスピノル次元 2·NUM の下限(非コリニア)
`OPENMX_DC_GPU_S_CACHE` / `_RESERVE_MB`	1 / 4096	S12 パネルのデバイス常駐キャッシュとその予算予約
`(OPENMX_)GPUSOLVER_MIN_FREE_AFTER_MB`	1536	固有値ソルバ用確保後の最低空き VRAM(preflight)
`(OPENMX_)CUBLAS_MIN_FREE_AFTER_MB`	3072	cuBLAS 用の同上
DC-LNO 法(Divide_Conquer_LNO.c)		
`OPENMX_DCLNO_GPU_THRESHOLD`	800	GPU 化する局所次元の下限(コリニア)
`OPENMX_DCLNO_NONCOL_GPU_THRESHOLD`	400	GPU 化するスピノル次元の下限(非コリニア)
`OPENMX_DCLNO_GPU_TURN_RELEASE`	1	フェーズ末尾のソルババッファ+GEMMul8 解放(未設定時は BAND 設定を継承)
`OPENMX_DCLNO_PROFILE`	0	DC-LNO GPU 経路の SCF 単位の集計行(全ランクが同じ報告用集団通信に参加する)。
Krylov 法(Krylov.c)		
`OPENMX_KRYLOV_GPU_FUSED`	1	原子別パイプラインの融合パス(0 で per-call シーケンス)
`OPENMX_KRYLOV_GPU_KU_CACHE` / `_KU_RESERVE_MB`	1 / 4096	Krylov 基底のデバイス常駐キャッシュとその予算予約
`OPENMX_KRYLOV_GPU_DGEMM_MIN_FLOPS`	5×10^{7}	GEMM を GPU に投げる下限(m·n·k)
`OPENMX_KRYLOV_GPU_EIGEN_MIN`	300	部分空間対角化をデバイスで解く射影次元 Msize3 の下限(§7.6)
`OPENMX_KRYLOV_GPU_PROFILE`	0	融合パスの内訳(KRYFUSE 行)出力

環境変数	既定値	意味
Set_Hamiltonian(Set_Hamiltonian.c / set_hamiltonian_gpu.cu)		
`OPENMX_SETHAM_MATRIX_GPU`	1	dVH+Vxc+VNA 格子積分の GPU 実行(0 で CPU)
`OPENMX_SETHAM_CUDA_KERNEL`	1	専用 CUDA カーネル(0 で OpenACC カーネルのみ。数値同一)
`OPENMX_SETHAM_GPU_RESIDENT` / `_FLOOR_MB`	1 / 4096 or 16384	SCF 不変表のデバイス常駐と、常駐後に空けておくフロア(対角化需要の公開有無で既定が変わる。§7.7)
`OPENMX_SETHAM_GPU_MAX_RANKS_PER_DEVICE`	制限なし	波の同時ランク数の明示上限
`OPENMX_SETHAM_GPU_SERIAL_WAVES`	0	1 で全波を GPU 直列実行(既定は第 1 波 GPU+残余ランク CPU 並行のハイブリッド)。A100 でメモリ不足を模擬すると(8 ランク中 2〜3 ランクだけが丸ごと GPU に入る)、sidia333 の 12 反復で既定のハイブリッド 165 秒に対し直列ウェーブは 37.6 秒だった。RTX 5080 で 11〜14 ランクが CPU に退避している状況はこれと同じ
`OPENMX_SETHAM_STREAMING`	1	GPU に丸ごと入るランクが 1 つだけのとき、対の束をストリーミングして計算する(ランクあたり 256 MiB、`OPENMX_SETHAM_STREAM_MB`)。**2** で、全ランクが入りきらないとき全般にストリーミングする(A100 の模擬: ハイブリッド 165 秒、直列ウェーブ 37.6 秒に対し 52 秒)。**0** でオフ
`OPENMX_SETHAM_ONTHEFLY`	1	軌道テーブルがデバイスに入りきらないランクは、CPU で計算したりテーブルをストリーミングしたりする代わりに、原子対のバッチごとに軌道の値をデバイス上で計算する(Set_Orbitals_Grid のデバイスコード、orbs_grid_gpu.h)。常駐するのは自分の原子の球内格子点の索引(数 MiB)と `OPENMX_SETHAM_ONTHEFLY_MB`(既定 128)MiB のバッチ作業領域だけ。Set_Density_Grid の密度もそのランクでは同じタイルから計算する。0 でオフ(従来どおり CPU かストリーミング)。各ランクの作業領域はこの上限と「計画時の空きからステージング組のピークと余裕 `OPENMX_SETHAM_ONTHEFLY_MARGIN_MB`(既定 0)MiB を除いた分け前」の小さいほうで、明示した上限と実際の空きメモリの取り分を超えない(予算はステージング組が確保中に測った値ではなく計画値から決めるので全ランクで同じになる)。大きい原子対は重なり格子点を分割し、確保の失敗時は CPU にフォールバックする。全ランクのテーブルが載らない GPU では、波の外のランクが上限分の作業領域を得られるまでステージング組を縮める(相の時間を決めるのはステージング組ではなくその場計算のランク: RTX 5080 で 1 ステップ 0.65 s 対 1.4 s)。その場計算の状態は SCF サイクルの終わりに解放する。評価は角度の cos/sin を座標から直接作り、動径メッシュの索引は対数メッシュから直接求める(二分探索なし)。ホストの Get_Orbitals と double の最下位ビットまで一致する
`OPENMX_SETDEN_ONTHEFLY_MB`	128	その場計算の密度のバッチ作業領域上限(MiB)。実際の予算は空きからノード共通の 512 MiB を除いてその場計算のランクで等分し、各ランクの固定密度行列・出力領域と実行時の余裕 64 MiB を引く。明示する上限は 32 MiB 以上で、それ未満は既定値を保つ。対全体が既定の上限を超える場合は、空きメモリを利用して全対が 1 バッチに収まる大きさまで自動拡張する。明示した上限は拡張しない。なお収まらない対は重なり格子点を分割するため、大きい原子や明示した上限だけを理由にランクが辞退することはない(§7.9)

環境変数	既定値	意味
`OPENMX_SETHAM_ONTHEFLY_PLAN_CACHE`	1	その場計算の経路はランクごとに「計画」を保持する: バッチ(どの対か、各オフセット)、対の記述子、点ごとの索引(Nc、行列要素ではポテンシャルの格子添字 MN も)を、デバイスのアリーナと同じ配置のピン留めホスト画像としてバッチごとに 1 つ持ち、バッチごとに 1 回の転送で送る。毎ステップ動くのは H ブロック(ピン留めバッファ)、ポテンシャル行(コード自身のピン留めステージングバッファ経由。OpenMX が持つ行を cudaHostRegister で登録する方式は、後の無関係な転送が CUDA_ERROR_INVALID_VALUE で散発的に失敗した)、タイルだけで、密度の累積配列はデバイス側でゼロ化し、そのホストバッファはピン留めして持ち越す。計画は SCF サイクルの終わりまで、または作業領域・スピン数・原子分割・軌道格子の形状が変わるまで生きる(ランクの対の重なり点あたりホストメモリ約 8 バイト)。以前は毎ステップ各バッチを組み直し、約 90 回の小さな同期転送(Nc リストは対ごと)で送っており、18 ランクの MPS 下では 1 回約 0.3 ms かかっていた: RTX 5080(sidia333 非共線クラスター)で行列要素の詰め替えがランク・ステップあたり 0.25 秒→0.10 秒、密度の詰め替えが 0.20 秒→0.02 秒、その場計算ランクの行列要素 0.68 秒→0.48 秒、密度 0.41 秒→0.26 秒、Set_Density_Grid 18.1 秒→14.8 秒、D 107.7 秒→103.2 秒(band_nc の D は 144.4 秒→133.8 秒)、Utot は 1e-12 Ha 以内。**0** = 呼び出しごとに計画を作り直す
`OPENMX_SETHAM_MATEL_SPLIT`	4	df 行列要素カーネルで、対の格子点を何ブロックに分けるか(各ブロックは自分の分の 32 点タイルを積分し、部分和を H ブロックへ原子的に加える)。1 = 対あたり 1 ブロック、上限 64
`OPENMX_ORBS_EVAL_PRECISION`	df / fp64	その場計算の軌道値評価(常駐テーブルのないランクがハミルトニアンと密度のタイルに使う Set_Orbitals_Grid のデバイスコード)の演算。**df** = double-float(既定)。FP64 で求めた格子座標の後はすべて float の対(hi + lo、相対 2^-48 程度)で計算する: PAO の動径テーブルとメッシュ点ごとのスプライン定数を対で保持し、メッシュ添字は FP32 の対数で見積もって添字探索で確定し、軌道値として書く float は積の hi 部、すなわち FP64 値に最も近い float(FP64 値が丸め境界の ~1e-14 以内にあるときだけ 1 ulp 異なる。約 10^6 個に 1 個。最初のメッシュ点より内側の点は FP64 ルーチンで計算)。**fp64** = ホストの Get_Orbitals のFP64 転記(テーブルの値とビット一致)。GeForce は FP64 が FP32 の 1/64 で、FP64 評価はそのパイプで律速される: RTX 5080 の sidia333 非共線クラスター(18 ランク + MPS、23 ステップ)で、ランク・ステップあたりの評価が 0.77 秒→0.41 秒(ハミルトニアンのタイル)、0.42 秒→0.07 秒(密度のタイル)、Set_Hamiltonian 43.5 秒→35.8 秒、Set_Density_Grid 32.6 秒→24.1 秒、D 141.4 秒→129.5 秒で Utot は 1e-12 Ha まで同一。band_nc の D は 167.5 秒→156.8 秒。テーブル自体(常駐ランクの Orbs_Grid)は常に FP64 で作る
`OPENMX_XC_GRID_GPU`	1	格子上の GGA-PBE 交換相関ポテンシャル(Set_XC_Grid。毎 SCF ステップ Set_Vpot から、また Total_Energy / Force から呼ばれる)をデバイスで計算する: 27 点の勾配ステンシル、点ごとの汎関数(XC_PBE・XC_PW92C・XC_EX のデバイス転記)、発散のステンシル、非共線の回転をランクの D 格子上の 4 つの OpenACC カーネルとして、ホストと同じ FP64 で、一時的な acc_malloc アリーナ(格子点あたり double 約 22 個)から実行する。LDA、LDA+U の cFLL 回転(dc_Type 3/4)、XC_P_switch 3、アリーナを確保できないときはホストのループに戻る。ホストのループはランクごとに単一スレッドで、RTX 5080(18 ランク、sidia333 非共線クラスター)では全ランクで 1 ステップ 0.36 秒かかっていたが、デバイスでは数ミリ秒: Set_Vpot が 1 ステップ 0.35 秒→0.18 秒(残りは格子の再分配)、Set_Hamiltonian 24.9 秒→20.2 秒、D 117.1 秒→111.1 秒。1 ランクで全格子なら 1 ステップ 2.38 秒→0.21 秒。Utot はホスト計算と 1e-12 Ha で一致(デバイスライブラリの pow / exp / log が glibc と最後の桁で異なる)。**0** = ホストのループ

環境変数	既定値	意味
`OPENMX_GRID_PRECISION`	df / fp64	格子積分カーネル(dVH+Vxc+VNA の行列要素と密度)の演算: **df** = double-float(FP32 演算に誤差なし変換を組み合わせる: hi/lo 分割、FMA による積、タイルごとの補償付き和、タイル間は FP64。1 項あたり FP32 命令約 12 で FP64 相当の結果)。FP32:FP64 の性能比が 8 以上の GPU(GeForce)での既定。**fp64** = FP64 カーネル(データセンター GPU の既定) 2026-10-08 以降、df カーネルは項あたりの FP64 演算を持たない: 行列要素カーネルはタイルごとに grid_vol*vpot を一度だけ (hi, lo) に分けて重みを FMA で作り、密度カーネルは行の和を float の対(TwoSum)で累積して原子的加算だけ FP64、lo 側は単純加算(項あたり FP32 命令 10)。RTX 5080(18 ランク、sidia333 非共線クラスター)でその場計算ランクの行列要素がステップあたり 1.42 秒→0.61 秒、Set_Hamiltonian 35.8 秒→24.9 秒、D 129.5 秒→116.3 秒、Utot は FP64 カーネルと 1e-12 Ha 以内 2026-10-09 以降、df モードのその場計算の密度は CUDA カーネル (set_hamiltonian_gpu.cu の density_kernel)で積分する: 対の密度行列をブロックごとに一度だけ共有メモリに (hi, lo) の float 対で置き、点の軌道行はレジスタ(16 / 32 / 48 軌道の段階)、演算は同一。RTX 5080 でランク・ステップあたり 0.37 秒→0.098 秒(1 ランクでは 3.55 秒→0.23 秒)、Set_Density_Grid 21.9 秒→17.8 秒、Utot は 1e-12 Ha 以内。OpenACC カーネルはフォールバックとして残る 2026-10-09 以降、df の行列要素カーネルはレジスタタイリングする: 1 スレッドが要素の 2x2 ブロック(spin, i0..i0+1, j0..j0+1)を持ち、格子点ごとに重み付き phi0 の対 2 つと phi1 の値 2 つを読んで 4 つの積を作る(共有メモリ読み出しが半分)。対の格子点は OPENMX_SETHAM_MATEL_SPLIT 個のブロック(既定 4)に分け、各ブロックの部分和を H ブロックに原子的に加える(対あたり 1 ブロックではステージングの gather を隠せず遅かった)。RTX 5080 の sidia333 非共線クラスターで、カーネルは 18 ランク時にその場計算ランク・ステップあたり 0.24 秒→0.11 秒(1 ランクでは 2.88 秒→1.09 秒)、行列要素は 0.48 秒→0.35 秒、Utot は同一
`OPENMX_GRID_FP32`	0	実験用: 前ステップの NormRD が `OPENMX_GRID_FP32_UNTIL` (1e-5)より大きい間は素の FP32 カーネル、以後 FP64(`OPENMX_GRID_FP32_RESTART` =1 で切替時に混合履歴を再開)。FP32 のステップはカーネルの誤差の水準で SCF が停滞する(sidia333 で 23 回が 27〜40 回)ためオフ。設定された SCF 最終ステップは `scf.maxIter=1` の場合も非損失の格子カーネルを使う。残差が NaN/Inf の場合も FP32 を終了する。FP32 ステップ完了後に `_SCF_keywords` で maxIter を引き下げた場合、そのステップを遡って再計算することはしない
`OPENMX_SETHAM_GPU_RESERVE_MB` / `_RANK_OVERHEAD_MB`	max(総/10,1536) / 512	予約量とランクあたりオーバーヘッド見積り
`OPENMX_SETHAM_GPU_KOWNER_ONLY`	0	1 で旧来の k オーナー限定 GPU 実行に戻す
`OPENMX_SETHAM_BASE_GPU`	0	H の base 結合を GPU 実行(実測では CPU が速い)
`OPENMX_SETHAM_GPU_TEST_NEED_MB` / `_FROM_CALL`	—	偽のフェーズ需要を注入するテスト用フック
`OPENMX_SETHAM_STREAM_MB`	256	ストリーミングする対バッチのランクあたり作業領域（MiB、最小 8）。
`OPENMX_SETHAM_ONTHEFLY_MB` / `_MARGIN_MB`	128 / 0	その場計算の行列要素: ランクのバッチ作業領域の上限（MiB、最小 32）と、その場計算の予算から除外しておくデバイスメモリ（0: 作業領域は一時的でソルバー実行前に解放される）。
`OPENMX_SETHAM_FP64_TILED` / `_SPLIT`	auto / auto	FP64 行列要素カーネル: タイル版を使うか（auto = A100（sm_80）のみ）と、対ごとの格子点方向のブロック数（auto = SM 数から決定、最大 64）。
`OPENMX_SETDEN_FP64_CUDA` / `_SPLIT_SPIN`	1 / auto	その場計算の密度: 専用 FP64 CUDA カーネル（0 で汎用経路）と、そのスピン分割（auto = 原子あたり 32 軌道超で分割。A100 で実測）。

環境変数	既定値	意味
`OPENMX_GRID_FP32_UNTIL` / `_RESTART`	1e-5 / 1	実験的な FP32 格子段（`OPENMX_GRID_FP32=1`）: この NormRD より大きい間は素の FP32 カーネルを使う。`_RESTART` は FP64 への切り替え時に混合履歴をやり直すか。
C++ コアのホスト格子カーネル（openmx_grid_assembly.c）		
`OPENMX_SETHAM_SIMD` / `OPENMX_DENSITY_GRID_COL_SIMD` / `_NC_SIMD`	auto	C++ コアのホスト格子カーネル（ハミルトニアン行列要素、共線・非共線の密度）の SIMD バックエンド: `scalar`、`avx2`、未設定 = CPU が対応する最も広いもの（AVX-512、AVX2、scalar）。
`OPENMX_SETHAM_SIMD_TRACE` / `OPENMX_DENSITY_GRID_COL_SIMD_TRACE` / `_NC_SIMD_TRACE`	0	それぞれのカーネルで選ばれた SIMD バックエンドを rank 0 が一度出力する。
その他の行列要素構築		
`OPENMX_SETNL_OPENACC`	1	Set_Nonlocal の GPU バッチ(0 で CPU パス)
`OPENMX_SETPRO_GPU`	1	Set_ProExpn_VNA の 3 バッチ(0 で CPU)
`OPENMX_SETPRO_VERBOSE`	0	Set_ProExpn_VNA のデバイス段ごとにランク 1 行を stderr へ: 採用判定の数値(必要量、予算、空き、ペアバッチ、ランクターン)、バッチの大きさ、所要時間、その時点のデバイス空き容量
`OPENMX_OLPKIN_RADIAL_GPU`	0	Set_OLP_Kin の動径積分を GPU 実行(スレッド数 1 必須。転送律速で通常は CPU が速い)
`OPENMX_SETNL_BESSEL_CACHE_MAX` / `OPENMX_OLPKIN_BESSEL_CACHE_MAX`	128 / 128	対の距離をキーとする球ベッセル表キャッシュのスレッドあたりエントリ数（Set_Nonlocal、Set_OLP_Kin）。0 でキャッシュ無効。
`OPENMX_ORBS_GRID_GPU` / `_VERBOSE`	1 / 0	Set_Orbitals_Grid の GPU 実行（統合対カーネルと一時アリーナ。L0 ≤ 3、Mul ≤ 8 の軌道。0 で CPU）と、経路が有効・失敗したときの一度だけの通知。
`OPENMX_UCELL_OLG_GPU` / `_VERBOSE`	1 / 0	truncation.c: UCell_Box の重なり格子の交差計算を GPU で行う（CPU 結果とビット単位で一致。0 で CPU）と、その一度だけの通知。
密度グリッド(Set_Density_Grid_GPU.c)		
`OPENMX_DENSITY_GRID_GPU`（別名 `OPENMX_SETDENSITY_GPU`）	1	密度グリッド GPU 全体のゲート
`OPENMX_DENSITY_GRID_GPU_LOCAL`	1	分散求積モード: 0=無効、1=共有テーブル常駐時のみ、2=常に
`OPENMX_DENSITY_GRID_GPU_RESERVE_MB` / `_PERSISTENT`	256 / 0	オーナーサービスの予約量と計算後のデバイス常駐
`OPENMX_DENSITY_GRID_GPU_CACHE_MB`	64	所有者サービス方式の密度キャッシュ: `_PERSISTENT` 未設定時、この大きさ（MiB）までの幾何テーブルを SCF ステップ間でデバイスに残す。それより大きいエポックは縮約のたびに稠密固有値ソルバーへ領域を譲る。
`OPENMX_DENSITY_GRID_GPU_LOCAL_CACHE_MB`	32	分散（ローカル）密度モード: ランクごとに保持する CSR とバッファの上限（MiB。空き VRAM の小さな割合でさらに制限）。0 で呼び出しごとに解放。
全エネルギー・力(Total_Energy.c / Force.c)		
`OPENMX_TOTALENERGY_GPU`	1	全エネルギー系デバイスカーネルの一括ゲート(NC 含む)
`OPENMX_FORCE_GPU`	1	力計算の全 GPU 経路の親ゲート(集団判定)
`OPENMX_FORCE3_GPU` / `OPENMX_FORCE3_GPU_DORB`	1 / 1	Force3 トレースの GPU 化 / dOrbitals のオンデバイス再計算
`OPENMX_FORCE4B_GPU` / `OPENMX_FORCE4B_FUSED`	1 / 1	Force4B の GPU バッチ / CPU 側融合トレース(GPU バッチの前提でもある)
`OPENMX_FORCEHNL_GPU`	1	Force_HNL の GPU バッチ
`OPENMX_FORCE_PROFILE`	0	[Force profile] 行(ステージ別 max/avg 秒)の出力

環境変数	既定値	意味
`OPENMX_FORCE_RELEASE_SCF_CACHES`	1	バンド計算: 共有軌道テーブルの最後の利用者である Force3 の後、VNA バッチを受け入れる前にそのデバイスメモリを返す（密度キャッシュも無効化）。0 で保持。
`OPENMX_FORCE4B_ARCHIVE_GPU`	auto	Force4B: アーカイブした射影子積をデバイスに置く（auto = デバイスの FP64 が弱い GeForce などを除いて置く。0/1 で強制）。
`OPENMX_FORCE4B_CASE1_STREAM` / `_STREAM_MB`	auto / 1024	Force4B ケース 1（常駐対）: バッチ全体が入らないとき、ハローのソースを上限付きデバイスアリーナ経由でストリーミングする（auto。0 = しない、1 = 常に）。`_STREAM_MB` はアリーナの上限（MiB）。
`OPENMX_FORCE4B_CASE1_TURNS` / `_TURN_MAX_RANKS`	auto / 制限なし	Force4B ケース 1: 共有デバイス上のランク交代（auto = 全ランクの常駐アリーナが同時に入らないとき。0/1 で強制）と、1 回の交代で動くランク数の上限。
`OPENMX_FORCE4B_CASE2_STREAM`	auto	Force4B ケース 2: バッチ全体が入らないときのストリーミング（auto。0 = しない、1 = 常に。親の `OPENMX_FORCE4B_GPU` が優先）。
電荷/DM 混合(Mixing_H.c ほか)		
`OPENMX_MIX_FAST`	1	RMM-DIIS/DIISK/DIISV/GR-Pulay/Kerker の CPU 高速パス(0 でレガシー)
`OPENMX_MIXH_GPU`	auto	RMM-DIISH の層選択: 未設定=自動、0=レガシー、1=GPU 層強制、2=増分 CPU 層強制
`OPENMX_MIXH_GPU_RESERVE_MB` / `_VERBOSE`	max(総/10,1536) / 0	GPU 層の予約量 / 選択結果の表示
gpusolver2(ELPA-GPU / COSMA)		
`OPENMX_GS2_ELPA_SOLVER`	2	gpusolver2 の ELPA ソルバ選択。1 で 1stage、それ以外は 2stage(既定)
`OPENMX_CLUSTER_GS2_FALLBACK`	auto	クラスター法の `gpusolver` で密な固有値計算が GPU 1 枚に入らないときの代替: まず rank 0 のノードの GPU で MAGMA、それが使えなければ(マルチノード実行で)ジョブの全ランクによる分散 gpusolver2(ELPA GPU カーネル + COSMA。各ランクの GPU を使う。スピン分極の共線は全ランクのグリッドで両スピンを順に解く)に集団的な GPU メモリ判定のうえで切り替える。`off` = 分散段に進まない、`on` = 単一ノードでも進む、`force` = MAGMA を飛ばす。強制 MAGMA(gpusolver3、`OPENMX_MGPU_MEMORY=host`)は分散段に進まない。`OPENMX_CLUSTER_HOST_MEM_AVAILABLE_MB` で MAGMA のホストメモリ判定の値を上書きできる
`OPENMX_GS2_HOST_RESERVE_MB`	ノードメモリの 1/4(最小 4096)	gpusolver2 ブリッジ(明示指定時と容量フォールバック時): COSMA が実行終了まで保持するホストバッファ(メモリプールと COSTA の送受信バッファ。COSMA 自身の分割戦略から見積もる)を確保した後も各ノードに残すべき空きメモリ。足りなければその変換は ScaLAPACK で行う
`OPENMX_GS2_ELPA_GPU` / `OPENMX_GS2_GEMM_GPU`	auto	gpusolver2 の ELPA 固有値ソルバー / COSMA 行列積を GPU に強制（**1**）、CPU カーネル / ScaLAPACK に強制（**0**）。未設定ならメモリ判定で決める。設定は rank 0 から全ランクへ配布されるのでグリッド内で食い違わない。
`OPENMX_GS2_GPU_FACTOR` / `_FIXED_MB`	3 / 128	ELPA GPU ソルバーのデバイスメモリモデル: ローカルのブロック巡回行列の FACTOR 倍と固定分（MiB）。実測の約 2 × local + 約 110 MB/ランクに校正。
`OPENMX_GS2_GPU_RESERVE_MB`	512	GPU 判定で ELPA/COSMA の確保後に残っていなければならない空きデバイスメモリ。
`OPENMX_GS2_GPU_TILE` / `COSMA_GPU_MAX_TILE_M` / `_N` / `_K` / `COSMA_GPU_STREAMS`	1024 / 5000 / 5000 / 5000 / 2	COSMA へ渡す Tiled-MM のタイル上限（`COSMA_GPU_MAX_TILE_M/N/K` が未設定ならこの値を設定する。利用者の設定が優先）と COSMA のストリーム数。どちらも GPU GEMM 作業領域の見積もりに使う。

環境変数	既定値	意味
`OPENMX_GS2_GPU_VERBOSE` / `OPENMX_GS2_PROFILE`	0 / 0	gpusolver2 の GPU/CPU 判定をすべて報告する（遷移時だけでなく）/ 受け入れと準備を含むホスト配列呼び出しごとの時間を計測する（グリッドのルートの壁時計）。
`COSMA_CPU_MAX_MEMORY`	未設定（COSMA の既定）	COSMA 自身のホストメモリ上限（MiB）。ブリッジは CPU 退避の計画に使う。
`OPENMX_CLUSTER_HOST_MEM_AVAILABLE_MB`	計測値	テスト用: MAGMA の容量判定で rank 0 のノードの利用可能ホストメモリを計測せず、この値（MiB）とみなす。
マルチ GPU ランタイム・ワーカー・協調積（openmx_mgpu_runtime.cu、openmx_gpu_workers.c、openmx_mgpu_gemm.cu）		
`OPENMX_MGPU`	auto	ノード内の複数 GPU 協調（ワーカーと協調ソルバーが借りるデバイスプール）。**0** で禁止、**1** で探索が却下するデバイスも含めて強制。未設定なら入力キーワード（`scf.Gpu.Num` と GPU 入力スイッチ）で決まる。
`OPENMX_MGPU_NUM_GPUS`	可視な全デバイス	プールのデバイス数の上限（`CUDA_VISIBLE_DEVICES`、`scf.Gpu.Num` と併用）。
`OPENMX_MGPU_BAND_SHARED`	0	バンド: 小さな GPU プールで K 個の k 点グループに min(K, floor(G/2)) 個のロック付き GPU グループを時分割で共有させる（UUID ロックが重複する所有者を直列化する）。
`OPENMX_MGPU_SOLVER` / `_MIN_N` / `OPENMX_MGPU_TUNE_TRIALS`	single / 4096 / 2	クラスター所有ランクの固有値分解をノードの GPU 群で協調して解く: `single` / `off` = 所有ランクのネイティブソルバー（既定）、`magma` = マルチ GPU の MAGMA ソルバー、`auto` = 行列形状ごとに cuSOLVER と MAGMA を一度計時して（`_TUNE_TRIALS` 回、最大 3）速い方を採用。`_MIN_N` 未満の行列は強制しない限りネイティブ。
`OPENMX_MGPU_MEMORY`	device	`host` = 所有ランクのデバイスに入る場合でも、クラスター稠密分解を MAGMA 経由のホスト常駐方式（gpusolver3 の容量段）で実行する。
`OPENMX_MGPU_GEMM` / `_MIN_N`	off / 8192	デバイス常駐の行列積を許可された GPU 群へ分散する（実験的）: `on` / `1` = すべてのサイズ、`auto` = 形状と NVLink の経験則（max(M,N) ≥ `_MIN_N`、min(M,N) ≥ その半分、K ≥ `_MIN_N`、双方向 P2P）、**0** = しない。ホスト常駐の MAGMA 積と gpusolver2 の変換はこの設定に関係なく協調する。
`OPENMX_MGPU_GEMM_BACKEND`	bridge	ワーカーのローカル積の演算: `bridge`（ブリッジ経由の GEMMul8、既定）または `native`（ローカル cuBLAS）。
`OPENMX_MGPU_GEMM_WORKSPACE_MB` / `_RESULT_MB` / `_HOST_RESULT_MB` / `_RESULT_HOST` / `_HOST_MB` / `_RESERVE_MB`	1024 / 2048 / 65536 / 0 / 256 / 512	協調積の予算: GPU あたりの A+B+C タイルバッファ、ソース GPU 上の完全な結果の上限（超えるか `_RESULT_HOST=1` なら結果は `_HOST_RESULT_MB` までホストメモリに置く）、ワーカーあたりの固定（pinned）ステージング、空きデバイスメモリの予約。
`OPENMX_MGPU_GEMM_FORCE_HOST` / `_ASYNC_GATHER` / `_TRAVERSAL` / `_PAGEABLE`	0 / 1 / auto / 0	転送の詳細: ピアアクセスがあってもオペランドをホスト経由にする（`_FORCE_HOST=1`、テスト用）、CUDA のホスト結果ダウンロードをまとめてキューに入れる（`_ASYNC_GATHER`）、ホストタイルの走査順（`auto`、`row`、`column`）、固定ではなくページング可能なホスト行列（`_PAGEABLE`）。
`OPENMX_GPU_WORKERS`	auto	原子・格子のローカル作業の GPU ワーカー: 未設定 = 1 台に入りきらないときだけプールの他のデバイスを借りる（容量）、**1** = 1 台で足りる場合も性能のために分散、**0** = 借りない。
`OPENMX_GPU_WORKER_OVERLAP`	auto	ハードウェア方針が許すデバイスでワーカーの準備段階を前の作業と重ねる（auto = ネイティブ FP64 で較正済みの CUDA デバイス。0/1 で上書き）。
`OPENMX_GPU_WORKER_COPY`	0	**1** でワーカー間のピアビューを無効化する（すべてコピーで転送。テスト用）。
`OPENMX_GPU_LOCK_DIR` / `_TIMEOUT_MS`	/tmp / 300000	プロセス間でデバイスの受け入れを直列化するノードローカルなロックファイルのディレクトリ（ノード内のランクが共有する絶対パス）と、受け入れを諦めるまでロックを待つ時間（ms）。

環境変数	既定値	意味
`OPENMX_MGPU_VERBOSE`	0	マルチ GPU ランタイムと各ワーカー実行の報告（名前、ワーカー数、項目数、デバイス）。`level.of.stdout 2` でランタイム部分は有効になる。
`OPENMX_GPU_CAPACITY_TEST_RECOVERY`	0	回帰テスト用の故障注入: 従来の常駐高速経路を飛ばし、通常の容量プランナーを実際の GPU メモリで動かす。
MAGMA ホスト常駐ソルバー（openmx_magma_mgpu.cpp）		
`OPENMX_MAGMA_FP32` / `_COMPLEX_MAX_N`	0 / 0	ホスト常駐の MAGMA 分解を GEMMul8 で補正した FP32 で行う（ハードウェア条件あり。FP32 精密化積が必要）。`_COMPLEX_MAX_N` を超える複素行列はネイティブ FP64 のまま（0 で上限なし）。
`OPENMX_MAGMA_REFINE_STEPS` / `OPENMX_MAGMA_RR_GPU_MIN`	2 / 1024	FP32 MAGMA 経路の補正回数（2〜5）と、Rayleigh-Ritz 段を CPU LAPACK ではなく受け入れ済み GPU 群で解くクラスターの大きさの下限。
`OPENMX_MAGMA_REFINE_RESIDENT_MB` / `_RESERVE_MB`	0 / 4096	デバイス常駐の精密化作業領域: `_RESIDENT_MB` までの積を各ワーカーに常駐させ（0 = 常にステージング）、デバイスごとに `_RESERVE_MB` を空けておく。
`OPENMX_MAGMA_HOST_CACHE_MB`	64	MAGMA がプロセスごとに分解間で保持する固定（pinned）ホスト作業領域の合計上限。
`OPENMX_MAGMA_VALIDATE_FULL` / `_FAILURE_DUMP`	0 / 未設定	MAGMA の結果検証: **1** で 4 本の標本固有ベクトルの 32 行ではなく全行を検査する。検証に失敗したら入力・ベクトル・固有値を `_FAILURE_DUMP` のファイル（絶対パス）へ書き出す。
プロセス起動と自己テスト（openmx.c）		
`OPENMX_ZHEEVD_SELFTEST`	0	HIP 版: 起動時と最初の DFT 呼び出し前に zheevd の自己テストを実行する（rank 0）。
`OPENMX_GPU_HEAP_REEXEC` / `LIBOMPTARGET_HEAP_SIZE`	1 / 未設定	HIP 版: `LIBOMPTARGET_HEAP_SIZE` が未設定なら openmx は 64 MiB のオフロードヒープを設定して自分自身を再実行する。`OPENMX_GPU_HEAP_REEXEC=0` で再実行しない。
ランチャーから読む値（プロセスごと。配布しない）		
`OMPI_COMM_WORLD_LOCAL_RANK` / `MV2_COMM_WORLD_LOCAL_RANK` / `SLURM_LOCALID` / `PMI_LOCAL_RANK`	—	ノード内ランク番号（この優先順）。MPI を使わずにランクをデバイスへ割り当てるために読む。
`OMPI_COMM_WORLD_LOCAL_SIZE` / `MV2_COMM_WORLD_LOCAL_SIZE` / `SLURM_NTASKS_PER_NODE` / `PMI_LOCAL_SIZE`	—	ノード内のランク数（同じ優先順）。デバイスごとの予算に使う。
`OMPI_COMM_WORLD_RANK` / `PMI_RANK`	—	ワールドランク。GEMMul8 統計の表示にのみ使う。
`HOSTNAME`	—	`gethostname` が使えないときの EXX ログ用ホスト名。

表 10-1: GPU 関連環境変数の一覧(サブシステム別)

10.3 標準出力の GPU 診断行

コミット `30148e1e` ほかで、どのステージが GPU 実行されたかが標準出力から読み取れるよう整備された。どこまで出力するかは OpenMX の入力キーワード `level.of.stdout` に従う(stderr への通知も含む。§10.2 の `OPENMX_*_PROFILE` と `*_VERBOSE` 系の環境変数はこれとは独立に働く):

`level.of.stdout`	出力される GPU 関連の行
1(既定)	OpenMX 本来のステージ行に付く "(GPU-accelerated)"・"(GPU reduction)" の表記、エラー、GPU が使えないこと（`GPU SOLVER requested, but GPU initialization failed on … MPI ranks; using ELPA2.`）、デバイスメモリ不足や失敗で GPU 固有値ソルバを断念したこと（`Falling back to the ELPA/ScaLAPACK diagonalization…`、DC・DC-LNO のランクが CPU ソルバへ切り替えたこと)、精錬固有値ソルバの基底なしで GPU の密対角化を続けること（`… holds its GPU diagonalization buffers but not the … basis of the refined eigensolver …: FP64 eigensolver for this SCF cycle`）

level.of.stdout	出力される GPU 関連の行
2	さらにデバイス計画・キャッシュ採用・バナー、デバイス初期化時のカーネルスタック上限の引き上げ (<GPU> kernel stack limit raised from … to … bytes per thread)、SCF を終えるステップの FP64 再求解、個々のステージ・求解のフォールバックと再試行(ノードごとに 1 行。Force4B、Force_HNL、Set_ProExpn_VNA、Set_Density_Grid、Mixing_H、GEMMul8 の workspace fallback、cuSOLVER のエミュレーション再試行と FP32 再試行)
3	さらに SCF ステップごと・k 点ごと・ランクごとの詳細: 精錬 FP32 固有値ソルバの精錬残差とウォームスタート (このゲート導入前の 18 ランク -runtestL では 85,600 行中約 38,000 行)、Force4B・Force_HNL・HVNA のフォールバック通知と Set_Density_Grid のその場計算作業領域の通知での各ランク自身の数値、作業領域が変わるたびのその場計算バナー

level 2 以上では OpenMX 自体が原子ごとのデバッグ出力を出す(216 原子のノンコリニア sidia333 クラスター、18 ランクで 4 SCF ステップ: level 1 で 726 行、level 2・3 で約 51,000 行)ので、level 3 の GPU 詳細が増やす量はわずかである。tools/run_forward_variants.sh は要約がこれらの行を読むため、指定がなければ level 3 で実行する。代表的な行(いずれも Host_ID または各デバイスの代表ランクが出力):

- ステージバナー (level 1): <Set_Hamiltonian> Hamiltonian matrix for VNA+dVH+Vxc (GPU-accelerated)... 、 <MD=…> Calculation of the VNA projector matrix (GPU-accelerated) 、 Force calculation #2 (GPU reduction) / #3 (GPU-accelerated) / #4 (GPU-accelerated) / #5 (GPU reduction) 、 Force calculation #6/#7 (GPU-accelerated) (Total_Energy 側)
- 並列度の決定(level 2、値が変わったときのみ): <Band_DFT_Col> GPU device 0: 8 k-owner rank(s), GPU concurrency=3, turn group=3, per-rank=1.9 GiB, peak=5.7 GiB, free=7.4 GiB, reserve=1.5 GiB (NonCol はモード名付き、Set_Hamiltonian は Hamiltonian rank(s) 表記)
- 計画とキャッシュ採用(level 2): <DC> GPU S-cache: rank 0 keeps 12 of 16 cluster overlaps on the device (812.5 MiB) 、 <Krylov> GPU KU-cache: … 、 <Set_Hamiltonian> GPU resident cache: … 、 <Set_Density_Grid> distributed GPU quadrature: every rank integrates its own atoms 、 <Set_Density_Grid> one-rank GPU service cache: owner=… atoms=… atom-grids=… terms=… device=… GiB
- フォールバック通知: Falling back to the ELPA/ScaLAPACK diagonalization… (§ 5.6、level 1)、 Force4B GPU: not enough free device memory …; CPU fallback. (level 2 はノードごとに 1 回、level 3 は各ランクが Force4B GPU: … GiB free for … rank(s) but … GiB needed per rank; CPU fallback. を出力)、 <Mixing_H> The RMM-DIISH residual history … does not fit on the GPU … (level 2) 、 <Set_Density_Grid> … cannot hold a minimum on-the-fly chunk … と <Set_Density_Grid> CPU loop on … rank(s) with atoms: … (level 2 は状況が変わったときに Host_ID が 1 行、level 3 は各ランク自身の行も。§ 7.9)、GEMMul8 の workspace fallback (stderr、level 2 または OPENMX_GEMMUL8_VERBOSE=1)

> **読み方:** 「GPU で走っているつもりなのに遅い」ときは、level.of.stdout 2 で実行して まずこれらの行を確認する。(a) ELPA フォールバック行が出ていれば VRAM 不足で対角化が CPU に落ちている、(b) GEMMul8 の workspace fallback が出ていれば GEMM が FP64 に落ちている、(c) concurrency=1 が出ていれば共有ランクが直列化されている、(d) <Set_Density_Grid> CPU loop on … が出ていれば密度を CPU で積分したランクがある(§ 7.9) — が典型である。

10.4 チューニング指針

1. **まず既定値で走らせる** — 既定値は「16 GB 級 GPU を多数ランクで共有」する構成に合わせて調整されており、並列度・常駐・フォールバックはすべて自動で決まる。旧版のように環境変数で並列度を手動指定する必要は原則なくなった。
2. **stdout / stderr の診断行を確認する**(§ 10.3)— チューニングの出発点は常にここ。
3. **VRAM に余裕がある(GPU 占有)場合** — OPENMX_BAND_GPU_PERSISTENT=1 、 OPENMX_GPUSOLVER_SERIAL_GPU_TURNS=0 、 OPENMX_*_GEMMUL8_TURN_RELEASE=0 で転送・再確保を減らせる余地がある。Krylov 系の対角化は既定(OPENMX_KRYLOV_GPU_EIGEN_MIN =300)でデバイスに移るようになった(実測 117.5 s → 78.8 s。§ 7.6)。さらに下げて得をする系は稀。

4. **VRAM が厳しい場合** — ランク数を減らすのが最も効く。次いで予約系 (`*_RESERVE_MB`)を既定より上げて GEMMul8 フォールバックを防ぐ、`OPENMX_SETHAM_GPU_RESIDENT=0` で常駐を切る、など。
5. **再現性を最優先する場合** — `OPENMX_GEMMUL8_DISABLE=1` で GEMM を FP64 に固定し、メモリ状況によるエンジン切替の影響を排除する(§4.8)。GPU⇄ELPA の切替も含めて固定したい場合は `OPENMX_BAND_GPU_DIAG` / `OPENMX_CLUSTER_GPU_DIAG` を明示する。
6. **問題の切り分け** — GPU 経路のどこかを疑う場合は、サブシステム別ゲート (`OPENMX_SETHAM_MATRIX_GPU=0`、`OPENMX_FORCE_GPU=0`、`OPENMX_MIX_FAST=0` など)を 1 つずつ 0 にして二分探索できる。すべての高速パスはレガシーコードを温存している。

10.5 scf.eigen.lib=gpusolver2 — マルチノード分散クラスター対角化

単一 k 点しか必要としないコリニア/ノンコリニアの**クラスター計算**では、従来の gpusolver 経路は 対角化を 1 台の GPU で開始し、容量不足時には MAGMA や分散ソルバへ切り替えられる。`scf.eigen.lib gpusolver2` は分散クラスター対角化を明示選択し、**ELPA 2026.02.002(CUDA または HIP の GPU カーネル、新しいオブジェクト型 C API)**と**COSMA(通信最適分散 GEMM、GPU バックエンド)**へ差し替え、全ランク・全 GPU で解く。元の CUDA 版の対象は筑波大 CCS の Pegasus(1 ノード = H100 × 1、多ノード)のような構成。CUDA は選択した NVHPC/MPI、HIP は ROCm flang と互換の MPI を使う(§9.7)。

- **設計** — 既存 ELPA 経路の骨格(2 次元ブロックサイクリック分散、`myworld1` による 2 スピン並行、固有ベクトル再分配、DM 構築)をそのまま流用し、差し替えは呼び出し点のみ。Col: S 対角化 → `openmx_gs2_eigen_real(n,n,…)`、変換 3 GEMM → `cosma_pdgemm_`、変換後 H(固有ベクトルは下位 MaxN 本、固有値は全数)→ `openmx_gs2_eigen_real(n,MaxN,…)`。NonCol: 実 S 対角化 + 実 GEMM ×12 + 複素 2n×2n の `openmx_gs2_eigen_complex(n2,MaxN,…)` + 逆変換 `cosma_pzgemm_`。S 小固有値のクランプなど OpenMX 独自の数値処理は不変。ブリッジは新設の `elpa_cosma_bridge.c` に集約(ELPA ハンドルの (BLACS ctx, n, nev, 型) 別キャッシュ、`nvidia-gpu=1` または `amd-gpu=1`、`use_gpu_id` 設定、2stage ソルバ既定)。
- **CPU/GPU 共通の ELPA** — `elpa1`、`elpa2`、`gpusolver2` はすべて 同梱の ELPA 2026.02.002 ライブラリを新しい C API から使う。`openmx_elpa_cpu.c` が CPU ソルバ、`elpa_cosma_bridge.c` が GPU バックエンドを選ぶ。旧 ELPA 2018.05 の埋め込み実装は削除済み(§9.4)。
- **適用範囲とエラー停止** — 対象はクラスター本流(Col/NonCol)のみ。`scf.EigenvalueSolver` が cluster 以外だと入力段階でエラー停止する(Band + `scf.Kgrid 1 1 1` の自動クラスター化は通る)。クラスター対角化以外(SetH/密度/力/混合などの GPU 経路、CWF 等の付加処理)は gpusolver と完全同一 (内部では `scf_eigen_lib_flag=GPUSOLVER` のまま `gpusolver2_flag` で対角化だけを切替)。GPU 不在時は gpusolver 同様 ELPA2 へ降格する。
- **DLA-Future を採用しなかった理由(記録)** — 当初 DLA-Future + COSMA で実装・検証したが、DLA-Future の非同期集団通信は HPC-X の Open MPI 4.1 では 3 ランク以上で必ず破綻する (MPI_ERR_TRUNCATE/ハング。PMPI トレースで当方の通信列の正しさまで確認済み。Open MPI 5 でのみ解消)。MPI 総入れ替えを避けるため、Open MPI 4 系で実績のある ELPA の GPU カーネルを採用した。COSMA は Open MPI 4.1 で問題ないことを検証済み(np1/4/8 で ScaLAPACK と 1e-12 以下の一致)。
- **自己完結ビルド** — `third_party/dist/` に ELPA・COSMA・COSTA・Tiled-MM・cmake の 公式リリースアーカイブを同梱(計約 23 MB、`README.md` と `SHA256SUMS` 付き)。`make openmx` が初回に `make gpusolver2-stack` を自動実行し、ネット接続なしで `third_party/gpusolver2-install/` に構築する。ELPA は GitLab タグアーカイブのため autoconf/automake/libtool/m4/python3 が必要(Ubuntu は標準装備)。主な Makefile 変数: `GPUSOLVER2_GPU_ARCH`(既定 nvidia-smi 自動検出; H100 は 90)、`GPUSOLVER2_HOST_CC/CXX`(COSMA 用 GCC、既定 gcc/g++)。
- **実行時ノブ** — `OPENMX_GS2_ELPA_SOLVER=1` で ELPA 1stage ソルバに切替(既定は 2stage)。表 10-1 参照。

- **Pegasus での想定手順** — `module load nvhpc` の後、`cd source && make openmx && make install`(初回のみスタック構築)。実行は従来どおり HPC-X の `mpirun` でよい。GPU アーキは `make GPUSOLVER2_GPU_ARCH=90 gpusolver2-stack` のように明示できる。

11. 制約・注意点の総まとめ

11.1 ビルド・実行環境の前提

- CMake は `CUDA` または `HIP` を選び、CPU 専用バックエンドは持たない。両実行ファイルとも `OPENMX_GPU=0` または `elpa1` / `elpa2` で 共通の新 ELPA を使う CPU 実行ができる(§9.4・§9.8)。
- CUDA は NVHPC 26.9 + CUDA 13.4 が既定。HIP は ROCm・互換 MPI・AOCL を使う(§9.7)。NVHPC 変更時は §6.5 の CUDA 側ミスコンパイル回避一覧を再検証すること。
- GEMMul8 の CUDA/HIP ビルドと native FP64 の選択は対象デバイスに従う。アーキテクチャ変更時は GPU ライブラリを再ビルドする。CUDA 固有の制御がそのまま HIP に適用されるわけではない。
- 全コアにランクを置けるあいだはフラット MPI が第一選択。デバイスメモリの都合でランク数がコア数を下回るときは、OpenMP スレッド(`-nt`。1 ランクを複数コアに束ねる: `mpirun --map-by ppr:6:node:PE=3 --bind-to core -np 6 ./openmx in.dat -nt 3`)がホスト側の段階で効く(§10.4): Set_OLP_Kin の Bessel キャッシュはスレッドごとになり、クラスターの密度行列と Set_Nonlocal の受信原子の積は原子で スレッド分割され、vanilla の OpenMP 領域(truncation、Set_ProExpn_VNA、Set_Orbitals_Grid、Force、Total_Energy)は CPU 版と同じく動く。デバイス側の段階はスレッド数で変わらず、スレッドありでは vanilla の OpenMP リダクションの 加算順序により実行ごとに 1e-11 程度の非決定性が出る。混合の高速パスの一部(バッチ Gram)は 1 スレッドのとき レガシーと加算順序が一致する。Set_OLP_Kin のオプトインの OpenACC 動径積分(`OPENMX_OLPKIN_RADIAL_GPU=1`)は 引き続き 1 スレッドを要求する。

11.2 メモリ設計の目安

- 1 ランクあたりの VRAM 需要(バンド計算・次元 n): 複素密行列 $16n^2$ バイト × 3 本 + syevdx ワークスペース(`_bufferSize` の実照会値。目安 $4n^2$ 複素要素)+ GEMMul8 ワークスペース(約 1 GiB、複素は実数の約 3 倍) + オーバーヘッド 256 MiB。この見積りは `BandCol_GpuTurnRequiredBytes` がそのまま実装している。
- 共有 GPU では「同時ランク数 × 上記」が空き VRAM に収まる必要があるが、現在は**全経路が自動で 並列度を縮退・直列化・ELPA フォールバックする**ため、収まらない設定でもハングや abort にはならない (性能が落ちるだけ)。旧版にあった「Cluster root-dense や DC-LNO には自動絞り込みがない」という制約は解消済み。
- ホストメモリにも注意: Set_Hamiltonian / 密度グリッドの共有テーブルは数 GiB 級で、CPU 実行ランクに構築させない制御(Ready プローブ)が入っている。ランク数を増やすとホスト側の ワークスペース(混合履歴・クラスタスクラッチ等)がランク数倍で効く。

11.3 数値再現性

- **ビット単位の再現を壊し得る要因:** (1) GEMMul8 ⇄ cuBLAS FP64 のメモリ状況による切替、(2) GPU ⇄ ELPA の実行時フォールバック(verdict は SCF 内 sticky だが、実行ごとのメモリ状況で変わり得る)、(3) scatter-add の `acc atomic`(加算順非決定)、(4) S の小固有値処理の経路差、(5) GEMMul8 の環境変数設定差(num_moduli / fastmode)。
- **再現に配慮した設計:** DM 構築・格子トレースの `loop seq` 逐次縮約、力バッチの「float 積 → double 加算」のキャスト位置まで含めた CPU 再現、混合高速パスのポインタ回転 (ビット単位の同値性)、cublasDgeam 転置(純データ移動)、GEMMul8 自体の bitwise 再現性、ベッセルテーブルの bit 一致キャッシュ。多くの GPU 化コミットは「Uele/力が CPU とビット一致」を 検証条件にしている。
- 検証時は `OPENMX_GEMMUL8_DISABLE=1` と十分な VRAM 余裕、必要なら `OPENMX_*_GPU_DIAG` の明示固定で条件を固定するのが確実。

11.4 ハング・異常終了

- `wait_cudafunc` は恒久エラーで無限ループする(§3.6)。ただし「VRAM が恒久的に足りない」ケースは 有界リトライ+フォールバック(`TryDeviceMalloc` 系、`Force_gpu_arena_wait` の 180 秒上限)に 移行済みで、無言のフリーズになる箇所は大幅に減った。力ステージには abort し得るデバイス確保が存在しない(§7.11)。
- 固有値計算失敗時の挙動は経路により abort / CPU フォールバック / 素通りと異なる(表 5-2)。
- fits 判定・波計画・混合 preflight は内部で MPI 集団操作を行うため、**全ランクが同じ回数だけ到達する**ことが 前提。経路改修時に片側のランクだけスキップさせるとデッドロックする(§5.9)。
- collective 版デバイス割当関数は全ランク一斉呼び出しが前提(片側だけ呼ぶとデッドロック)。
- `cudaGetLastError()` のスレッド別ラッチは「回復済み失敗」の残骸を含み得る。自前のカーネル検査を追加する場合は起動前に pop する作法を守ること(§3.6、§7.7)。

11.5 統合された AMD HIP バックエンド

AMD 実装はリポジトリに取り込み済み。ソースマニフェストと CMake が対象ファイルを選び、HIP 互換ヘッダが CUDA の include パスへ混入することを防ぐ。CUDA と HIP は別実行ファイルであり、実行時にベンダーを切り替える方式ではない。

層	CUDA	HIP
ランタイムとカーネル	CUDA、NVHPC OpenACC	HIP、ROCm OpenMP target offload
密行列ライブラリ	cuBLAS / cuSOLVER	hipBLAS / hipSOLVER と互換ラッパ
GEMMul8	CUDA 実体化	HIP 実体化、MI300A/Radeon 別の演算方針
分散ソルバ	ELPA NVIDIA カーネル / COSMA CUDA	ELPA AMD カーネル / COSMA ROCm
CPU 固有値ソルバ	共通の ELPA 2026.02.002。GPU オプションは無効。	

`hip_runtime_compat.h`、`hipblas_compat.h`、`hipsolver_compat.h` が C ソースへ必要なインターフェースを公開し、C++ 翻訳単位は ROCm のヘッダを使う。hipSOLVER ラッパは generic 呼び出しを FP32/FP64・実数/複素数の型別 API に変換する。CUDA 名のヘッダを include する 同梱ライブラリ向けには shim ヘッダも残す。GEMMul8 と native FP64 は AMD のデバイス方針で選ぶ。第 3〜6 章の OpenACC プール管理や cuSOLVER エミュレーションなどは NVIDIA 固有の説明として読むこと。

GPU が見えない環境での CPU 実行: 非共線密行列ワークスペースの後処理は、GPU リソースを持たない場合に HIP 同期とデバイス復元を呼ばない。使用中または部分確保されたリソースは引き続き解放し、ホスト状態は常に 初期化する。これにより、GPU を非表示にした HIP 実行ファイルの CPU 経路を修正した (`57a3a3fc`、検証は §9.8)。

MI300A: GPU 実行時は `HSA_XNACK=1` を設定する。`source/backends/hip/tools/mi300a_rank_launcher.py` は OpenMPI のランク情報と `OPENMX_AFFINITY_DIR` を使って ROCm 初期化前に CPU 配置を固定し、割当 CPU 数と `-nt` の一致を確認する。起動例は `docs/GPU_BACKENDS.md` を参照。NVIDIA MPS は CUDA 固有で、HIP の起動機構ではない。記録された Radeon 試験は MI300A や 複数ノードの実機検証を意味しない(§9.8)。

11.6 オリジナルから挙動が変わる非 GPU 項目

- OpenMP 並列時のヒープ破壊バグ 2 件が修正済み(§8.1)— オリジナルにも存在するバグ。
- 起動時にスタックのソフトリミットが自動で引き上げられる(§8.2)。
- runtest の判定・実行手順が厳密化されている(§8.3)。
- バージョン文字列は "4.0 GPU"。

11.7 休眠・準備コード一覧(改修時の誤解防止)

コード	状態と位置づけ
`gpusolver_Syevd` (Xsyevd 版)	呼び出し 0 件(Syevdx 版に統一された)。旧式の「大域 rank % デバイス数」割当が残る
`LU_Zinverse_GPU`	実装完了・呼び出しコメントアウト。NEGF 表面グリーン関数の GPU 化布石(§5.8)
DMmu/CWF/LNO 系の GEMMul8 ヘルパ、`BandCol_GpuSolver_Zgemm_OpenACC` など	定義のみ(将来の GEMM GPU 化の準備)。`my_cublasDgemm/Zgemm` は削除済み、`openmx_cuda_kernels.cu` はツリーから撤去済み(§6.4)
DC-LNO の GPU プロキシ機構	`DCLNO_ENABLE_SHARED_GPU_PROXY 0` で温存(実験用)
`use_force4b_openacc = 0` (Force.c)	Force4B の旧 OpenACC 経路の恒久無効化定数(現行は GPU バッチ+融合 CPU トレースが実経路)
`OutData_Binary.o` / `getDeviceCount()` 宣言	デッドリンク / デッド宣言
`OPENMX_SETHAM_GPU_TEST_NEED_MB` 系フック	テスト専用(フェーズ需要の注入)。運用では使わない

表 11-1: 休眠・準備コードの一覧

付録 A. 変更ファイル一覧(2026 年 7 月、47 ファイル)

+n/−m はオリジナル OpenMX 4.0 ソースとの生の diff 行数(2026-07-23 時点)。全面再整形されたファイルは raw 行数が実質変更より大きく見える点に注意。分類: **GPU**=GPU 化、**OMP**=OpenMP 修正、**FIX**=バグ修正・堅牢化、**OPT**=CPU 最適化、**BLD**=ビルド、**MISC**=その他。

ファイル	+n/−m	分類	変更概要
`Force.c`	+15,123/−11,241	GPU	力 #2/#3/#4/#4B/#5/HNL の GPU 化(融合トレース、チャンクバッチ、dOrbitals オンデバイス、アリーナ+ピア待ち)。raw 行数の一部は全面再インデント
`Divide_Conquer.c`	+4,816/−3,038	GPU	DC 法の GPU 化: 原子別クラスタの Xsyevdx、GEMMul8→cuBLAS→CPU 多段 GEMM、S12 デバイス常駐キャッシュ、cublasDgeam 転置、sticky 無効化
`Band_DFT_Col.c`	+4,724/−2,544	GPU	コリニアバンドの全面 GPU 化: 密行列構築キャッシュ、Xsyevdx(第 1 パス NOVECTOR)、GEMMul8 三重積、適応並列度、ELPA フォールバック
`Band_DFT_NonCol.c`	+3,582/−88	GPU	非コリニアバンドの GPU 化: OpenACC 常駐 root-dense 経路、ステージ別(固有値/固有ベクトル/DM)適応並列度、ELPA フォールバック
`Set_Hamiltonian.c`	+2,972/−1,149	GPU	dVH+Vxc+VNA 格子積分の GPU 化(既定有効): 適応波スケジューリング、SCF 不変キャッシュ、オンデバイス Vpot ギャザー、常駐キャッシュ+需要協調、packed H/S キャッシュ API
`Cluster_DFT_NonCol.c`	+2,424/−218	GPU	単一アリーナの root-dense 対角化、GEMMul8 × 12、S/Ss2・index・ハンドル・ワークスペースの反復間キャッシュ、固有ベクトル常駐/退避、ELPA フォールバック
`Divide_Conquer_LNO.c`	+2,315/−1,896	GPU	direct per-rank dispatch(コリニア)+非コリニアクラスタ解の GPU 化(スピン倍化変換+Zgemm×3)、共有ランク群の合算メモリ判定
`Set_ProExpn_VNA.c`	+2,029/−0	GPU	DS_VNA 構築・HVNA トレース・VNA2/3 の 3 バッチ GPU 化(球ベッセルのデバイス漸化、Gaunt/変換表のホスト表引き化)
`Set_Nonlocal.c`	+1,738/−305	GPU/FIX	非局所射影子積分のバッチ GPU 化(デバイス生成球ベッセル+Gaunt 事前表)+固定配列のヒープ化
`Cluster_DFT_Col.c`	+1,534/−90	GPU	root-dense GPU 対角化、実予約プローブ+ELPA フォールバック、2 スピン直列化、DM/EDM の GPU 蓄積、固有ベクトルのデバイス常駐
`Mixing_H.c`	+1,350/−0	GPU	RMM-DIISH 混合の 2 層高速パス(GPU 常駐 Gram / 増分 CPU)
`Krylov.c`	+840/−24	GPU	原子別射影解パイプラインの融合(GEMM×3+Xsyevd を 1 回のデバイス訪問)、KU キャッシュ、スレッド別ワークスペース
`Set_OLP_Kin.c`	+528/−180	GPU/OPT	動径積分の OpenACC 化(オプトイン)+bit 一致ベッセルテーブルキャッシュ
`Total_Energy.c`	+367/−123	GPU/OMP	EH0 二中心バッチ・EXC/EH1・双極子・CWF-Dc・Exc0 細メッシュの GPU 縮約呼び出し(NC 対応)+OpenMP private 修正
`DIIS_Mixing_DM.c`	+265/−2	OPT	RMM-DIIS のバッチ Gram(1 掃引+1 Allreduce)+残差履歴のポインタ回転
`Mixing_V.c`	+246/−53	OPT	RMM-DIISV の増分 Gram、Kerker(VKSk)のシフト高速化
`DFT.c`	+235/−4	GPU	GPU デバイス初期化、Set_Hamiltonian 作業ランク選定、力計算直前の GPU キャッシュ一括解放フック、VNA バナー、NonCol 固有ベクトル scatter
`DIIS_Mixing_Rhok.c`	+228/−44	OPT	RMM-DIISK の増分 Gram(新残差行のみ dgemv+短い Allreduce)
`GR_Pulay_DM.c`	+204/−2	OPT	GR-Pulay のバッチ Gram+融合混合掃引
`Eigen_lapack.c`	+143/−8	GPU	n≥1000 で gpusolver_Syevdx へ分岐、OpenACC 直結版を追加。呼び出し元 20+ ファイルが自動恩恵

ファイル	+n/−m	分類	変更概要
`openmx_common.h`	+142/−1	GPU/MISC	CUDA ヘッダ取込、GPUSOLVER enum、wait_cudafunc(エラーラッチ掃除つき)、GPU 系プロトタイプ、SetHamiltonianMETables 型、include ガード、Version "4.0 GPU"
`Lapack_LU_inverse.c`	+134/−4	GPU	cublasZgetrf/ZgetriBatched による LU_Zinverse_GPU を追加(呼び出しはコメントアウト=休眠)
`EigenBand_lapack.c`	+112/−3	GPU	複素版: gpusolver_zheevx 分岐、失敗時 Eigen_HH フォールバック
DMmu/CWF/LNO/分極系 11 ファイル(§7.13)	各 +27〜76	GPU	ELPA2 分岐手前への定型 GPUSOLVER 分岐挿入(dense_utils 経由 Dsyevx/Zheevx)
`Kerker_Mixing_Rhok.c`	+62/−10	OPT	Rhok 履歴シフトの高速化(回転+memmove)
`openmx_common.c`	+60/−0	GPU	GPU フェーズ需要レジストリ(OpenMX_GpuPhaseNeed_*)
`Set_Density_Grid.c`	+40/−5	GPU	分散 GPU 求積 → 単一ランク GPU サービス → CPU の 3 段ディスパッチ
`Free_Arrays.c`	+28/−17	GPU/FIX	終了時の GPU キャッシュ無効化+GEMMul8 ワークスペース解放+free 後 NULL 代入
`truncation.c`	+23/−13	FIX/GPU	VNA/VEF グリッドの free 後 NULL 代入+GPU キャッシュ無効化フック
`Runtest.c`	+19/−7	FIX	.dat 判定の厳密化、テスト前の旧 .out 削除
`openmx.c`	+16/−0	FIX	Raise_Stack_Limit()(スタック上限自動引き上げ)
`Input_std.c`	+15/−3	GPU	scf.eigen.lib への gpusolver 追加(cusolver は非推奨エイリアス)、scf.Gpu.Num 追加
`Spherical_Bessel.c`	+12/−24	OPT	ホットパスの毎呼び出し malloc/free を除去
`tran_prototypes.h`	+10/−0	GPU	CUDA_CHECK マクロ追加
`Set_Orbitals_Grid.c`	+8/−3	GPU	軌道グリッド再構築時の GPU キャッシュ無効化フック
`Stress.c`	+1/−1	OMP	OpenMP private リストに i を追加(ヒープ破壊修正)
`mpao.c`	+1/−0	MISC	include 1 行差のみ(ビルド対象外)
`makefile` → `Makefile`	全面改修	BLD	Intel oneAPI+MKL 構成から NVHPC+CUDA+自前 FFTW+GEMMul8 構成へ(第 9 章)。クリーンツリーから make 一発で完走

補足: 上表の当時の比較で削除されたソースは `my_cublasDgemm.c` / `my_cublasZgemm.c`(休眠ラッパ)の 2 件。その後、旧 `elpa-2018.05.001/` と旧ソルバラッパーも削除し、共通の ELPA 2026.02.002 ライブラリと `openmx_elpa_cpu.c` に置き換えた(§9.4)。第 2 版で「オリジナルと差分なし」だった `Set_ProExpn_VNA.c` と、混合 6 ファイル・密度グリッドは 本版までの 40 コミットで新たに変更された。

付録 B. 新規追加ファイル一覧(2026 年 7 月)

B.1 新規ソース(14 ファイル、計約 4,000 行)

ファイル	行数	役割
`Set_Density_Grid_GPU.c`	1,366	密度グリッドの GPU 化(単一ランク GPU サービス+分散 GPU 求積)
`Total_Energy_GPU.c`	1,064	全エネルギー計算の OpenACC カーネル集(EXC/EH1・双極子・CWF-Dc・EH0 バッチ・Exc0 細メッシュバッチ)。-acc 分離ビルドのため別 TU
`gpusolver_Syevdx.c`	516	cusolverDnXsyevdx による部分固有値分解ラッパ(実/複素 × ホスト/OpenACC/キャッシュ付きの 5 変種)。GPU 対角化の実体
`gemmul8_bridge.cu`	419	GEMMul8 呼び出しブリッジ: ワークスペースキャッシュ、サイズ照会 API、環境変数制御、cuBLAS フォールバック
`set_hamiltonian_gpu.cu`	194	Set_Hamiltonian 格子積分の共有メモリタイル型 CUDA カーネル(stale エラー耐性つき)
`gpusolver_Syevd.c`	131	cusolverDnXsyevd 版ラッパ(NVIDIA CUDA Library Samples 由来)。現在は未使用
`openmx_gpusolver_dense_utils.h`	79	密対角化の共通ディスパッチヘルパ(0/1-based 変換+失敗時 MPI_Abort)。11 ファイルが使用
`gemmul8_openmx.cu`	67	GEMMul8 テンプレートの単一 TU 明示的実体化(4 シンボルのみ)
`set_cuda_default_device_from_local_rank.c/.h`	73+9	ノード内 local rank による CUDA デバイスのラウンドロビン割当
`set_openacc_device_from_local_rank.c/.h`	79+12	同上の OpenACC 版(acc_set_device_num)
`LU_Zinverse_GPU.h`	9	GPU 版複素 LU 逆行列の宣言(休眠)

補足: 過去に存在した汎用 GEMM ラッパ `my_cublasDgemm.c` / `my_cublasZgemm.c` は削除された(§3.5)。第 2 版に記載した未結線カーネル集 `openmx_cuda_kernels.cu/.h` もツリーから撤去された(§6.4)。

B.2 ビルド・文書ファイル

ファイル	役割
`Makefile` (1,377 行)	新ビルドシステム本体。各種スパコン向け構成例のコメント付き。NVHPC 26.3 + CUDA 13.1 既定。クリーンツリーから make 一発(-j 可)で third_party 込みの全ビルドが完走(§9.3)
`Makefile.bunshiken` (1,366 行)	共用計算機向け変種(NVHPC 25.9-nompi + OpenMPI 5.0.8 + gcc-toolset-15)。一世代以上古い構成
`makefile.cpu_intel.bak` (1,383 行)	オリジナル makefile のバイト同一バックアップ(Intel CPU 構成)
`third_party/`	GEMMul8(submodule、コミット 6bf409b = v3.1.1 固定)、fftw3(submodule、タグ fftw-3.3.11)、fftw-3.3.11.tar.gz(codelets 補完用)。いずれも欠けていれば make が自動取得
`doc/`	本書(日本語/英語、HTML/PDF)

付録 C. 開発履歴(2026 年 7 月、91 コミット)

#	ハッシュ	題名(原文)
1	`0cf2acbc`	Initial commit
2	`60b6aa1e`	Performed GPU acceleration
3	`2f94ca06`	Performed GPU acceleration on Divide_Conquer_LNO.c
4	`e3fc656e`	Optimized Band_DFT_NonCol.c
5	`24e8b34f`	Optimized Cluster_DFT_Col.c
6	`33e9459b`	Optimized Cluster_DFT_Col.c
7	`82579d12`	Bug fix
8	`33a07450`	Fixed bugs in Band_DFT_Col.c and Cluster_DFT_Col.c
9	`b4221bba`	Fixed a bug in runtestL
10	`18d92700`	Bug fix
11	`563f69bd`	Add files that were not included in the commit
12	`1b020264`	Optimized Band_DFT_Col.c and Band_DFT_NonCol.c
13	`f4041307`	Optimized Set_Hamiltonian.c
14	`9f1633da`	Optimized Set_OLP_Kin.c
15	`d6b463a8`	Explicitly indicate that the generation of overlap matrices is GPU-accelerated
16	`5383544b`	Optimized Total_Energy.c
17	`c9e111ea`	Optimized Set_ProExpn_VNA.c
18	`8162b8ec`	Optimized Set_ProExpn_VNA.c and Total_Energy.c
19	`91416fb1`	Bug fix
20	`24a113a9`	Add files that were not included in the commit
21	`87ea7062`	Bug fix
22	`faf76fa2`	Bug fix
23	`a8224724`	Add .gitignore for build artifacts and runtime output
24	`d3c45cb3`	Optimized Bnad_DFT_NonCol.c
25	`db7b9327`	Add files that were not included in the commit
26	`a61f24b3`	Bug fix
27	`d6d19f99`	Add README.md
28	`a91f920f`	Bug fix
29	`88b135f4`	Optimized Krylov.c
30	`34cf8bc1`	Optimized Divide_Conquer_LNO.c
31	`685a2bc9`	Reverted Set_ProExpn_VNA.c to the CPU version
32	`2aa31a43`	Add files that were not included in the commit
33	`d3effce1`	Bug fix of Divide_Conquer.c
34	`34b52269`	GPU acceleration for paths where all_knum !=1 in Band_DFT_Col.c
35	`75ef67c7`	GPU acceleration for paths where all_knum !=1 in Band_DFT_NonCol.c
36	`1e0c777e`	Remove runtest.result from Git

#	ハッシュ	題名(原文)
37	`c8c1e3e1`	Modify the Makefile
38	`cb4d348d`	Now compatible with the latest GEMMul8
39	`d451eeb8`	Fixed OpenMP bugs in Total_Energy.c and Stress.c
40	`2302f965`	The name has been changed from CuSOLVER to GPUSOLVER
41	`d5b897fd`	Band_DFT_Col: serialize GPU turns in pairs and release GEMMul8 workspace per turn
42	`788acb4c`	Delete unnecessary files, such as the MAGMA library
43	`d8219384`	Rename the identifier from cusolver to gpusolver
44	`8647a5ed`	Set large2_example and Large3_example to be processed on the GPU.
45	`f508a2a6`	Enable direct GPUSOLVER dispatch for DC-LNO
46	`3545a9d5`	Edit README.md
47	`ca3d97c2`	Edit README.md
48	`43e30c65`	Limit band GPU solver concurrency adaptively
49	`66ce1e62`	Use GEMMul8 for remaining GPU GEMM paths
50	`a7f8cba6`	Remove obsolete cuBLAS wrapper files
51	`8c7801fa`	Limit band GPU k-turn concurrency
52	`64da9a3d`	Add Implementation Manual
53	`bff93d08`	Add single-rank GPU density grid service
54	`95fa51ed`	Accelerate Hamiltonian construction with adaptive GPU scheduling
55	`fe68cca6`	Accelerate force evaluation with fused contractions
56	`58ea6471`	Adapt band GPU k-turn concurrency to device memory
57	`7c414f91`	Adapt non-collinear band GPU concurrency to device memory
58	`75fc30d3`	Accumulate cluster density matrix on the GPU
59	`a9b6fa31`	Trim per-iteration GPU overhead in noncollinear cluster path
60	`d62cac60`	Run noncollinear DC-LNO cluster solves on the GPU
61	`f5de630e`	Cut per-iteration overhead in the noncollinear cluster GPU path
62	`527ed524`	Keep collinear cluster eigenvectors on the GPU for the DM step
63	`211f0684`	Batch the nonlocal-projector integrals on the GPU in Set_Nonlocal.c
64	`6b84a1f9`	Start the Set_Hamiltonian GPU wave at the full device-sharing rank count
65	`48921b36`	Start the Band_DFT_Col k-point wave at the full device-sharing rank count
66	`99f4014b`	Start the Band_DFT_NonCol k-point wave at the full device-sharing rank count
67	`697383e6`	Run the Force3 grid trace on the GPU
68	`7d38382d`	Batch the Force4B and Force_HNL traces on the GPU, move dOrbitals on device
69	`19b9b69c`	Run the Total_Energy device kernels in the NC case, batch the Exc0 mesh
70	`77d2a22c`	Batch the Set_ProExpn_VNA construction and traces on the GPU
71	`28361f7e`	Reuse fixed-size device buffers across Force3 trace chunks
72	`80a3a8f7`	Drop the one-rank GPU service cache diagnostic line
73	`b569f272`	Print the GPU-fallback notice once per node, not once per rank
74	`098adddc`	Fixed a bug in Force.c

#	ハッシュ	題名(原文)
75	`bff0af37`	Reduce Set_Hamiltonian GPU staging with on-device Vpot gather and a resident cache
76	`69dde552`	Make the Set_Hamiltonian resident cache yield to published GPU-phase needs
77	`30148e1e`	Label the GPU-accelerated VNA projector and force stages on stdout
78	`d14ef297`	Add a distributed GPU quadrature to Set_Density_Grid
79	`6f7f31b2`	Cut per-atom staging overhead in the GPU divide-conquer solver
80	`3ee45228`	Fuse the per-atom GPU pipeline in the Krylov O(N) solver
81	`ae3dd189`	Keep large cluster GPU runs from freezing or exhausting memory
82	`206b7e06`	Make the force-stage GPU batches immune to shared-device memory races
83	`cff323e6`	Fall back to ELPA when the cluster GPU diagonalization cannot reserve its buffers
84	`d0e1d17a`	Close the last fatal allocation windows in the force-stage GPU batches
85	`3ef5347a`	Fall back to ELPA when the non-collinear cluster GPU diagonalization cannot reserve its buffers
86	`a4200a04`	Fall back to ELPA when one k-point's band GPU diagonalization cannot fit
87	`94b6ca8c`	Accelerate RMM-DIISH mixing with a two-tier GPU/incremental fast path
88	`6e94a9d3`	Add fast paths to the remaining charge/DM mixing schemes
89	`c5eb6845`	Fix phantom Set_Hamiltonian abort caused by stale latched CUDA errors
90	`bf08a9d8`	Serialize the two-spin cluster GPU diagonalization when one device cannot hold both owners
91	`9fe8ce13`	Make a pristine tree build with a single make invocation

参考文献

1. OpenMX 公式サイト: `http://www.openmx-square.org/` (コード本体・マニュアル・入力キーワード)
2. GEMMul8(GEMMulate)リポジトリ: `https://github.com/RIKEN-RCCS/GEMMul8` (MIT ライセンス、責任開発者 Yuki Uchino, RIKEN R-CCS)
3. Y. Uchino, K. Ozaki, T. Imamura, "High-Performance and Power-Efficient Emulation of Matrix Multiplication using INT8 Matrix Engines", SC Workshops '25, doi:10.1145/3731599.3767539
4. K. Ozaki, Y. Uchino, T. Imamura, "Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique", arXiv:2504.08009
5. Y. Uchino, S. Ma, T. Imamura, K. Ozaki, T. Gutsche, "Emulation of Complex Matrix Multiplication based on the Chinese Remainder Theorem", arXiv:2512.08321(複素対応の理論)
6. Y. Uchino, K. Ozaki, T. Imamura, "Double-Precision Matrix Multiplication Emulation via Ozaki-II Scheme with FP8 Quantization", arXiv:2603.10634
7. H. Ootomo, K. Ozaki, R. Yokota, "DGEMM on integer matrix multiplication unit", Int. J. High Perform. Comput. Appl. (2024), doi:10.1177/10943420241239588 (ozIMMU / Ozaki Scheme I 系の先行研究)
8. H. Kawai ほか、GPU 版 OpenMX のベンチマーク: J. Phys. Soc. Jpn. 94, 124003 (2025), doi:10.7566/JPSJ.94.124003
9. NVIDIA cuSOLVER Documentation(cusolverDnXsyevd/Xsyevdx): `https://docs.nvidia.com/cuda/cusolver/`
10. NVIDIA cuBLAS Documentation: `https://docs.nvidia.com/cuda/cublas/`
11. NVIDIA HPC SDK Documentation(OpenACC): `https://docs.nvidia.com/hpc-sdk/`
12. ELPA(Eigenvalue soLvers for Petaflop Applications): `https://elpa.mpcdf.mpg.de/`
13. FFTW: `https://www.fftw.org/`

本書について: 第 4 版は `c0689442` (2026-10-17)を基準に CUDA/HIP 統合、CMake ビルド、ELPA 共通化、HIP の CPU 後処理修正を反映した。参照資料は `CMakeLists.txt`、`tools/prepare_build.py`、`docs/GPU_BACKENDS.md`、`source/backends/hip/` と、`bdcba87d`・`57a3a3fc` の検証記録である。CUDA の詳細コード例、ソース行番号、付録 A〜C は 7 月 23 日の調査(`9fe8ce13`、91 コミット)を基準とする。過去の測定値は現在の性能結果ではない。ソースコードを一次情報として優先されたい。